

## Содержание

|                                                                                                                                                                                                   |    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1. Две формы представления информации. Способы представления дискретной информации. Системы счисления, используемые в вычислительной технике: двоичная, 8-я, 10-я, 16-я и двоично-десятичная..... | 3  |
| 2. Представление чисел с фиксированной точкой. Прямой, обратный и дополнительный код. Формирование битовых признаков переноса, переполнения, отрицательного результата и нуля. ....               | 6  |
| 3. Представление символьных и строковых данных. Принципы построения кодовых таблиц ASCII, КОИ-8, ISO8859-8, Win-1251, UTF-8 и UTF-16.....                                                         | 9  |
| 4. Базовые элементы вычислительной техники: ячейки, регистры, шины, вентили, тактовые генераторы, логические схемы, триггеры, регистры, счётчики, сумматоры. ....                                 | 13 |
| 5. Структура и принцип функционирования ЭВМ. Порядок функционирования простого процессора на примере калькулятора. ....                                                                           | 17 |
| 6. Операционная система Unix – ядро ОС и файловая система.....                                                                                                                                    | 19 |
| 7. Операционная система Unix – интерпретаторы, стандартные потоки ввода вывода, фильтры.....                                                                                                      | 21 |
| 8. Операционная система Unix – основные команды, права файлов и способы их задания.....                                                                                                           | 23 |
| 9. Состав и структура БЭВМ. Адресные пространства БЭВМ. Система команд БЭВМ, форматы команд. Машинные циклы.....                                                                                  | 25 |
| 10. Организация вычислений в БЭВМ. Сдвиги, арифметические и логические операции. Цикл выборки команды. ....                                                                                       | 29 |
| 11. Организация массивов данных. Режим адресации. Цикл выборки адреса и операнда БЭВМ. ....                                                                                                       | 31 |
| 12. Управление вычислительными процессами в БЭВМ. Команды ветвлений, цикл исполнения команды LOOP.....                                                                                            | 34 |
| 13. Подпрограммы в БЭВМ. Цикл исполнения команд перехода и возврата из подпрограммы. Стек, передача параметров. Позиционно-независимый код. Загрузчик и библиотеки.....                           | 35 |
| 14. Организация ввода-вывода в вычислительных системах. Инициация обмена, передача информации и завершение обмена. Драйверы. ....                                                                 | 38 |
| 15. Организация ввода-вывода в БЭВМ. Устройства ввода-вывода, команды .....                                                                                                                       | 40 |
| 16. Организация асинхронного обмена в БЭВМ. Пример программы. Временные издержки асинхронного обмена. ....                                                                                        | 46 |
| 17. Организация прерываний в БЭВМ. Вектора прерываний, контроллер прерывания. ....                                                                                                                | 48 |
| 18. Организация обмена по прерыванию программы в БЭВМ. Пример программы. Цикл прерывания. ....                                                                                                    | 51 |
| 19. Понятие многоуровневой ЭВМ. Понятие и пример программы на разных уровнях. ....                                                                                                                | 53 |
| 20. Микропрограммный уровень БЭВМ. Структура МПУ. Форматы микрокоманд. ....                                                                                                                       | 56 |
| 21. Структура и принципы работы АЛУ и коммутатора. Регистр Состояния БЭВМ.....                                                                                                                    | 59 |
| 22. Микропрограммное управление вентильными схемами. Схема управления. Интерпретатор БЭВМ.....                                                                                                    | 62 |
| 23. Архитектура ЭВМ. Гарвардская и фон-Неймановская архитектура. Организация обмена архитектуры ЭВМ с использованием шин.....                                                                     | 65 |
| 24. Архитектура многопроцессорных ЭВМ. Системный коммутатор. Архитектуры UMA и NUMA. ....                                                                                                         | 67 |
| 25. Структура современных процессоров. Окружение процессора. CISC, RISK, VLIW.....                                                                                                                | 71 |

|                                                                                                                                                           |     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 26. Адресуемая память, организация и временные диаграммы. Конструктивные особенности современной памяти. ....                                             | 74  |
| 27. Память, ориентированная на записи (блочная память). Организация дисковой памяти и памяти на магнитных лентах. ....                                    | 77  |
| 28. Характеристики запоминающих устройств. Пирамида памяти. ....                                                                                          | 79  |
| 29. Ассоциативная память, Кэш-память. Влияние промахов кэш-памяти на производительность. ....                                                             | 80  |
| 30. Предназначение и организация виртуальной памяти. Сегментно-страничная организация. Устройство управление памятью (MMU), буфер трансляции (TLB). ....  | 82  |
| 31. Сетевые технологии. Понятие сети ЭВМ, классификация компьютерных сетей. Сообщение и пакет. Модель взаимодействия открытых систем. ....                | 86  |
| 32. Модель TCP/IP: передающая среда, канальный и сетевой уровень. Адресация, передача и маршрутизация пакетов. ....                                       | 89  |
| 33. Модель TCP/IP: выделение адресов (DHCP), сервисы имён, транспортный и прикладной уровни. ....                                                         | 93  |
| 34. Интерфейсы ввода-вывода. Контроллеры внешних устройств. Уровни стандартизации, сопряжения с системной шиной, циклы обмена. Регистры контроллера. .... | 96  |
| 35. Параллельная передача данных. Контроллеры параллельной передачи и приема. ....                                                                        | 99  |
| 36. Синхронные последовательные интерфейсы. Контроллеры последовательной передачи и приёма. ....                                                          | 101 |
| 37. Асинхронный обмен. Принцип деления частоты, формат кадра. ....                                                                                        | 104 |
| 38. Контроллер передачи асинхронного последовательного интерфейса. ....                                                                                   | 107 |
| 39. Контроллера приёма асинхронного последовательного интерфейса. ....                                                                                    | 109 |
| 40. Организация прямого доступа к памяти. Контроллер ПДП (DMA).....                                                                                       | 110 |

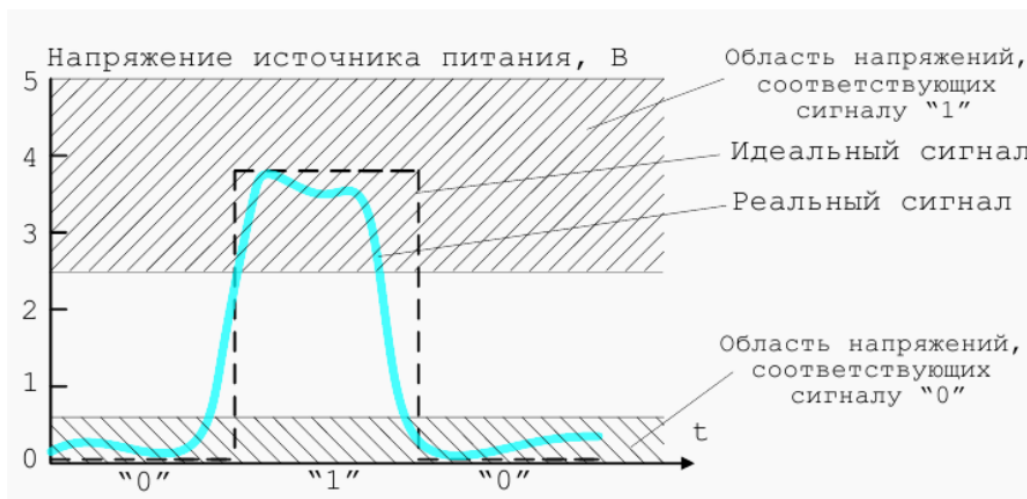
*1. Две формы представления информации. Способы представления дискретной информации. Системы счисления, используемые в вычислительной технике: двоичная, 8-я, 10-я, 16-я и двоично-десятичная.*

### Две формы представления информации

Первая форма представления информации считается аналоговой или непрерывной. Особенностью такого представления заключается в том, что функция копирует поведение физической величины, в аналог который мы и задаём информацию. Как следствие – в такой форме представления информация является непрерывной, даже если величина принимает значения на ограниченном диапазоне, и не имеет разрывов.

Вычислительные машины, работающие с аналоговыми данными называют аналоговыми. Такие машины эффективны для некоторого рода задач, например, для решения дифференциальных уравнений, поиска интеграла или производной функции. Подобные вычислительные устройства используются до сих пор в системах реального времени и в тех отраслях, где погрешность аналоговых ВМ не критична. Недостатком таких ВМ заключается в больших габаритах и сложности самих машин.

Вторая форма представления называется цифровой или дискретной. В современных машинах используется набор напряжений, который ставится в соответствии с одной из цифр представляемой цифровой величины. Подобные величины, в отличие от аналоговых, могут принимать лишь конечное число состояний и уже имеет разрывы. Но благодаря этому, ЭВМ становится проще в проектировании и реализации (заместо ухищрений в реализации физической залупы, например, как говорил Клименков про реализацию синуса в аналоговых ЭВМ через поворот катушки на определённый угол, то в ЭВМ уже всё основано на транзисторах, резисторах и тд).



Для передачи и хранения дискретной информации используются ряды сигналов, каждый из которых соответствует своей цифре рассматриваемой информации.

Из банальных примеров аналоговых величин можно считать винил как способ хранения музыки, так как дорожка на пластинке является непрерывной и как следствие сам звук по сути является непрерывным.

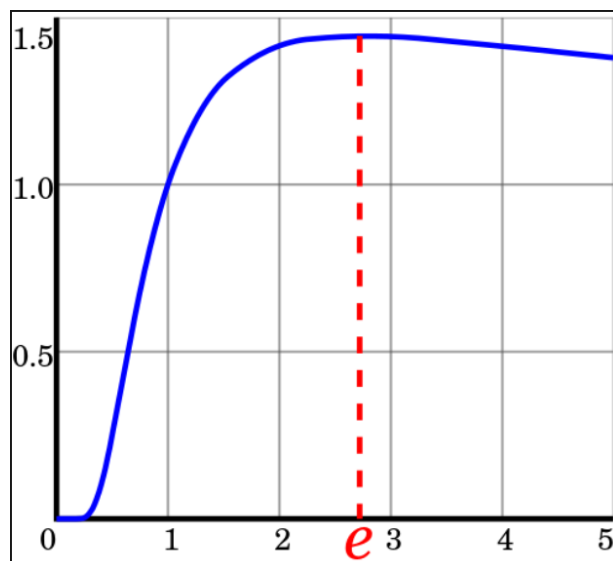
В контексте дискретных величин можно взять в пример аудиофайл формата mp3, где аудио представляет из себя огромное множество точек, аппроксимирующее исходный звуковой сигнал (привет частоте дискретизации).

### Способы представления дискретной информации и СС

Вообще сразу надо сказать, что во всех ЭВМ используются позиционные СС, то есть это такие СС, где значение числа зависит от позиции цифры. Вроде очевидно, но лучше написать на экзамене.

Как уже было сказано выше, в современных ЭВМ информация представляет из себя последовательность сигналов, где каждый из них соответствует определённой цифре дискретной величины. Казалось бы, для простоты человеческого восприятия можно использовать 10-ую СС, где последовательность сигналов уже по отдельности характеризовала количество единиц в числе, десятков, сотен и так далее. Но из этого вытекает проблема: устройство должно различать 10 различных состояний.

К тому же, исходя из функции объёма хранения на единицу данных при определённой СС, самая эффективная считается СС с основанием, равным числу  $e$ . Можно, конечно, использовать троичку, но это хуйня. Более помехоустойчивая считается всё же двоичная СС.



Удобная для компьютера двоичная СС неудобна для восприятия человеком. Поэтому для этого широко используются 8-ные и 16-ные СС, так как длинное двоичное число при делении на триады или тетрады сокращается в 3 или 4 раза соответственно в зависимости от выбранной СС. Объяснять как производится перевод, какие есть циферки и так далее вообще не имеет смысла.

Самое интересное – двоично-десятичная СС. Она используется в основном в тех цифровых устройствах, где основная часть операций завязана на вывод информации на индикаторы, а не её хранение. Как пример – калькулятор. В 2-10 СС цифры от 0 до 10 представляют из себя 4 двоичных цифр, которые принимают значения от 0000 до 1001. Две 10-ные цифры, то есть диапазон от 0 до 99, составляют уже целый байт.

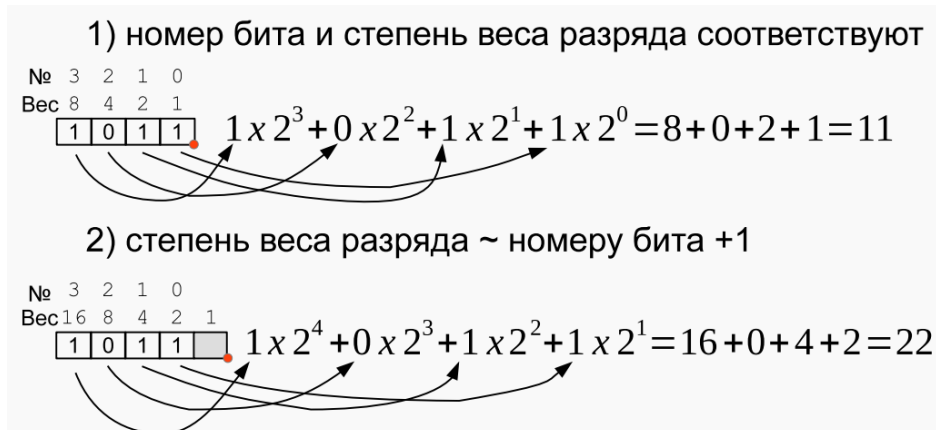
А теперь рассмотрим термины, использующиеся широко в вычислительной технике:

- Бит (Помним про бИты) – двоичная (и по совместительству минимальная) единица информации.
- Байт – восьмибитовая единица. Кстати, не всегда равно 8 битам, что забавно.
- Слово – это информация, которая хранится в одной ячейке памяти ЭВМ. Обычно в рамках одной системы ячейки и регистры имеют n-разрядов.

2. Представление чисел с фиксированной точкой. Прямой, обратный и дополнительный код. Формирование битовых признаков переноса, переполнения, отрицательного результата и нуля.

### Числа с фиксированной точкой

Числа с фиксированной точкой – это числа, где идёт чёткое разделение на целую и дробную часть. В ЭВМ она строго фиксирована, но никто не мешает менять вес каждого разряда (открываем лекции клименыча, часть 1):



### Беззнаковые числа

Целые двоичные числа без знака можно использовать для представления неотрицательных целых чисел. В качестве примера можно привести 16-разрядное число, которое может принимать значения от 0 до  $2^{16}-1$ . Такая запись называется прямым кодом числа. Она используется для беззнаковой арифметики для работы с логическими значениями.

### Знаковые числа

Целые двоичные числа со знаком используются тогда, когда нужно строго разделять неотрицательные и отрицательные числа. Одним из вариантов реализации “знака” является его отдельное хранение как незначащий старший разряд

- Нужно хранить признак знака числа достаточно 1-го бита, «0» значит «+», «1»=«-»
  - Прямое кодирование (прямой код числа)
 

|           |   |   |   |   |
|-----------|---|---|---|---|
|           | 3 | 2 | 1 | 0 |
| <b>+3</b> | 0 | 0 | 1 | 1 |

|           |   |   |   |   |
|-----------|---|---|---|---|
|           | 3 | 2 | 1 | 0 |
| <b>-3</b> | 1 | 0 | 1 | 1 |

$$-7 = -(2^3 - 1) \leq X \leq 2^3 - 1 = 7$$

Двойной ноль!

Но в ЭВМ используют дополнительный код, в котором по старшему биту числа можно определить его знак: 0 – число неотрицательно, 1 – числа отрицательно.

Дополнительный код неотрицательного числа совпадает с прямым кодом, а чтобы получить отрицательное число, нужно побитово инвертировать модуль числа (то бишь получить обратный код или первое дополнение числа) и далее инкрементировать (в иностранной литературе это ещё называют вторым дополнением). Другой способ получения дополнительного кода: воспользоваться следующей формулой:

$$b^n - x = -x$$

Где  $b$  – основание СС,  $n$  – разрядность числа.

Но конкретно в ЭВМ используется операция, описанная первая, так как легко реализуется с помощью логических элементов.

Благодаря использованию дополнительного кода, конструкция и реализация арифметических блоков ЭВМ серьезно упрощается, так как не требуется дополнительной обработки знака числа и можно использовать одну и ту же схему АЛУ и для положительных, и для отрицательных чисел.

### Признаки результата NZVC

C – (Carry Flag) – флаг переполнения, который используется в операциях с беззнаковыми числами. Флаг указывает на наличие выхода за разрядную сетку. В качестве примера приведём ситуацию для БЭВМ, где флаг реализован как выходной сигнал на последующий перенос у сумматора 15-го разряда:

$$\begin{array}{r} 1000\ 0000\ 0000\ 0000 \\ + 1000\ 0000\ 0000\ 0000 \\ \hline 1\ 0000\ 0000\ 0000\ 0000 \end{array}$$

Как мы видим, ожидаемый результат должен был получиться  $2^{17}$ , но из-за выхода за разрядную сетку, итоговый результат получится 0. Как раз для таких ситуаций обычно (Не упоминая циклические сдвиги) используется Керри флаг, чтобы сигнализировать разработчику о некорректной операции.

V – Overflow Flag – флаг переполнения, который используется в операциях с знаковыми числами. Если говорить совсем по-простому, то флаг возникает в ситуациях некорректной арифметики знаковых чисел, например, сумма двух отрицательных чисел дало неотрицательное (пример из Carry flag). В БЭВМ

данный сигнал формируется как исключающее или от переноса из 14 в 15 разряд и от переноса из 15 в 16 разряд (то бишь Carry flag, но не совсем).

Z (Zero flag) – флаг, который выставляется уже постфактум операции. Он выставляется, если в результате операции получили 0. В БЭВМ реализуется как логическое “или-не” от всех цифр числа.

N (Negative flag) – флаг, по аналогии с Zero flag, выставляется в самом конце и по сути своей дублирует старший бит числа, который по определению указывает на знак двоичного числа.

### 3. Представление символьных и строковых данных. Принципы построения кодовых таблиц ASCII, КОИ-8, ISO8859-8, Win-1251, UTF-8 и UTF-16

#### ASCII

ASCII (American Standard Code for Interchange Information) – одна из самых первых стандартизированных кодовых страниц. Кодировка является сама по себе 7-битной, где 7 бит является битом контроля чётности. Но рассмотрим extended ASCII, где 8 бит уже значащим:

| ASCII Code Chart |     |     |     |     |     |     |     |     |     |    |     |     |    |    |    |     |
|------------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|----|-----|-----|----|----|----|-----|
|                  | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9  | A   | B   | C  | D  | E  | F   |
| 0                | NUL | SOH | STX | ETX | EOT | ENQ | ACK | BEL | BS  | HT | LF  | VT  | FF | CR | SO | SI  |
| 1                | DLE | DC1 | DC2 | DC3 | DC4 | NAK | SYN | ETB | CAN | EM | SUB | ESC | FS | GS | RS | US  |
| 2                |     | !   | "   | #   | \$  | %   | &   | '   | (   | )  | *   | +   | ,  | -  | .  | /   |
| 3                | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9  | :   | ;   | <  | =  | >  | ?   |
| 4                | @   | A   | B   | C   | D   | E   | F   | G   | H   | I  | J   | K   | L  | M  | N  | O   |
| 5                | P   | Q   | R   | S   | T   | U   | V   | W   | X   | Y  | Z   | [   | \  | ]  | ^  | _   |
| 6                | `   | a   | b   | c   | d   | e   | f   | g   | h   | i  | j   | k   | l  | m  | n  | o   |
| 7                | p   | q   | r   | s   | t   | u   | v   | w   | x   | y  | z   | {   |    | }  | ~  | DEL |

Первые 32 позиции (00000000 - 00011111) занимают управляющие символы, которые использовались для передачи данных, для перемещения курсора при печати текста на экране, звуковых оповещений, разметка текста и так далее.

С 32 по 127 позиции (0010000 - 01111111) занимают строчные и прописные буквы латинского алфавита, десятичные цифры, знаки препинания, все виды скобок и другие служебные печатные символы, например, пробел.

С 128 по 255 позиции (10000000 - 11111111) занимают символы для нелатинского алфавита.

#### КОИ-8

Когда появилась рассмотренная выше Extended ASCII, то кириллические русские заглавные и строчные символы было решено расположить в верхней части таблицы. Причём символы расположены таким образом, чтобы они по начертанию были схожи с латинскими буквами. Это было сделано для того, чтобы при отбрасывании 7 бита (ПО может подумать, что бит является лишь битом контролем чётности) текст оставался ещё более-менее читаемым. Данная кодировка до сих пор используется в электронной почте

## ISO8859-8

8-битная кодировка из семейства кодовых страниц ISO8859 для представления кириллицы. Нижняя часть таблицы совпадает с символами из ASCII. В отличие от КОИ-8, русские символы расположены в алфавитном порядке, что упрощает операции сортировки строк. Но при этом свойство сохранения читаемости как у КОИ-8 пропало. Данная кодировка до сих пор используется в Болгарии и в Сербии на юникс-подобных ОС (И почему-то в ебучей джаве для хранения ресурсов).

Основной недостаток – это отсутствие полезных символов для организации текста, например, длинное тире, кавычки-ёлочки и так далее.

## Windows-1251

Несмотря на то, что в MSDOS использовалась кодировка CP866 для работы с русскими символами, в Windows до 10-й версии использовалась 8-битная кодировка Win-1251. Кодировка выигрывала у современников за счёт наличия практически всех символов, используемые в русской типографике.

Минус данной кодировки заключается в том, что строчная “я” имеет код 0xFF, что в других кодировках обычно является служебным символом. Ну и нет многих символов псевдографики, как в КОИ-8 и CP866

## UTF-8

Одна из наиболее используемых 8-битных и стандартизированных кодировок текста, которая позволяет кодировать символы Юникода. Кодировка имеет полную совместимость с 7-битной ASCII, так как символы оттуда отображаются как есть, а остальные символы будут выдавать как шум. Вследствие чего при преобладании латинских символов в тексте, UTF-8 будет выиграть у UTF-16. С другой стороны, с UTF-8 работать проблематично с точки зрения агрегации текста: подсчёт длины строки, сортировка и так далее.

### Принцип кодирования UTF-8

- 1) Если размерность символа в кодировке 8 бит (имеет вид 0aaa aaaa), то это просто код символа в ASCII.
- 2) Если слово больше 8 бит, то используется следующая упаковка:
  - а) Старшие биты хранят количество используемых байт символов, закодированных в единичной системе счисления
  - б) Каждая запись о длине символов оканчивается на 0.

в) В последующих байтах в начале пишется “10”, означая байт продолжения.

Пример:

0aaa aaaa – 1 байт

110a aaaa 10aa aaaa – 2 байта

1110 aaaa 10aa aaaa 10aa aaaa – 3 байта

1111 0aaa 10aa aaaa 10aa aaaa 10aa aaaa – 4 байта

1111 10aa 10aa aaaa 10aa aaaa 10aa aaaa 10aa aaaa – 5 байтов

UTF-16

Кодировка, которая имеет постоянную длину символов, за исключением так называемых суррогатных пар, где символы кодируются двумя 16-битными словам. Данная кодировка позволяет кодировать символы Unicode, но в общем случае делает это менее эффективно по экономии памяти, но выигрывает за счёт постоянной длины закодированного символа. Позволяет хранить значения от 0000 до FFFF.

Little/Big Endian и немного про NUL терминаторы

Little/Big Endian – это спор о том, в каком порядке делать пересылку байтов в память. Little endian подразумевает сначала пересылку, начиная с младшего байта (обратный порядок), а Big Endian соответственно наоборот, начиная со старшего байта. Оба метода не имеет особых преимуществ друг перед другом, но в современных ЭВМ чаще всего используют Little endian, но до сих пор возникают проблемы совместимости между процессорами Intel и Sparc



В современных машинах размещение строк организовано двумя разными способами. В Си-подобных языках используется NUL-терминатор в конце каждой строки, а саму строку называют NULL terminated String. В

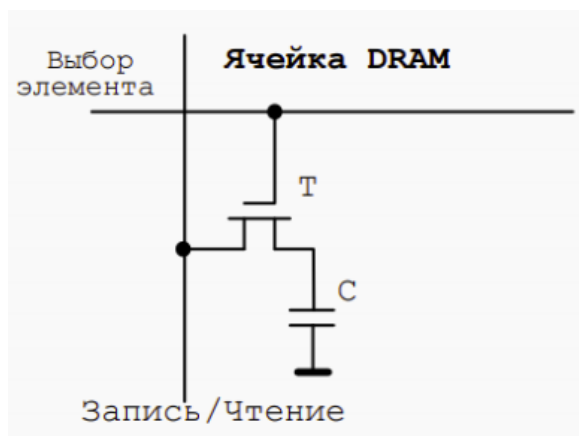
паскаль-подобных языках отдельным словом хранится длина строки. Первый вариант занимает меньше памяти, но при этом обработка строки (например, сплиттирование) будет сложнее, так как нужно добавлять ещё один NUL-терминатор. В этом плане второй вариант лучше, но занимает больше памяти.

4. Базовые элементы вычислительной техники: ячейки, регистры, шины, вентили, тактовые генераторы, логические схемы, триггеры, регистры, счётчики, сумматоры.

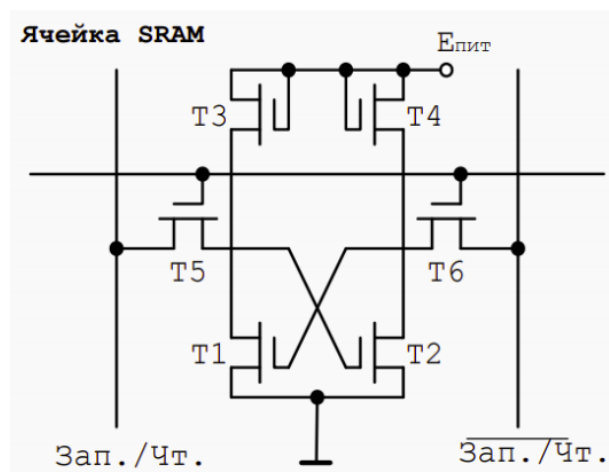
### Ячейка памяти, SRAM и DRAM

Ячейка памяти – минимальный адресуемый запоминающий элемент в вычислительной машине. Каждая ячейка памяти имеет свой адрес, по которому к ним могут обращаться функциональные элементы ЭВМ. Особенность ячейки памяти заключается в том, что она способна находиться в одном из двух устойчивых состояний: конденсатор разряжен или заряжен, транзистор находится в проводящем или в не проводящем состоянии. Одно из таких состояний создаёт высокий уровень напряжения, а другое – низкий. Обычно за высокий уровень напряжения берут “1”, а за низкий – “0”, но возможна противоположная реализация. Ячейка памяти может быть динамической, а может быть статической.

DRAM (Dynamic Random Access Memory) – динамическая память произвольного доступа. Схема такой памяти состоит из одного транзистора и одного конденсатора. Такая память требует постоянной регенерации, то есть ЭВМ в определённое количество времени (64 мс.) сканирует оперативную память, дабы данные с DRAM ячеек не пропали.



SRAM (Static Random Access Memory) – статическая память произвольного доступа. Схема данной памяти состоит из 6 транзисторов, которая поддерживает сама себя за счёт обратных цепей связи. Такая память будет явно дороже DRAM, при этом время доступа к такой памяти на порядок выше. Как пример, SRAM используется в кеше разного уровня.



Совокупность ячеек образует своего рода “прямоугольник”, состоящего из определённого количества строк и столбцов. Один такой “прямоугольник” называется страницей, а совокупность страниц – банками.

Регистры процессора – память, используемая внутри процессора для хранения часто используемых адресов, промежуточных вычислений и служебных данных, которые нужны для корректной работы ЭВМ. Регистр характеризуется количеством битов, которые могут храниться в данной памяти. Операция чтения данных из регистра по сути своей копирует содержимое регистра, то бишь информация в регистре после данной операции останется прежним.

Шина – совокупность проводов, которые нужно для соединения регистров и иных функциональных элементов внутри ЭВМ. Каждый провод в шине отвечает за передачу определённого разряда регистра. Обычно шина, помимо информационных каналов, снабжается дополнительными служебными каналами, например, для передачи сигнала синхронизации и так далее. Кроме того, шина чаще всего является однонаправленной, но в специальных схемах используются и двунаправленные варианты.

Вентильные схемы – это схемы, которые управляют передачей сигналов по шинам между функциональными элементами ЭВМ. Каждый вентиль имеет два входа и один выход. На информационный вход подаётся бит информации, который мы хотим передать на выход вентиля для дальнейшей передачи другим регистрам. Управляющий вход как раз регулирует, пройдёт ли информационный бит или нет. В самом простом варианте схема эквивалента логическому “И”. Обычно в многоразрядных системах вентиль имеет  $n+1$  входов, где  $n$  входов являются информационными.

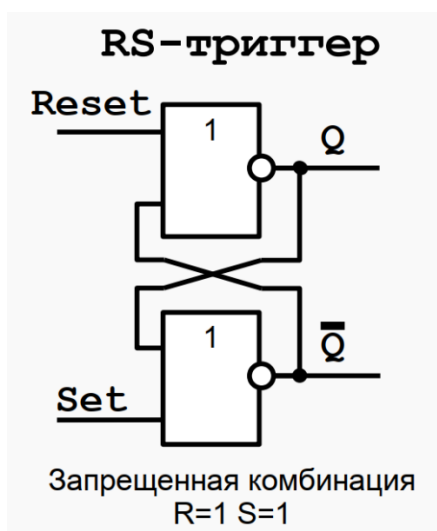
Тактовый генератор – сердце любой машины, которое генерирует электрические импульсы с чётким и строгим интервалом. Обычно импульс

имеет вид прямоугольника. Используется для синхронизации процессор между функциональными элементами ЭВМ.

### Триггеры

Триггер – класс электронных устройств, которые обладают способностью хранить одно из двух устойчивых состояний и которые под воздействием внешних сигналов способны менять это состояние. Главная особенность триггера заключается в том, что он способен хранить двоичную информацию.

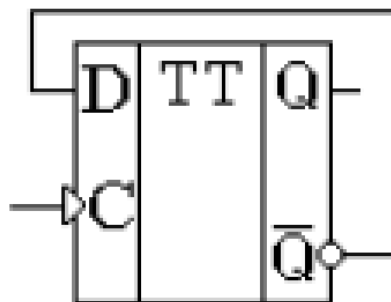
RS (Reset и Set) триггер – вид триггера с двумя управляющими входами, которые обозначаются как R и S. Управляющий сигнал S позволяет установить “1” в значение выхода Q. Управляющий сигнал R же сбрасывает значение выхода Q на 0:



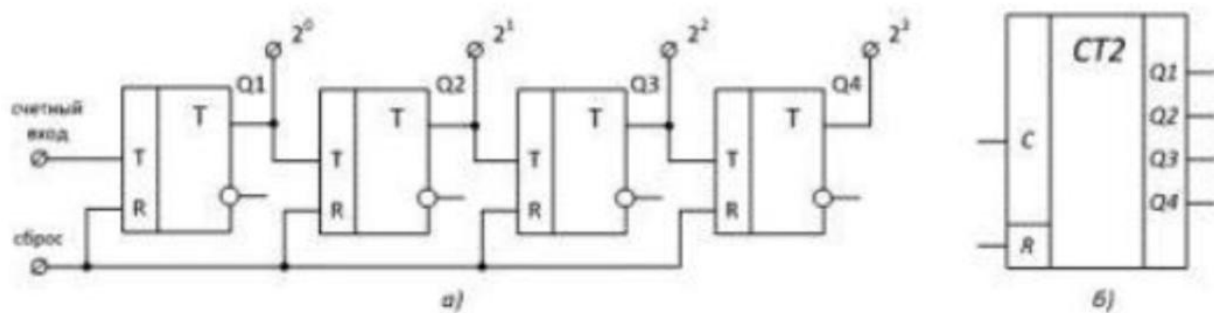
Синхронный RS триггер – подвид RS-триггера, который помимо входов SR имеет и вход C как сигнал синхронизации. В отличие от RS-триггера, синхронный RS-триггер позволяет работать с памятью в определённый такт времени.

D-триггер – триггер, который позволяет хранить один бит информации, то есть либо единичку, либо ноль. Для управляющих сигналов такого триггер достаточно использовать один провод, так как одновременных сигналов на оба входа поступить не может. Объединить входы можно при помощи схемы с инвертором.

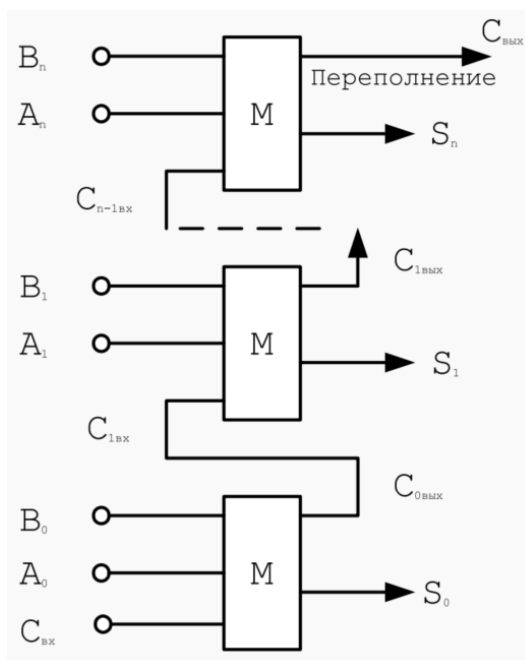
T-триггер – это счётный триггер, который имеет всего один вход. При поступлении сигнала на вход, состояние триггера меняется на противоположный.



Счётчик числа импульсов – устройство, которое подразумевает из себя последовательно соединённые Т триггеры с дополнительным входом для сброса состояния. Позволяет хранить количество поступивших импульсов, на выходе получается двоичный или двоично-десятичный код.



Сумматор – устройство, которое преобразует исходные информационные сигналы и на выходе выдаёт сумму данных сигналов:



## *5. Структура и принцип функционирования ЭВМ. Порядок функционирования простого процессора на примере калькулятора.*

### Структура ЭВМ

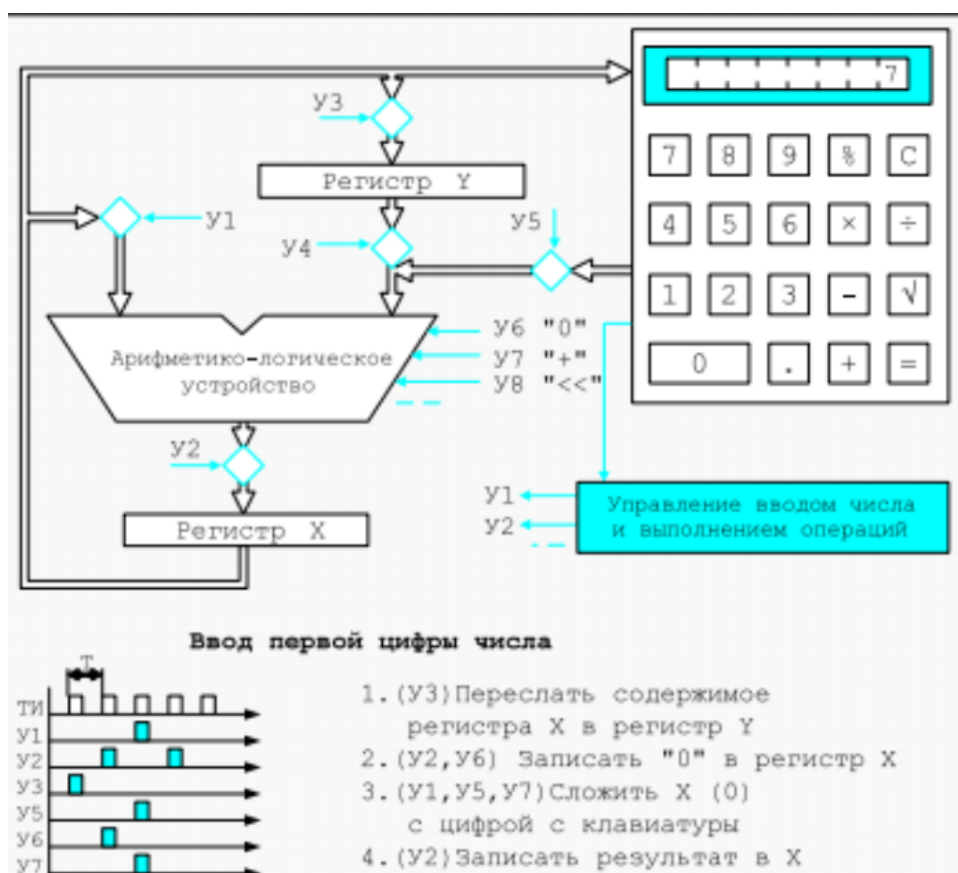
Любая ЭВМ состоит из 3-х основных частей: устройство ввода-вывода, процессор и память.

Все вычисления и обработка данных происходит внутри процессора. В свою очередь, процессор состоит из регистров, о которых было сказано в билете выше, АЛУ, которое воспроизводит все логические и арифметические операции, МПУ, которое выполняет микрокод для реализации основных команд ЭВМ.

Устройства ввода-вывода позволяют в удобном для человека формате вводить данные в ЭВМ, а также получать результаты выполнения программ в читаемом виде.

Память же нужна для хранения самих программ и входных для неё данных. Каждая ячейка памяти представляет из себя совокупность единиц и нулей. Процессор при помощи адресных команд может обращаться к конкретной ячейке памяти, зная её точный адрес.

### Порядок функционирования на примере калькулятора



Рассмотрим принцип функционирования ЭВМ на примере калькулятора. Калькулятор состоит из двух регистров: X и Y, который используются для хранения промежуточных вычислений. Есть устройство ввода-вывода – кнопки и экран калькулятора соответственно. Экран калькулятора дублирует содержимое регистра X. Есть Устройство управление, которое отправляет последовательность сигналов для реализации конкретной функции. Подобную последовательность сигналов называют циклом импульсов, которые управляют вентилями. Каждая группа импульсов выдаётся в определённый такт тактового генератора. Есть АЛУ с специальными управляющими входами, определяющими выбранную операцию.

Рассмотрим команду ввода операнда в калькулятор. Для начала нужно сохранить предыдущее состояние памяти калькулятора, то есть перебросить значение регистра X в регистр Y. Для ввода нового числа нужно воспроизвести операция "0 + вводимая цифра". Сначала обнулим регистр X, записав в него 0 со стороны АЛУ. Позже значение обновлённого регистра X подаётся на левый вход, а вводимая цифра на правый вход. Числа суммируются и результат записывается в регистр X.

## *6. Операционная система Unix – ядро ОС и файловая система.*

Unix – семейство переносимых, многозадачных и многопользовательских операционных систем. Название произошло от акронима UNICS (Uniplex Information and Computer System) и изначально была написана на ассемблере как однопользовательская ОС для PDP-7. Позже ОС была переименована в UNIX и была портирована на PDP-11, переписанная на язык C. В конце 70-х стали появляться первые ответвления данного семейства операционных систем.

Основное отличие Unix-подобных систем от остальных ОС в том, что это изначально многопользовательская и многозадачная система, то есть в один и тот же момент времени к одной системе могут подключиться сразу несколько пользователей, которые могут выполнять большее количество операций. Даже сама популярная ОС – Microsoft Windows, не считая некоторых серверных версий, является однопользовательской ОС. Плюс ко всему, UNIX подобные системы являются более легковесными для коммерческих машин операционной системой, чем Windows.

### Ядро \*NIX

Второй плюс системы – это мультиплатформенность. Ядро системы написано таким образом, чтобы его можно было легко адаптировать практически под любое железо. Поговорим конкретно про ядро ОС

Каждая операционная система выполняет служебные операции внутри ядра ОС. Ядро представляет из себя набор подсистем для адекватного функционирования программ на компьютере. Например, в ядре располагаются подсистемы управления оперативной памятью, сетевая подсистема, подсистема управления процессами и потоками, драйверы для взаимодействия с внешними устройствами, подсистема виртуальной памяти (В Unix всё есть файл) и платформо-независимый код. Любое взаимодействия пользователя или программы с ядром происходит через интерфейс системных вызовов.

Любое адресное пространство исполняемой программы логически делится на отдельные сегменты, например, сегмент стека или сегмент входных данных программы.

### Файловая система

ФС представляется в виде дерева. Верхним элементом иерархии такого дерева называют директорию root, Логические разделы дискового накопителя

могут подключаться в файловую систему в специальных точках монтирования, которые создаёт сам администратор. Каждая директория содержит две обязательные ссылки: “.” – ссылка на саму себя, “..” – ссылка на директорию выше по иерархии дерева. В файловой системе есть каталоги, которыми пользуется сама операционная система, например, /usr/bin – директория, в которой располагаются исполняемые файлы и утилиты.

В файловой системе можно адресоваться при помощи абсолютной адресации, указывая полный путь от корня ФС, либо при помощи относительной, указывая путь от текущей директории.

Помимо обычных файлов (А всё, что есть в UNIX, это файл), существуют файлы, которые помогают пользователю организовать иерархию файлов в ФС. К ним относятся жёсткие и символические ссылки. Каждый объект в файловой системе имеет inode – уникальный номер элемент внутри ФС.

Жёсткая ссылка на файл подразумевает, что она имеет тот же inode (как следствие и содержимое), что и файл, на который мы ссылаемся. Содержимое ссылки будет определено до тех пор, пока не будут удалены все объекты с таким inode. Но такая ссылка не может быть создана в другой точке монтирования, так как там будет иной набор inode

Символическая ссылка ссылается не на inode, на конкретный файл. Вследствие чего, символические ссылки могут располагаться в другой ФС и иметь другой inode, но при удалении искомого файла, содержимое символической ссылки станет недоступным.

## *7. Операционная система Unix – интерпретаторы, стандартные потоки ввода вывода, фильтры.*

### Интерпретатор команд

Всеми командами управляет интерпретатор команд `shell`. Эта программная оболочка позволяет организовывать выполнение команд, задавать им переменные окружения, анализировать коды возврата команд, организовывать ввод-вывод последовательно между несколькими командами и использовать простейший язык для написания исполняемых программ – скриптов.

`SH`, `BASH` и `KSH` – разные типы оболочек `shell`, которые предоставляют интерфейс для взаимодействия пользователя с командной строкой ОС.

`SH` – одна из самых первых оболочек, появившихся в `UNIX`. Имеет базовый синтаксис и простейшие операторы, например, циклы и операторы ветвления. По сравнению со старыми командными оболочками, `sh` имеет крайне скудный функционал, но его до сих пор используют для написания скриптов из-за высокой совместимости с остальными `shell`-оболочками

`BASH` – это расширенная версия `SH`, которая является стандартной во многих дистрибутивах `Linux`. Из нововведений можно отметить появление автодополнения команд, расширение функционала работы с циклами и ветвлениями, история команд и поддержка работы массивов и функций. `BASH` обратно совместим с `SH`, поэтому большинство скриптов, написанных на `SH`, будут работать в `BASH`.

`KSH` – командная оболочка, которая обычно используется при большом количестве пользователей в одной ОС.

### Потоки ввода-вывода

Файловый дескриптор – это целое число, которое ОС используется для идентификации открытого файла или ресурса. Оно выступает как ссылка на структуру данных, где хранится информация о текущем состоянии взаимодействия с файлами. Дескрипторы уникальны только внутри одного процесса. В каждом процессе `UNIX` автоматически создаёт три стандартных потоков:

- 0 – Стандартный ввод (`stdin`);
- 1 – Стандартный вывод (`stdout`);
- 2 – Стандартный вывод ошибок (`stderr`);

## Перенаправление потоков ввода-вывода

“>” – перенаправление выходного потока в файл, причём содержимое файла, если он непустой, будет перезаписан;

“>>” – перенаправление выходного потока в файл, который добавляет данные в конец файла;

“<” – перенаправление входного потока из файла;

“<<” – here-document, позволяющий передать текст напрямую, используя маркер EOF, который пишется в начале и в конце;

“<<<” – here-document, но данные записываются в одну строку (EOF не требуется);

“&>file или >&file” – направить стандартный поток вывода и стандартный поток ошибок в файл;

“|” – пайп или конвейер, позволяющий вывод одной команды передать на ввод следующей команды;

“> /dev/null” – перенаправление вывода в “чёрную дыру”;

## Фильтры

Это команды, которые на вход получают данные, как-то их обрабатывают или агрегируют и результат операции выводится в стандартный поток вывода. Пример команд: cat, grep, sort, tail, head, wc.

## 8. *Операционная система Unix – основные команды, права файлов и способы их задания.*

### Основные команды

- “mkdir” – создание пустой директории;
- “echo” – вывод символов в стандартный поток вывода;
- “cat” – конкатенация и вывод содержимого файла в стандартный поток вывода;
- “touch” – создание пустого файла или обновление последней даты модификации;
- “ls” – вывод списка файла в стандартный поток вывода;
- “pwd” – вывод текущей директории в стандартный поток вывода;
- “cd” – смена текущей директории на указанную;
- “rm” – удаление файлов или директории. Флаг “-r” рекурсивно удаляет содержимое директории;
- “rmdir” – удаление пустой директории;
- “mv” – перемещение файлов;
- “head” – вывод первых указанных строк;
- “tail” – вывод последних указанных строк;
- “sort” – сортировка строк файлов или стандартного ввода;
- “grep” – выборка строк по шаблону;
- “wc” – подсчёт количества строк, слов и байт в файле;
- “ln” – создаёт жёсткую ссылку. Флаг “-s” позволяет создать символическую ссылку;
- “man” – выводит справку по командам;

### Способ задачи прав пользователей

Команда для выдачи прав: “chmod”. Для каждого файла существуют следующие категории пользователей:

- U (user) – владелец файла;
- G (group) – группа, владеющая файлом;

- O (other) – остальные пользователи;

Каждая из категории может иметь любую комбинацию из следующих файлов:

- R (read) – право на чтение файла, просмотр каталога;
- W (write) – право на запись в файл, добавление или удаление каталога;
- X (execute) – право на исполнение файла, поиск и переход в каталог;

Права принято обозначать в виде трёх восьмиричных чисел:

- $100_2 = 4$  – чтение
- $010_2 = 2$  – запись
- $001_2 = 1$  – исполнение

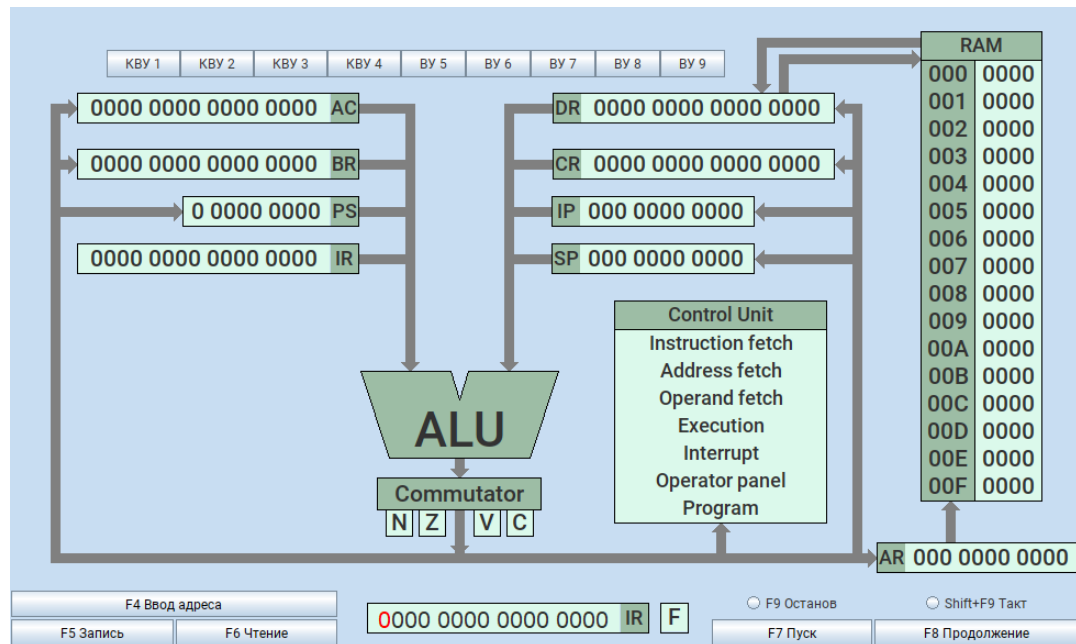
Пример использования: “chmod 644 file.txt”

Другой вариант: использовать формат ugo с помощью =,+,-:

Пример: “chmod ugo=rwx file.txt”, “chmod u+w,g-r,o=rwx file.txt”;

“chmod u=g file.txt” – копирование прав одной категории другой категории пользователей.

9. Состав и структура БЭВМ. Адресные пространства БЭВМ. Система команд БЭВМ, форматы команд. Машинные циклы.



Базовая ЭВМ – это простая гипотетическая ЭВМ аккумуляторного типа, обладающая характерными чертами, присущими многим ЭВМ. Память состоит из 2048 16-ти разрядных ячеек. Ячейки памяти с 000 – 00F выделяются под вектора прерывания. Процессор состоит из ряда регистров, АЛУ и МПУ:

- AC – 16-ти разрядный аккумулятор, используется как регистр свободного назначения;
- BR – 16-ти разрядный буферный регистр, используется для промежуточных вычислений;
- PS – 9-ти разрядный регистр состояния, нужный для корректной работы БЭВМ. Содержит следующие значения:
  1. C – Carry flag;
  2. V – Overflow flag;
  3. Z – Zero flag;
  4. N – Negative flag;
  5. 0 – для организации безусловного перехода;
  6. EI – разрешение на прерывание;

7. IRQ – разрешение на прерывание. Значение бита определяется как логическое “И” между EI и сигнала с шины запроса на прерывание;
  8. W – состояние тумблера “останов/работа” (1 – работа);
  9. P – программа;
- IR – 16-ти разрядный клавишный регистр.
  - DR – если адресный регистр, то будет хранить один из операндов операции, если безадресная команда, то будет храниться сама команда. Является 16-ти разрядным
  - CR – 16-ти разрядный регистр, хранящий полный код команды.
  - IP – 11-ти разрядный счётчик команд, указывает на адрес следующей исполняемой команды;
  - SP – 11-разрядный счётчик, указывающий на вершину стека
  - АЛУ – арифметическо-логическое устройство, производящее все вычислительные операции внутри процессора.
  - МПУ – микропрограммное устройство. Выполняет машинные коды конкретной команды, используя интерпретатор БЭВМ.

### Команды и машинные циклы

ЭВМ способна выполнять только определённый перечень команд, что должен учитывать программист. В зависимости от того, к каким блокам памяти БЭВМ обращается, выделяют четыре группы команд: адресные команды, безадресные команды, команды ветвления и команды ввода-вывода.

### Адресные команды

Адресная команда подразумевает взаимодействие процессора с памятью БЭВМ. Адресные команды имеют следующий вид:

... с прямой абсолютной адресацией

|     |    |    |    |    |       |   |   |   |   |   |   |   |   |   |   |
|-----|----|----|----|----|-------|---|---|---|---|---|---|---|---|---|---|
| 15  | 14 | 13 | 12 | 11 | 10    | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| КОП |    |    |    | 0  | Адрес |   |   |   |   |   |   |   |   |   |   |

... с относительной адресацией

|     |    |    |    |    |       |   |   |          |   |   |   |   |   |   |   |
|-----|----|----|----|----|-------|---|---|----------|---|---|---|---|---|---|---|
| 15  | 14 | 13 | 12 | 11 | 10    | 9 | 8 | 7        | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| КОП |    |    |    | 1  | Режим |   |   | Смещение |   |   |   |   |   |   |   |

... с непосредственной загрузкой операнда

|     |    |    |    |    |    |   |   |       |   |   |   |   |   |   |   |
|-----|----|----|----|----|----|---|---|-------|---|---|---|---|---|---|---|
| 15  | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7     | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| КОП |    |    |    | 1  | 1  | 1 | 1 | Число |   |   |   |   |   |   |   |

### Остальные команды

Безадресные команды подразумевают взаимодействие только с регистрами внутри процессора без обращения к ОЗУ. Например, инкрементирование аккумулятора, расширение знака аккумулятора и так далее. Команды ветвления нужны для организации условных операторов внутри БЭВМ. Команды ввода-вывода позволяют работать с внешними устройствами:

Безадресная команда

|    |    |    |    |                |    |   |   |   |   |   |   |   |   |   |   |
|----|----|----|----|----------------|----|---|---|---|---|---|---|---|---|---|---|
| 15 | 14 | 13 | 12 | 11             | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 0  | 0  | 0  | 0  | Расширение КОП |    |   |   |   |   |   |   |   |   |   |   |

Команда ввода-вывода

|    |    |    |    |        |    |   |   |            |   |   |   |   |   |   |   |
|----|----|----|----|--------|----|---|---|------------|---|---|---|---|---|---|---|
| 15 | 14 | 13 | 12 | 11     | 10 | 9 | 8 | 7          | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 0  | 0  | 0  | 1  | Приказ |    |   |   | Устройство |   |   |   |   |   |   |   |

Команда ветвления

|    |    |    |    |           |    |   |   |          |   |   |   |   |   |   |   |
|----|----|----|----|-----------|----|---|---|----------|---|---|---|---|---|---|---|
| 15 | 14 | 13 | 12 | 11        | 10 | 9 | 8 | 7        | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 1  | 1  | 1  | 1  | Расш. КОП |    |   |   | Смещение |   |   |   |   |   |   |   |

### Машинные циклы

Любая команда состоит из микрокоманд, которые, в частности, открывают определённые вентили для выполнения определённых операций, например, чтение и запись определённого регистра. Все микрокоманды хранятся в МПУ, набор из которых могут выполнять определённые машинные циклы.

Можно выделить следующие машинные циклы



## *10. Организация вычислений в БЭВМ. Сдвиги, арифметические и логические операции. Цикл выборки команды.*

Целые двоичные числа без знака можно использовать для представления нуля и целых положительных чисел. В 16-ти разрядном слове они могут меняться от 0 до 65535. Такие числа называют числами с фиксированной запятой. Целые двоичные числа со знаком используются в случае, когда нужно чётко разделять отрицательные и неотрицательные числа. В БЭВМ числа со знаком записываются в дополнительном коде, что упростило конструкцию всей машины.

На нативном уровне АЛУ в БЭВМ поддерживает всего несколько операций: арифметическое сложение (команда ADD, код 4XXX), логическое побитовое “И” (команда AND, код 2XXX) и инверсия (команда NOT, код 0280). Вычитание SUB (код 6XXX) реализуется через сложение первого операнда и второго операнда, записанного в дополнительном коде со знаком. INC (код 0700) и DEC (код 0740) позволяют инкрементировать и декрементировать содержимое аккумулятора. Остальные команды уже являются производными: команда NEG (0780) позволяет поменять знак у числа, логическое “ИЛИ” (код 3XXX) производит побитовое “или” между двумя операндами. Вообще в целом логические операции полезны при маскировании чисел, например, при помощи логического “И” можно отбросить ненужные биты числа или определить делимость числа. Команда ADC (код 5XXX) производит сложение с учётом переноса Carry flag.

В БЭВМ реализованы флаги признаков результата: N, Z, V и C. Тут всё очевидно, что писать, тем более в одном из билетов это было.

Операции умножения и деления на железном уровне не реализованы, поэтому их можно получить только программным путём. Однако можно воспользоваться операциями арифметического сдвига ASL (код 0500) и ASR (код 0580), где биты смещаются на разряд старше (значение умножается на 2), либо биты смещаются на разряд младше (значение делится на 2).

Команды ROL (код 0400) и ROR (код 0480) организуют циклический сдвиг влево или вправо соответственно, образуя некое кольцо, по которому перемещаются биты. Перенос из младшего разряда в старший и наоборот реализовано при помощи Carry флага.

Цикл выборки команды.

- 1)  $IP \rightarrow BR, AR$
- 2)  $BR + 1 \rightarrow IP; MEM(AR) \rightarrow DR$
- 3)  $DR \rightarrow CR$
- 4) Определение типа команды, адресации (возможно операнда)
- 5) Выполнение самой команды

## 11. Организация массивов данных. Режим адресации. Цикл выборки адреса и операнда БЭВМ.

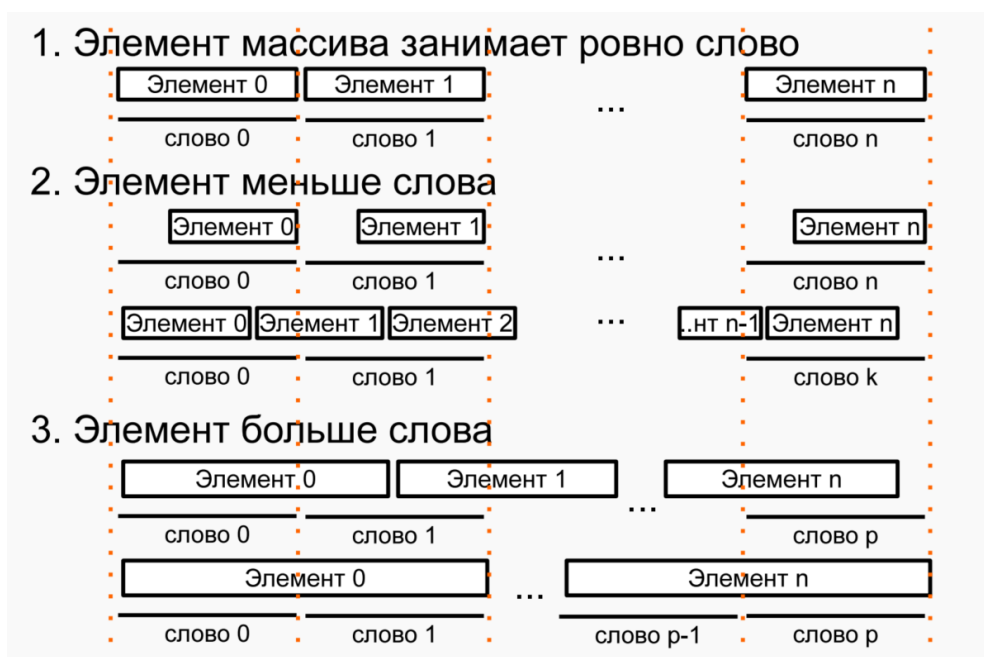
### Организация массивов данных. Реализация в БЭВМ

Массив данных представляет из себя множество элементов, идущие последовательно в ячейках памяти. Идеальная ситуация, когда элемент по длине совпадает со словом ЭВМ.

Но есть ситуации, когда элемент не совпадает по длине со словом. Здесь есть два варианта: либо мы в каждое слово (или в кратное число слов) помещаем по элементу, либо располагаем их по порядку в плотной упаковке.

Первый вариант удобен для визуального анализа массива и для работы с ним, однако из-за него появляется разреженность памяти, да и нужно делать расширение знака, если массив состоит из чисел.

Второй вариант решает вопрос с появившимися пустотами, но здесь нужно наверняка знать длину исходных данных, да и плюсом ко всему с плотной упаковкой работать сложнее.



В БЭВМ в каждом слове можно хранить максимум коды 2-х символов, записанных либо в Little-endian, либо в Big-endian (см. 3 билет). Для реализации массива в БЭВМ есть два конкретных пути, но в обоих из них мы задаём отдельную переменную индексирования, которая хранит адрес текущего рассматриваемого элемента:

1. Помимо самого массива в самом его конце располагается стоп-символ (Ну или же NUL-terminator), после которого программа, обрабатывающая массив, поймёт, что массив (строк) закончился.

2. Помимо самого массива в отдельной ячейке хранится длина массива. При помощи команды LOOP, программа обработает конечное количество символов, располагающихся в массиве. Взаимодействие с массивом прекратится в тот момент, когда значение ячейки длины массива обнулится.

#### Режимы адресаций в БЭВМ. Цикл выборки адреса и операнда

| Код | Мнемоника | Название                   | Потактово (AF и OF)                                                                                                                                                                           |
|-----|-----------|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x0 | ADD \$L   | Прямая абсолютная          | <i>Operand Fetch:</i><br>$DR \rightarrow AR$<br>$MEM(AR) \rightarrow DR$                                                                                                                      |
| 0xE | ADD L     | Прямая относительная       | <i>Address Fetch:</i><br>$SXTB(CR(0...7)) \rightarrow BR$<br>$BR + IP \rightarrow DR$<br><i>Operand Fetch:</i><br>$DR \rightarrow AR$<br>$MEM(AR) \rightarrow DR$                             |
| 0x8 | ADD (L)   | Косвенная относительная    | <i>Address Fetch:</i><br>$SXTB(CR(0...7)) \rightarrow BR$<br>$BR + IP \rightarrow AR$<br>$MEM(AR) \rightarrow DR$<br><i>Operand Fetch:</i><br>$DR \rightarrow AR$<br>$MEM(AR) \rightarrow DR$ |
| 0xA | ADD (L)+  | Косвенная автоинкрементная | <i>Address Fetch:</i><br>$SXTB(CR(0...7)) \rightarrow BR$<br>$BR + IP \rightarrow AR$<br>$MEM(AR) \rightarrow DR$<br>$DR + 1 \rightarrow DR$<br>$MEM(AR) \rightarrow DR$                      |

|     |          |                              |                                                                                                                                                                                                                                                      |
|-----|----------|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     |          |                              | $DR - 1 \rightarrow DR$<br><i>Operand Fetch:</i><br>$DR \rightarrow AR$<br>$MEM(AR) \rightarrow DR$                                                                                                                                                  |
| 0xB | ADD -(L) | Косвенная автодекрементная   | <i>Address Fetch:</i><br>$SXTB(CR(0...7)) \rightarrow BR$<br>$BR + IP \rightarrow AR$<br>$MEM(AR) \rightarrow DR$<br>$DR - 1 \rightarrow DR$<br>$MEM(AR) \rightarrow DR$<br><i>Operand Fetch:</i><br>$DR \rightarrow AR$<br>$MEM(AR) \rightarrow DR$ |
| 0xC | ADD &L   | Со смещением относительно SP | <i>Address Fetch:</i><br>$SXTB(CR(0...7)) \rightarrow BR$<br>$BR + SP \rightarrow DR$<br><i>Operand Fetch:</i><br>$DR \rightarrow AR$<br>$MEM(AR) \rightarrow DR$                                                                                    |
| 0xF | ADD #L   | С прямой загрузкой операнда  | $SXTB(CR(0...7)) \rightarrow BR$<br>$BR \rightarrow DR$                                                                                                                                                                                              |

Цикл выборки операнда

$DR \rightarrow AR$

$MEM(AR) \rightarrow DR$

## 12. Управление вычислительными процессами в БЭВМ. Команды ветвлений, цикл исполнения команды LOOP.

Управление вычислительными процессами в БЭВМ организовано при помощи операторов ветвления, циклов и стоп-командой HLT.

Операторы ветвления в БЭВМ реализованы при помощи так называемых команд перехода, которые, основываясь на флагах NZVC, делают переход в памяти машины. Это достигается тем, что сумма смещения из команды и самого IP записывается в сам IP. Обычно команды перехода используют вместе с командой сравнения CMP (код 7XXX), которая выставляет флаги NZVC на основе операции вычитания из аккумулятора значения сравниваемого объекта.

| Наименование                          | Мнемон.         | Код  | Описание                   |
|---------------------------------------|-----------------|------|----------------------------|
| Переход, если равенство               | BEQ D           | F0XX | IF Z==1 THEN IP+D+1 → IP   |
| Переход, если неравенство             | BNE D           | F1XX | IF Z==0 THEN IP+D+1 → IP   |
| Переход, если минус                   | BMI D           | F2XX | IF N==1 THEN IP+D+1 → IP   |
| Переход, если плюс                    | BPL D           | F3XX | IF N==0 THEN IP+D+1 → IP   |
| Переход, если выше или равно /перенос | BCS D<br>BHIS D | F4XX | IF C==1 THEN IP+D+1 → IP   |
| Переход, если ниже/нет переноса       | BCC D<br>BLO D  | F5XX | IF C==0 THEN IP+D+1 → IP   |
| Переход, если переполнение            | BVS D           | F6XX | IF V==1 THEN IP+D+1 → IP   |
| Переход, если нет переполнения        | BVC D           | F7XX | IF V==0 THEN IP+D+1 → IP   |
| Переход, если меньше                  | BLT D           | F8XX | IF N⊕V==1 THEN IP+D+1 → IP |
| Переход, если больше или равно        | BGE D           | F9XX | IF N⊕V==0 THEN IP+D+1 → IP |
| Безусловный переход                   | BR D<br>JUMP D  | CEXX | IP+D+1 → IP                |

Циклы в БЭВМ реализованы при помощи комбинации команд: LOOP (8XXX) и JUMP (CXXX). LOOP обращается к ячейке, где хранится число итераций, декрементирует его и, если значение больше 0, то выполняется следующая команда, иначе делается переход через команду. Обычно после LOOP ставят команду безусловного перехода JUMP, чтобы вернуться вновь в ту часть кода, которую мы хотим зациклить.

DR после м.ц. OF содержит значение операнда  
 AR адрес операнда

- $\sim 0 + DR \rightarrow DR$  ;  $\overline{0x0} = 0xFFFF = -1$ ; Вычитание единицы из DR
- $\sim 0 + DR \rightarrow BR, DR \rightarrow \text{MEM}(AR)$  ; Записываем значение операнда в память. Вычитание еще единицы (ЗАЧЕМ?!!)
- **if** BR(15) = 0 **then** **GOTO INT** ; Проверка на положительный (DR-1) и если да, то завершение цикла
- $IP + 1 \rightarrow IP$  ; Перескок через команду, если BR=DR-1 отрицательное
- **GOTO INT** ; Завершение цикла

*13. Подпрограммы в БЭВМ. Цикл исполнения команд перехода и возврата из подпрограммы. Стек, передача параметров. Позиционно-независимый код. Загрузчик и библиотеки.*

При написании кода часто возникает ситуация, когда отдельные части программы выполняются практически одну и ту же функцию. В целях рефакторинга эти фрагменты кода выделяются в отдельную область – подпрограмму. Обратный процесс вклинивания подпрограммы в саму программы называется инлайнинг, чем, кстати, занимаются многие современные компиляторы.

## CALL

В БЭВМ подпрограммы реализованы при помощи команды CALL (DXXX), которая записывает адрес возврата на вершину стека и делает переход на тот адрес, который мы указали в коде команды:

CALL: DR после м.ц. OF содержит адрес перехода

- $DR \rightarrow BR$  ; Адрес перехода записать в BR
- $IP \rightarrow DR$  ; Подготовить адрес возврата для записи в стек
- $BR \rightarrow IP$  ; Переход на подпрограмму
- $\sim 0 + SP \rightarrow SP, AR$  ; Уменьшить стек на 1
- $DR \rightarrow \text{MEM}(AR)$  ; Записать адрес возврата
- **GOTO INT** ; Завершение цикла

## RET

В конце каждой подпрограммы в БЭВМ располагается команда RET (0A00), которая осуществляет возврат из подпрограммы. По реализации напоминает команду CALL, но только в обратном порядке.

- **SP** → **AR** ; Вершину стека поместить в AR
- **MEM(AR)** → **DR** ; Прочитать адрес возврата
- **DR** → **IP** ; Вернуться из подпрограммы
- **SP + 1** → **SP** ; Увеличить стек на 1
- **GOTO INT** ; Завершение цикла

Стек – структура данных типа LIFO (Last In, First Out). Стек полезен для работы с временными данными, обработки подпрограмм и прерываний. В полноценном виде стека в БЭВМ так такого нет, есть лишь SP – регистр, который указывает на вершину стека. Сам квази-стек располагается в конце пространства памяти и растёт от конца к началу ОЗУ.

При работе со стеком подразумеваются две операции: записать данные на вершину стека командой PUSH (код 0C00) и считать данные с вершины стека командой POP (код 0800).

Первая команда сначала декрементирует SP и по полученному адресу записывает данные. Вторая команда сначала читает данные по адресу из SP, потом увеличивает счётчик на единицу. Здесь подразумевается, что ячейка после прочтения как бы стирается, но и не стирается, вооот...

Передать входные данные в подпрограмму можно несколькими способами:

1. При помощи аккумулятора (регистры общего назначения). Максимум можно передать 16 однобитовых логических параметров, что иногда может не хватать
2. Адресуемые ячейки памяти. Тут возникают уже сложности с организацией самих ячеек.
3. Использовать стек и рассмотренные выше команды.
4. Регистровые окна (юзлесс, но пусть будет).

### PIC – Position Independent Code

Это такой код, который остаётся работоспособным при перемещении внутри памяти. Такой код можно получить, если адресация к входным данным

(то есть к тем ячейкам, которые лежат вне программы) является прямой абсолютной, при этом адресация внутри самой программы относительна. Обычно используются в драйверах и модулей ядра, чтобы ОС смогла самостоятельно их располагать внутри ОЗУ.

#### Загрузчик и динамический линковщик программ. Библиотеки

Множество подпрограмм обычно формируют библиотеки. При обращении к какой-либо библиотеке за нужной подпрограммой, ОС использует загрузчик для того, чтобы загрузить нужную библиотеку и линковщик, чтобы установить адреса переходов внутри программы.

Функции динамического линковщика:

- Загрузка по выбранному ОС адресу.
- Изменение константных частей адресов в программе.
- Загрузка базовых значений регистров.
- Динамическая загрузка разделяемых библиотек.
- Связывание адресов основной программы с вызываемыми библиотеками.

Библиотеки можно разделить на два типа: динамически линкуемые, которые подтягиваются по требованию в процессе исполнения кода, и статические линкуемые, которые одновременно линкуются с программой в явном виде на этапе компиляции.

#### *14. Организация ввода-вывода в вычислительных системах. Инициация обмена, передача информации и завершение обмена. Драйверы.*

##### Организация ввода вывода в вычислительных системах

В вычислительных системах подключение ВУ осуществляется при помощи специальных шин ВУ, к которым уже подключаются контроллеры ВУ. Сами по себе ВУ гораздо медленнее процессора, поэтому, если мы подключимся напрямую, процессор будет простаивать в ожидании готовности ВУ. Как раз по этой причине мы и используем контроллеры ВУ.

Шина ВУ имеет следующие сегменты:

1. Шина данных. По ней поступают данные для обмена.
2. Шина адреса. По ней передаются адреса регистров контроллера, к которым мы будем обращаться.
3. Шина управления. По ней передаются управляющие сигналы или приказы, как обрабатывать ту или иную информацию внутри контроллера.

Обычно шину данных и шину управления не используют в одно и то же время, поэтому их можно мультиплексировать, то есть объединить в единую шину, где в один момент времени передаются управляющие сигналы, в другой – данные.

##### Драйверы

Драйвер – это специальная программа в памяти ЭВМ, которая организует взаимодействие ВУ, процессором ЭВМ и ОЗУ соответственно. Драйвер знает режимы работы ВУ, его регистры. Эта программа содержит алгоритм обращения к ВУ, обработки данных со стороны ВУ и со стороны процессора к ВУ. Все драйверы управляются единым программным интерфейсом, который задаётся ОС. Драйверы реализуют три стадии обмена:

##### 1. Инициация обмена

Этап, который характеризуется началом обмена информацией и выводом приказа на управляющую шину для ВУ. Есть три вида инициации обмена:

1. Синхронный – драйвер отдаёт приказ на шину ВУ в определённый момент времени.
2. Асинхронный – драйвер отдаёт приказ ВУ только в случае, если само ВУ явно дало понять о своей готовности. Готовность проверяется процессором (как было в 5 лабе).

3. Управляемый прерываниями – ВУ сигнализирует процессору о своей готовности. В отличие от асинхронного обмена, процессор сразу же приступает к обработке прерывания, то есть к обмену данными между ЭВМ и ВУ.

## 2. Передача данных

На данном этапе данные поступают с регистров ВУ в регистры процессора через шину данных. Есть два вида передачи данных:

1. Асинхронный – процессор должен подтверждать получение каждого сегмента данных.
2. Синхронный – процессор получает все данные в определённый момент времени без каких-либо подтверждений.

## 3. Завершение обмена

На этом этапе на процессор приходит сигнал о том, что передача данных завершена. Есть два вида завершения обмена.

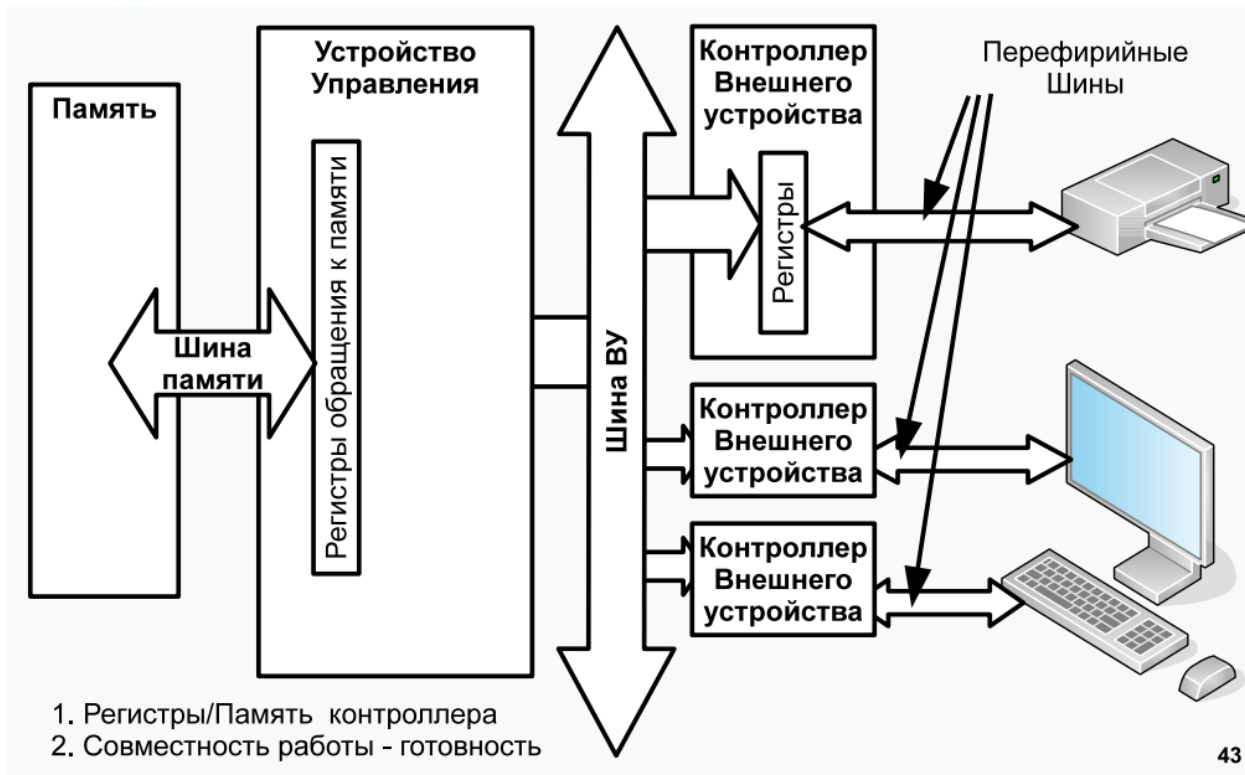
1. Синхронный – оповещение о завершении обмена происходит сразу же после завершения работы драйвера.
2. Асинхронный – оповещение о завершении обмена происходит в любой момент времени после завершения работы драйвера.

## Программно-управляемый ввод-вывод

Организуется программой, которую обрабатывает процессор. С одной стороны, является самой простой в плане реализации обмен, но его можно использовать лишь для обмена небольших кусков данных. Для обмена с ВУ используются драйверы.

## Управляемый аппаратурой ввод-вывод

Используется для обмена больших объёмов информации. Драйверы здесь уже не играют такой огромной роли, они нужны лишь для управления режимами работы контроллера. Всю работу выполняют контроллеры при помощи контроллера прямого доступа в память (DMA), который самостоятельно инициирует обмен и транспортирует информацию с ВУ напрямую в память в специально подготовленные ячейки. Процессор в данном вводе-выводе принимает минимальное участие.



К ЭВМ можно подключать огромное количество различных устройств ввода-вывода или ВУ. Эти устройства принимают и передают большой объём данных, которые, по понятным причинам, не могут быть размещены в регистрах ЭВМ. Поэтому информация с ВУ передаётся как в память ЭВМ, так и с неё. При этом обмен может идти под руководством процессора (программно-управляемый обмен), так и под управлением специального внешнего устройства (управляемый аппаратурой обмен), минуя процессор при помощи контроллера прямого доступа к памяти.

#### Программно-управляемый ввод-вывод

Все действия с вводом-выводом регулируются процессором, то есть все данные, идущие от ВУ к памяти и наоборот, обязательно проходят через процессор. Минусом такого ввода-вывода заключается в том, что процессор пребывает в ожидании готовности ВУ. С другой стороны, реализация такого ввода-вывода довольно простая.

#### Управляемый прерываниями ввод-вывод

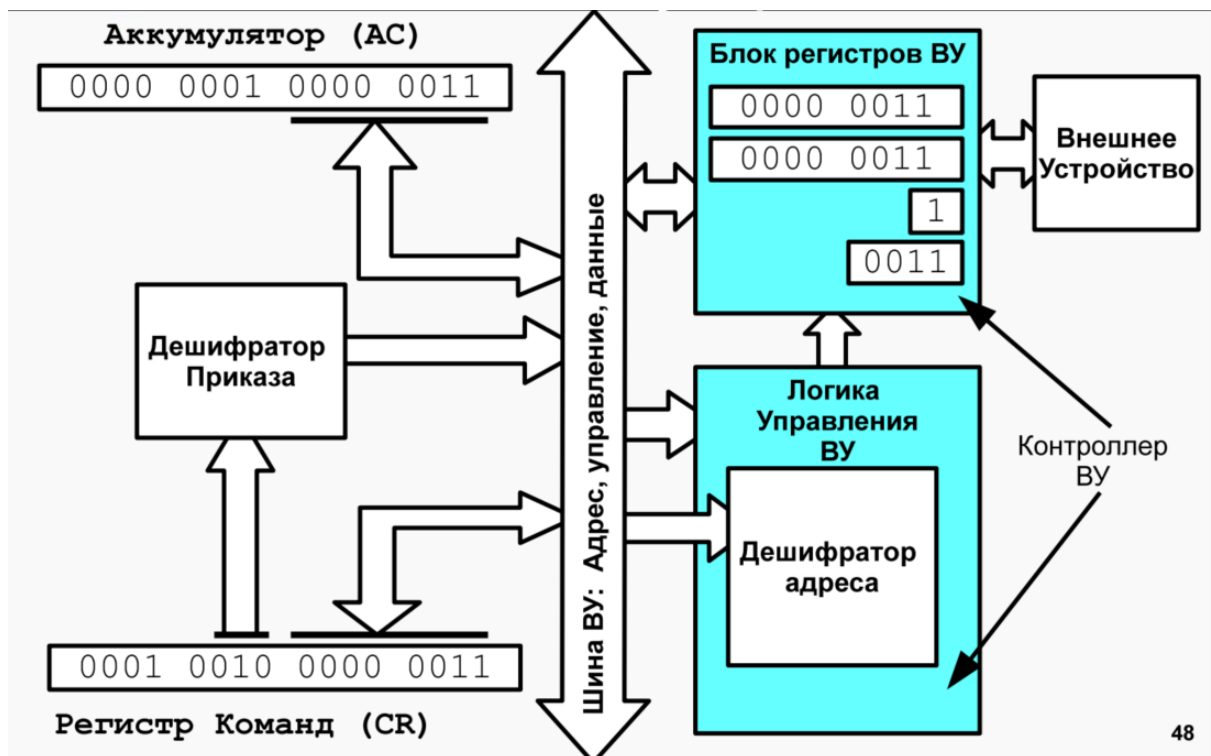
Работа ввода-вывода также завязана на процессоре. Однако во время ожидания готовности ВУ, процессор выполняет какую-то полезную работу. Как только ВУ отправит сигнал о своей готовности, происходит прерывание

процессора. Такой подход более производительный, но при этом требует больших усилий на обработку прерываний.

### Прямой доступ к памяти (Direct Memory Access)

ВУ работает напрямую с памятью, избегая процессор. Такой способ практически не нагружает процессор, следовательно, сам ввод-вывод будет работать быстрее. Но для реализации данного подхода нужно проектировать более сложные контроллеры ВУ, способных в прямой доступ к памяти.

В контексте БЭВМ реализованы лишь программно-управляемый ввод-вывод и управляемый прерываниями. Прямой доступ к памяти реализовать нельзя, так как нет соответствующих на это схем.



### Регистры контроллеров

SR – однобитовый регистр, в котором хранится бит готовности ВУ к вводу-выводу. Содержит единицу, если устройство готово, и ноль, если не готово. При извлечении его в аккумулятор, ставится в 6 бит слова. При этом старший байт затрагиваться не будет. То есть IN SR вернёт либо 0000.0000, либо 0100.0000

MR – 4-битовый регистр управления. Содержит в себе номер вектора прерывания и бит на запрет/разрешение прерывания.

DR – 8-битовый регистр данных, через который и происходит обмен данных. При его извлечении занимает лишь младший байт аккумулятора.

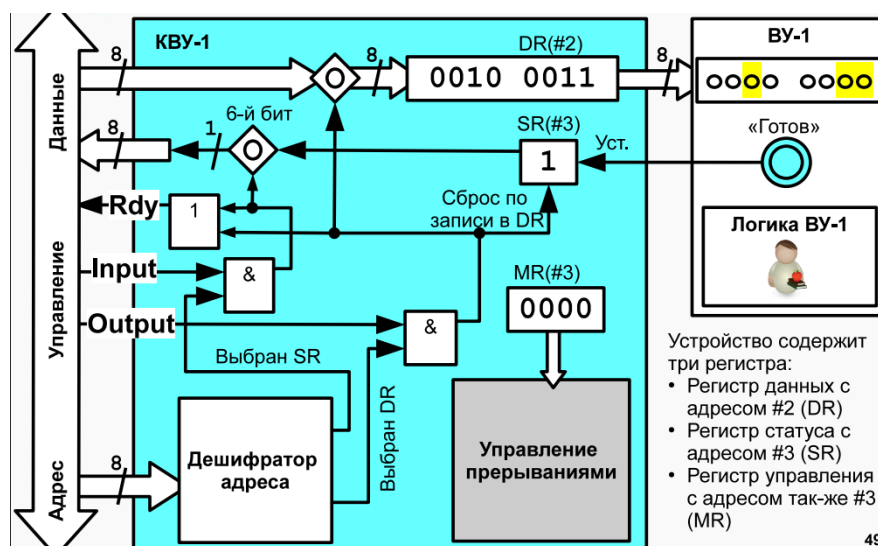
## Команда ввода-вывода

| 15 | 14 | 13 | 12 | 11     | 10 | 9 | 8 | 7                  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|----|----|----|----|--------|----|---|---|--------------------|---|---|---|---|---|---|---|
| 0  | 0  | 0  | 1  | Приказ |    |   |   | Регистр устройства |   |   |   |   |   |   |   |

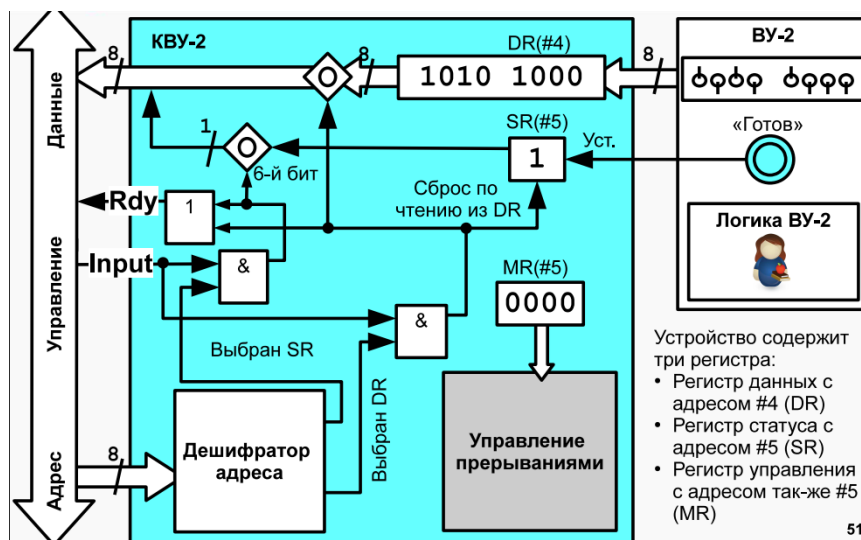
| Наименование          | Мнемон. | Код  | Описание                             |
|-----------------------|---------|------|--------------------------------------|
| Запрет прерываний     | DI      | 1000 |                                      |
| Разрешение прерываний | EI      | 1100 |                                      |
| Ввод                  | IN REG  | 12XX | REG → мл. байт AC                    |
| Вывод                 | OUT REG | 13XX | мл. байт AC → REG                    |
| Прерывание            | INT NUM | 18XX | Програмное прерывание с вектором NUM |
| Возврат из прерывания | IRET    | 0B00 | (SP)+ → PS, (SP)+ → IP               |

## Внешние устройства БЭВМ

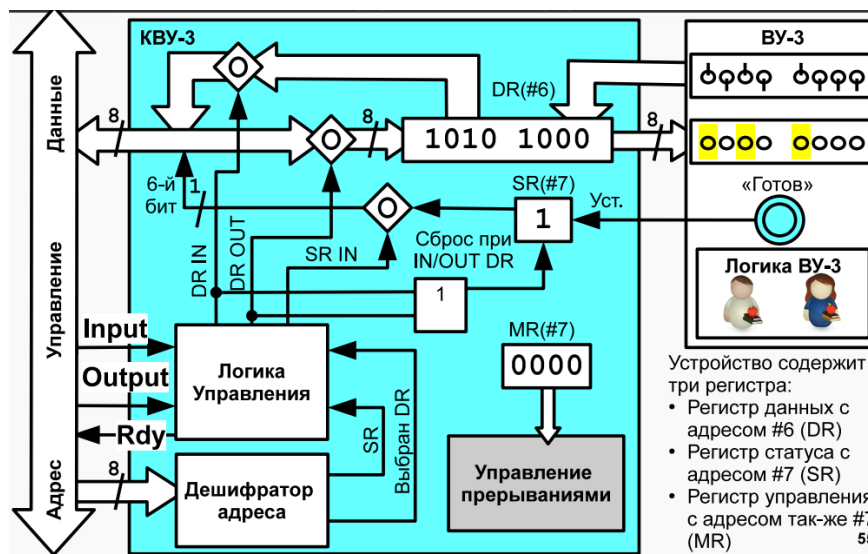
ВУ-1:



ВУ-2:



BY-3:



ВУ-0 (Таймер). Регистры 0x0, 0x1

- Устанавливает готовность и вызывает прерывание раз в  $100 \cdot DR$  миллисекунд.
- Если  $DR == 0$ , то готовность не устанавливается.
- Можно использовать для организации синхронного обмена (Но это фейк).

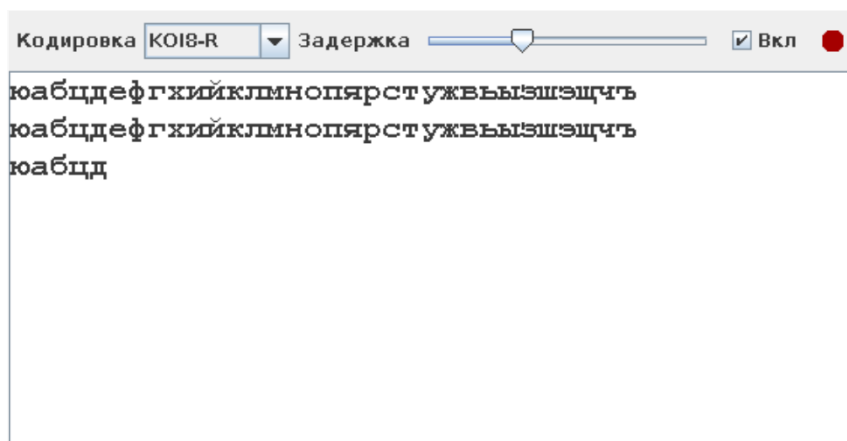
#### ВУ-4. Регистры 0x8 – 0xB:

- По функционалу похож на ВУ-3.
- Адресуется 4-мя регистрами.
- Отдельные регистры для входных (0x8) и выходных данных (0x9) данных.
- Регистр состояния по адресу 0xA.
- Регистр управления по адресу 0xB.
- Все регистры доступны для чтения/записи.
- Позволяет реализовать сложные конфигурации подключения.

ВУ-5 (Текстовый принтер). Регистры 0xC – 0xF:

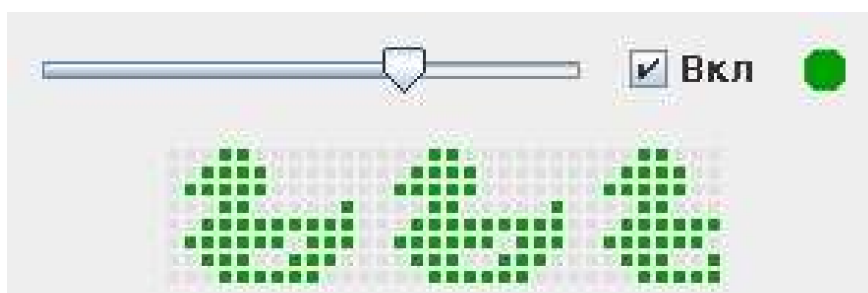
- Печатает символы из РДВУ в заданной кодировке.
- Регулируемая задержка (время печати).
- Перевод строки по символу CR.

- NUL – очистка.
- Остальные – неопределённое поведение.



ВУ-6 (Бегущая строка). Регистры 0x10 – 0x13:

- Размер матрицы: 32x8
- Сдвиг при записи нового значения в РДВУ
- Новое значение – справа
- Младший бит – нижний



ВУ-7 (8-ми разрядный 7-сегментный индикатор). Регистры 0x14 – 0x17.

- Формат РДВУ:

| 7 | 6   | 5    | 4 | 3 | 2 | 1 | 0 |
|---|-----|------|---|---|---|---|---|
| 0 | Поз | Симв | . |   |   |   |   |

- Симв==(A<sub>16</sub>) — установка в разряде знака «-»
- Симв==(B<sub>16</sub>-F<sub>16</sub>) — сброс разряда

ВУ-8 (Клавиатура). Регистры 0x18-0x1C:

- Код нажатой клавиши в выбранной кодировке устанавливается в РДВУ
- Автоматически устанавливается готовность.



ВУ-9 (Цифровая клавиатура). Регистры 0x1C-0x1F:

- При нажатии клавиши ее код помещается в РДВУ
- Клавиша 0-9 код 0-9
- Клавиша «-» код A
- «+» код B
- «/» код C
- «\*» код D
- «.» код E
- «=» код F

|   |   |   |   |
|---|---|---|---|
| 7 | 8 | 9 | / |
| 4 | 5 | 6 | * |
| 1 | 2 | 3 | - |
| 0 | . | = | + |

## 16. Организация асинхронного обмена в БЭВМ. Пример программы. Временные издержки асинхронного обмена.

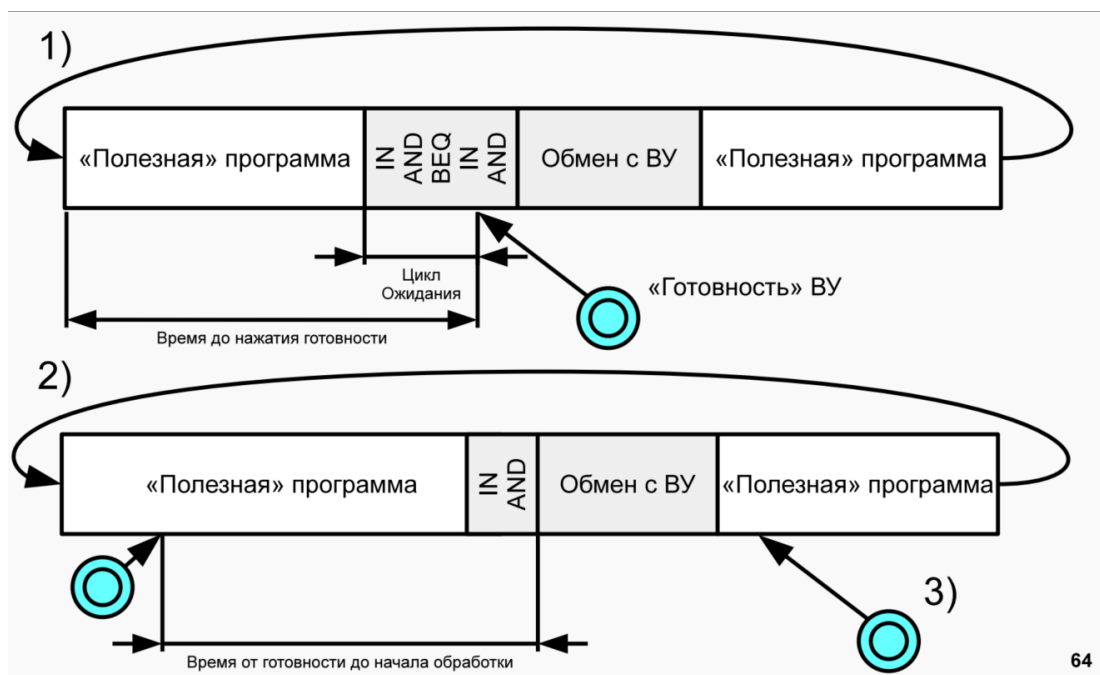
Кратко: асинхронный обмен подразумевает ситуацию, когда БЭВМ запрашивает готовность у КВУ до того момента, пока КВУ не даст сигнал о своей готовности

Пример программы на ассемблере:

| Ввод двух символов с устройства ввода ВУ-2 (DR#4, SR#5) |      |       |                                                  |  |
|---------------------------------------------------------|------|-------|--------------------------------------------------|--|
|                                                         | ORG  | 0x10  |                                                  |  |
| START:                                                  | CLA  |       |                                                  |  |
| S1:                                                     | IN   | 5     | ; Ожидание ввода первого символа                 |  |
|                                                         | AND  | #0x40 | ; Бит 6 SR == 0 («Готов» нажата?)                |  |
|                                                         | BEQ  | S1    | ; Нет - "Спин-луп"                               |  |
|                                                         | IN   | 4     | ; Ввод первого символа                           |  |
|                                                         | SWAB |       | ; Перемещение первого символа в старшую часть AC |  |
|                                                         | ST   | RES   | ; Сохранение его в ячейке RES                    |  |
| S2:                                                     | IN   | 5     | ; Ожидание ввода второго символа                 |  |
|                                                         | AND  | #0x40 | ; Бит 6 SR == 0 («Готов» нажата?)                |  |
|                                                         | BEQ  | S2    | ; Нет - "Спин-луп"                               |  |
|                                                         | LD   | RES   |                                                  |  |
|                                                         | IN   | 4     | ; Ввод второго в младшие 8 разрядов A            |  |
|                                                         | ST   | RES   |                                                  |  |
|                                                         | HLT  |       |                                                  |  |
| RES:                                                    | WORD | ?     | ; Ячейка для записи слова "ДА"                   |  |

Сколько циклов команд БЭВМ будет ждать ввода второго символа?

Временные издержки асинхронного обмена.



Первый вариант подразумевает наличие спин-лупа внутри всей программы. То есть программа входит в часть кода, где происходит проверка готовности. Программа не пойдёт дальше до тех пор, пока ВУ не даст сигнал о своей готовности.

Второй вариант подразумевает отсутствие спин-лупа. То есть если на моменте проверки готовности ВУ не готово, то программа пропускает фрагмент кода с обменом ВУ и продолжает выполнять полезную программу. Из плюсов такого подхода можно отметить полезность работы самого процессора. С другой стороны, невозможно быстро начать стадию обмена с ВУ (всё зависит от того, когда была нажата кнопка готовности).

## 17. Организация прерываний в БЭВМ. Вектора прерываний, контроллер прерывания.

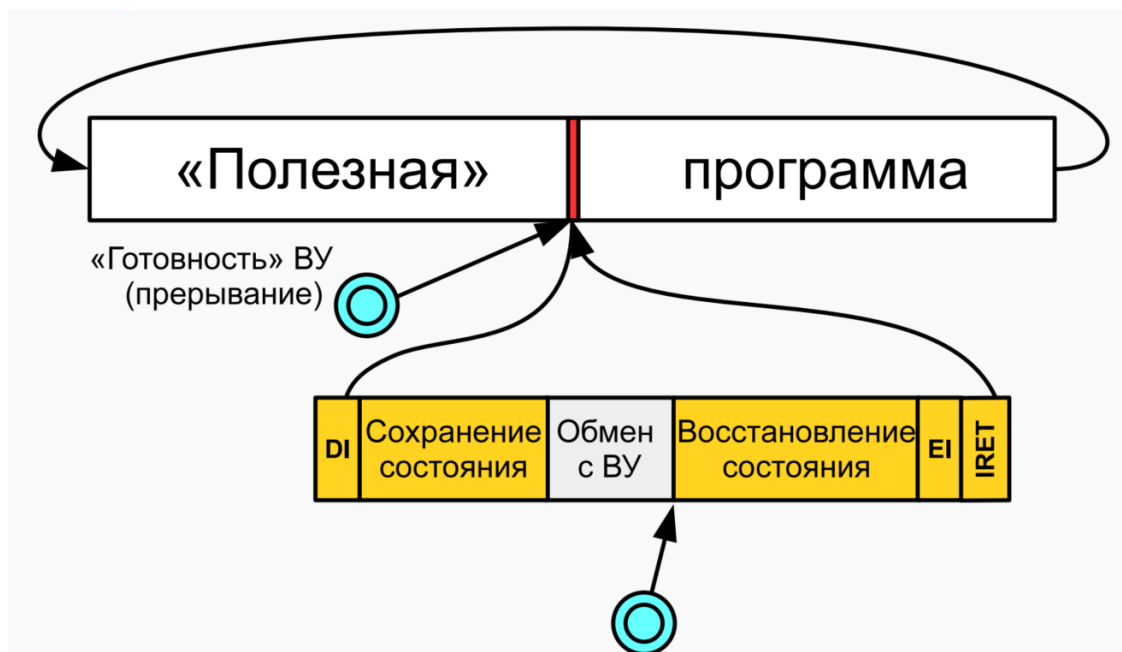
Прерывание – сигнал от программного или аппаратного обеспечения, сообщающий процессору о наступлении какого-либо события, требующего немедленной обработки. В рамках БЭВМ это сигнал о готовности ВУ к обмену данными.

Вектор прерывания – в контексте БЭВМ это совокупность адреса обработчика прерывания (то есть какой-то подпрограммы) и регистра состояния PS, с которым он стартует. Перед началом обработки прерывания необходимо их инициализировать, то есть хотя бы установить на ничего не делающую подпрограмму. В БЭВМ доступно 8 векторов прерываний, расположенных по адресу с 0x0 по 0xF включительно. На один вектор прерывания может приходиться несколько прерываний.

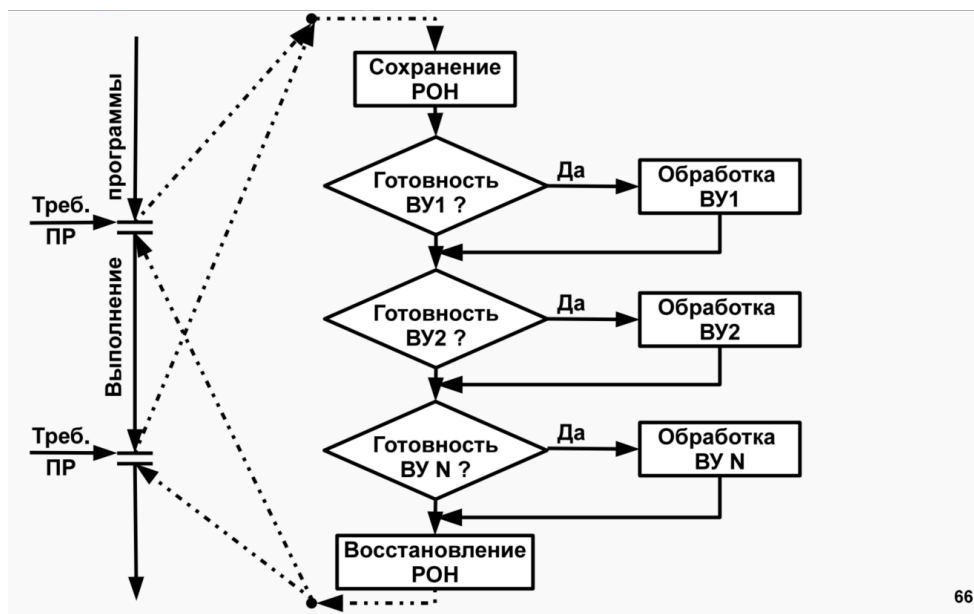
Если говорить об ЭВМ в общем, то количество ячеек в векторах прерывания варьируется, да и за инициализацию векторов прерывания отвечает ОС и БИОС.

| Адр . | Сод . |
|-------|-------|
| 0x0   | Адр 0 |
| 0x1   | PS 0  |
| 0x2   | Адр 1 |
| 0x3   | PS 1  |
| 0x4   | Адр 2 |
| 0x5   | PS 2  |
| 0x6   | Адр 3 |
| 0x7   | PS 3  |
| 0x8   | Адр 4 |
| 0x9   | PS 4  |
| 0xA   | Адр 5 |
| 0xB   | PS 5  |
| 0xC   | Адр 6 |
| 0xD   | PS 6  |
| 0xE   | Адр 7 |
| 0xF   | PS 7  |

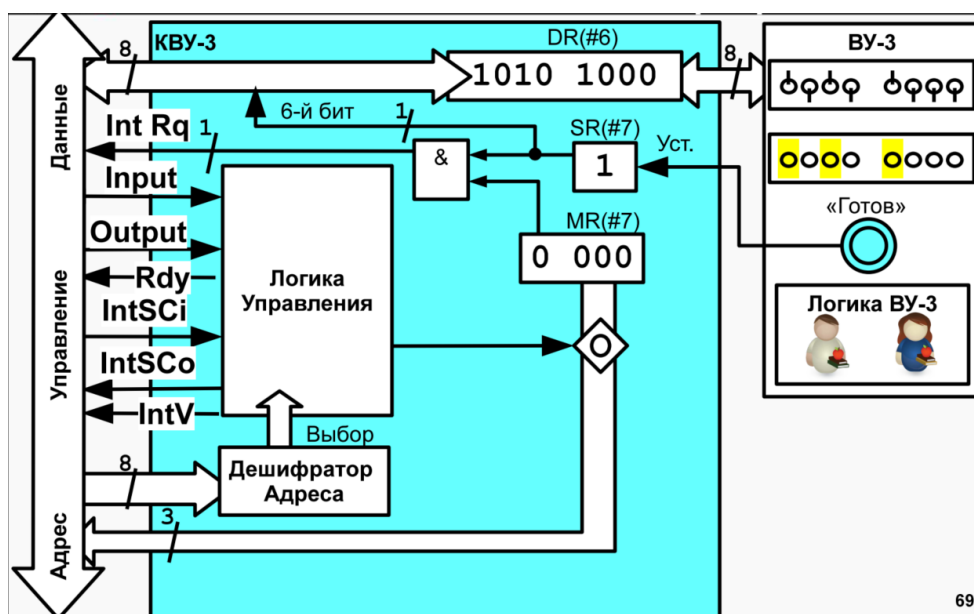
Инициация обмена по прерыванию:



Логика обработки и приоритет. Одно прерывание, много ВУ.



КВУ-3, управление прерывания.



| Сигнал (ы)                  | Направление<br>CPU . . . CNTRL                                      | Описание                                                                                                                 |
|-----------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Addr0..7                    | CR → Деш.адреса                                                     | Шина адреса регистров контроллеров внешних устройств                                                                     |
| Int#<br>Addr0..2            | CR ← MR контр.                                                      | Шина номера вектора прерывания (НВП)                                                                                     |
| IntV                        | CPU ← л/у контр.                                                    | Сигнал подтверждения передачи в CR НВП                                                                                   |
| Input                       | Деш.команд → л/у                                                    | Сигнал управления "Ввод"                                                                                                 |
| Output                      | Деш.команд → л/у                                                    | Сигнал управления "Вывод"                                                                                                |
| Rdy                         | CPU ← л/у                                                           | Сигнал подтверждения готовности                                                                                          |
| IntRq                       | PS (INT) ← SR контр.                                                | Запрос прерывания                                                                                                        |
| IntSC<br>IntSCi#<br>IntSCo# | УМК IRQS → IntSCi0<br>IntSCo0 → IntSCi1<br>IntSCo1 → IntSCi2<br>... | Цепочка сигналов предоставления прерывания.<br>Передается далее в случае отсутствия требования прерывания в контроллере. |
| SYN                         | Такт.ген → л/у                                                      | Синхросигнал тактового генератора                                                                                        |
| Data0..7                    | АС ↔ рег. контр.                                                    | Двунаправленная шина данных связи АС с регистрами контроллера                                                            |

## Команды для работы с прерываниями

- IRET (0B00) – возврат из прерывания:

$$\left. \begin{array}{l} SP \rightarrow AR \\ MEM(AR) \rightarrow DR \\ DR \rightarrow PS \end{array} \right\} \text{ Восстанавливает старый PS}$$

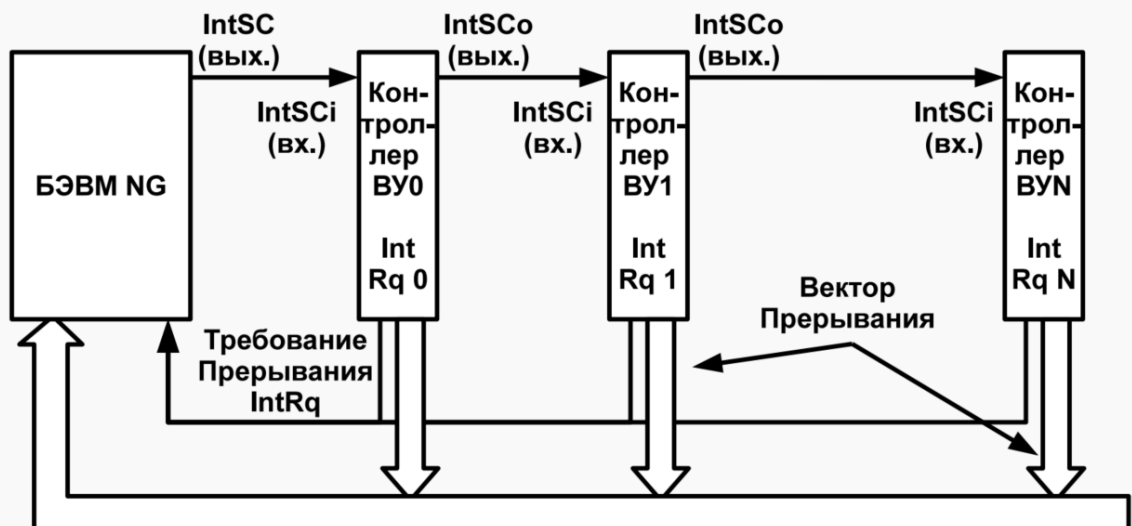
$$\left. \begin{array}{l} SP + 1 \rightarrow AR, AR \\ MEM(AR) \rightarrow DR \\ DR \rightarrow IP \end{array} \right\} \text{ Восстанавливает старый IP}$$

$$SP + 1 \rightarrow SP$$

Дальше идёт цикл обработки прерывания.

- EI (1100) – разрешение прерывания. Устанавливает в 5 бит регистра PS единицу.
- DI (1000) – запрет прерывания. Устанавливает в 5 бит регистра PS ноль.

Важно, что прерывания не обрабатываются в пошаговом режиме, так как в самом начале цикла прерывания БЭВМ проверяют наличия 7 бита W в регистре состояния, который отвечает за “РАБОТА/ОСТАНОВ” режим. Сигнал предоставления прерывания проходят через все КВУ, пока не дойдёт до того, где был запрос на прерывание. Как КВУ понимает, что именно его ВУ вызвало прерывание – спрятано в логике управления данного КВУ. Приоритет получения КВУ сигнала предоставления прерывания организуется аппаратно. Цикл прерывания идёт после всех команд, кроме HLT, DI и EI.



ПРП (IntSC) — ПРедоставление Прерывания (Interrupt supply chain).  
Может быть входной и выходной.

*18. Организация обмена по прерыванию программы в БЭВМ. Пример программы. Цикл прерывания.*

Подобный обмен используется при работе с низкоскоростными ВУ или когда момент передачи данных между ЭВМ и ВУ неизвестен. Обмен данным между ЭВМ и ВУ инициируется сигналом готовности с ВУ. Для реализации такого типа обмена используется аппаратная проверка наличия внешнего прерывания.

По картинке выше из прошлом билета видно, что при поступлении сигнала IntRQ в регистр PS, БЭВМ проходится по всем КВУ (если конечно прерывания разрешены, тут нужно смотреть в PS на 5 бит) в поисках того самого, что сделал запрос на прерывание. Как только БЭВМ нашла КВУ, его вектор прерывания помещается в CR. Дальше происходит цикл обработки прерывания, который помещает IP и PS в стек. Сохранение AC нужно организовывать самостоятельно. То есть минимальная информация о прерванной программы хранится на вершине стека в виде 3-х слов.

Цикл прерывания

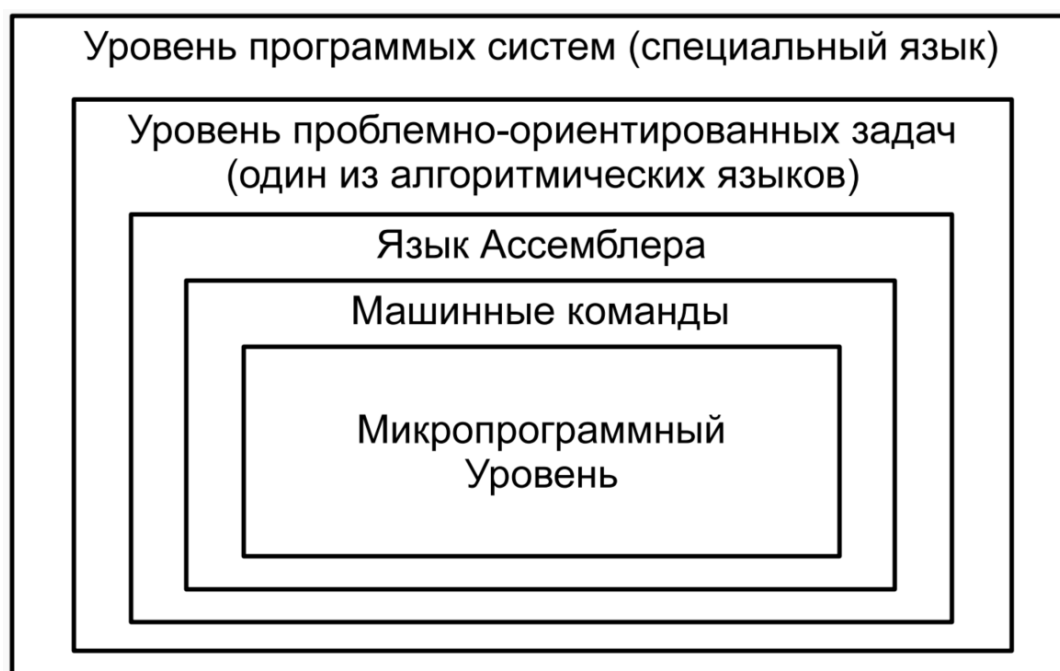
- If PS(W) = 0 then GOTO STOP. Проверка тумблера “Работа/Останов”. Если “Останов”, то программа останавливается.
- If PS(IRQ) = 0 then GOTO INFETCH. Если запроса на прерывание нет, то переходим к выполнению следующей команды.
- IRQSC (Interrupt Request Supply Chain). Формируем сигнал предоставления прерывания.
- $$\left. \begin{array}{l} \sim 0 + SP \rightarrow SP, AR \\ IP \rightarrow DR \\ DR \rightarrow MEM(AR) \end{array} \right\} \text{Записываем регистр IP на вершину стека.}$$
- $$\left. \begin{array}{l} \sim 0 + SP \rightarrow SP, AR \\ PS \rightarrow DR \\ DR \rightarrow MEM(AR) \end{array} \right\} \text{Записываем регистр PS на вершину стека.}$$
- LTOL(CR)  $\rightarrow$  BR. Младшие 8 бит (номер вектора прерывания) записываются в BR.
- SHL(BR)  $\rightarrow$  BR, AR. Вычисляем адрес ячейки, где лежит адрес обработчика прерывания (Простым умножением на 2).
- MEM(AR)  $\rightarrow$  DR. Достаём сам адрес обработчик прерывания.
- DR  $\rightarrow$  IP. Помещаем адрес обработчика прерывания в IP.

- LTOL(BR+1) → AR. Вычисляем адрес ячейки, в которой хранится PS для старта обработчика.
- MEM(AR) → DR. Содержимое PS обработчика прерываний достаётся из памяти и записывается в DR.
- DR → PS. Содержимое PS обработчика прерываний записывается в регистр PS в БЭВМ.

### Пример программы обработчика

| Готовность ВУ1: 2*А→РДВУ1, Готовность ВУ3: РДВУ3→ яч.3F |                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| V0:                                                     | ORG 0x0 ; Инициализация векторов прерывания<br>WORD \$DEFAULT,0x180 ; Вектор прерывания #0                                                                                                                                                                                                                                    |
| V1:                                                     | WORD \$INT1,0x180 ; Вектор прерывания #1                                                                                                                                                                                                                                                                                      |
| V2:                                                     | WORD \$DEFAULT,0x180 ; Вектор прерывания #2                                                                                                                                                                                                                                                                                   |
| V3:                                                     | WORD \$INT3,0x180 ; Вектор прерывания #3                                                                                                                                                                                                                                                                                      |
|                                                         | ...                                                                                                                                                                                                                                                                                                                           |
| V7:                                                     | WORD \$DEFAULT,0x180 ; Вектор прерывания #7                                                                                                                                                                                                                                                                                   |
| DEFAULT:                                                | IRET ; Просто возврат                                                                                                                                                                                                                                                                                                         |
| START:                                                  | ORG 0x020 ; Загрузка начальных векторов прерывания<br>DI<br>CLA<br>OUT 1 ; MR KBY-0 на вектор 0<br>OUT 5 ; MR KBY-2 на вектор 0<br>LD #9 ; разрешить прерывания и вектор №1<br>OUT 3 ; (1000 0001=1001) в MR KBY-1<br>LD #0xB ; разрешить прерывания и вектор №3<br>OUT 7 ; (1000 0011=1011) в MR KBY-3<br>...<br>JUMP \$PROG |
| PROG:                                                   | EI ; Установка состояния разр. прерывания<br>CLA ; Первоначальная очистка аккумулятора                                                                                                                                                                                                                                        |
| INCLP:                                                  | INC ; Цикл для наращивания<br>BR INCLP ; содержимого аккумулятора                                                                                                                                                                                                                                                             |
| IO3:                                                    | ORG 0x03F<br>; Ячейка для хранения кодов, поступающих с ВУ-3<br>WORD ?                                                                                                                                                                                                                                                        |
| INT1:                                                   | ORG 0x040<br>; Прерывание сохранило содержимое PS<br>NOP ; отладочная точка останова (NOP/HLT)<br>PUSH ; Сохранили AC<br>ASL ; Умножили AC на 2<br>OUT 2 ; Записали в РДВУ-1 (DR#2)<br>POP ; Вернули AC назад<br>NOP ; отладочная точка останова (NOP/HLT)<br>IRET ; Возврат из обработки прерывания                          |
| INT3:                                                   | ORG 0x050<br>; Прерывание сохранило содержимое PS<br>NOP ; отладочная точка останова (NOP/HLT)<br>PUSH ; AC кладем в стек<br>CLA<br>IN 6 ; РДВУ-3<br>ST \$IO3 ; сохранение<br>NOP ; отладочная точка останова (NOP/HLT)<br>POP ; AC вынимаем из стека<br>IRET ; Возврат из обработки прерывания                               |

*19. Понятие многоуровневой ЭВМ. Понятие и пример программы на разных уровнях.*



Возможность исполнения на ЭВМ программы, написанной на алгоритмическом языке, обеспечивается с помощью специальных системных программ: компиляторов и интерпретаторов.

Компиляция заключается в том, что процесс выполнения алгоритма осуществляется лишь после завершения процесса перевода исходной программы в машинный код.

В интерпретации же каждый оператор исходной программы заменяется программой-интерпретатором на эквивалентную последовательность машинных команд непосредственно перед исполнением программы. В отличие от компиляции, в интерпретации во время решения задачи машине нужны не только исходная программа, но и сама программа-интерпретатор.

Затраты на разработку компилятора или интерпретатора, а также затрачиваемое время на процесс перевода алгоритмического языка в машинный определяется сходством между рассматриваемыми языками. Поэтому обычно алгоритмический язык не сразу переводят в машинный код. Существует определённая иерархия языков, где более высокоуровневый язык базируется на более низкоуровневом.

Примером промежуточного языка между машинными и алгоритмическими кодами является язык символического кодирования команд, часто называемый ассемблером. По сути ассемблер – это первая попытка автоматизировать программирование в вычислительной технике. В нём допускается использование символических имён и меток. Компиляторы таких языком называются Ассемблерами. Они отводят определённые ячейки

памяти для символических переменных, организуют адресацию между различными частями программы, что сильно облегчает программирование.

Человеку, работающему с ЭВМ на том или ином языке, чаще всего кажется, что язык, на котором он общается с ЭВМ, и есть машинный. Как следствие, разным пользователям будет казаться, что они работают на разных вычислительных машинах. Отсюда растут ноги у таких понятий, как виртуальная (кажущаяся) ЭВМ и многоуровневая ЭВМ.

### Многоуровневая ЭВМ

Многоуровневая ЭВМ – это вычислительная машина, имеющая средства для работы с  $n$  различными уровнями языков программирования. Нижний язык, как правило, является самым простым в реализации, но наиболее сложным в написании кода. Верхний язык наоборот: самым сложным в плане аппаратурной реализации, но самым простым в написании.

Такую машину можно рассматривать как  $n$  различных виртуальных машин, каждая из которых имеет свой машинный язык.

#### 1. Уровень системных программ

В качестве примера можно привести в пример сложение в Excel как прикладную программу. Для этого в ячейку C1 запишем следующую формулу “=A1 + B1” и получим свой результат:

|   | A  | B  | C  |
|---|----|----|----|
| 1 | 10 | 20 | 30 |

#### 2. Уровень проблемно-ориентированных задач.

Java – один из алгоритмических языков. Программа сложения чисел на этом языке выглядит следующим образом:

```
public class Test {  
    public static void main(String[] args) {  
        int A = 10;  
        int B = 20;  
        System.out.println(A + B);  
    }  
}
```

### 3. Язык ассемблера

```
ORG 0x20
X: WORD 10
Y: WORD 20
Z: WORD ?
```

```
START:
    LD X
    ADD Y
    ST Z
    HLT
```

### 4. Машинные коды

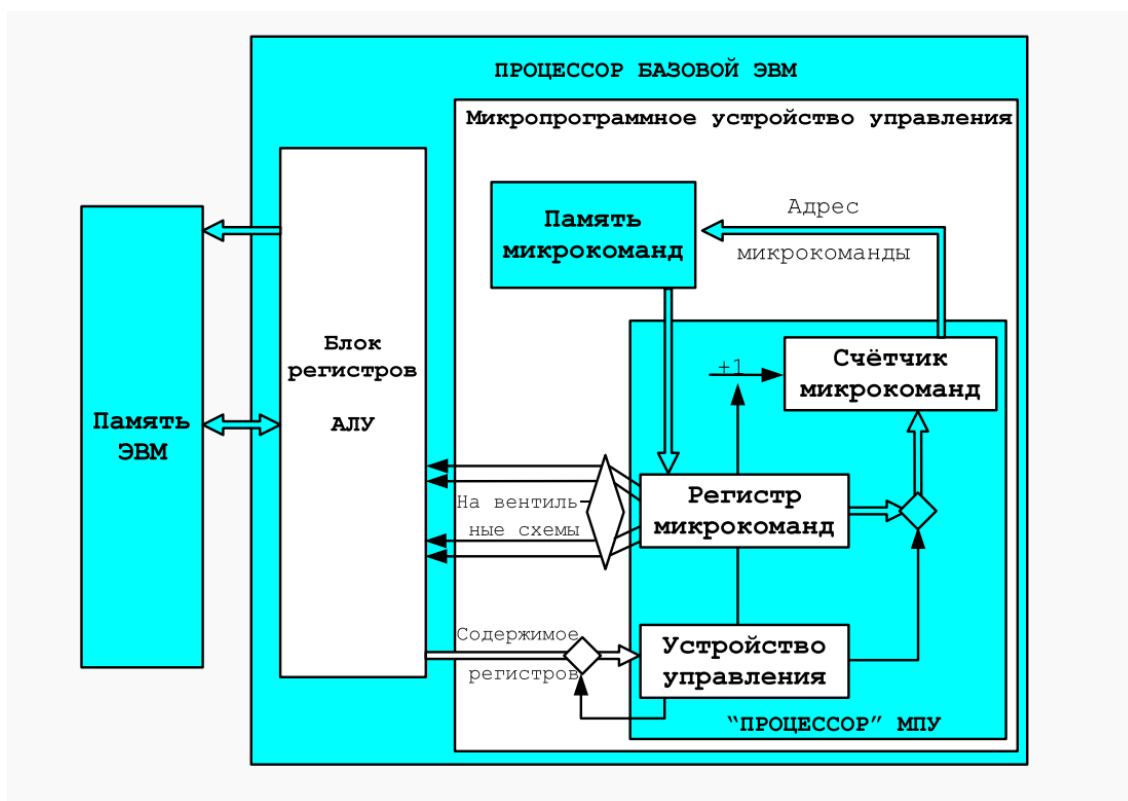
```
020 000A (X)
021 000B (Y)
022 0000 (Z)
023 A020 (LD X)
024 4021 (ADD Y)
025 E022 (ST Z)
026 HLT
```

### 5. Микропрограммный уровень

Тут рассмотрим только одну микропрограмму, так как писать много придётся

```
32    0010E09011    ADD    AC + DR → AC,  N, Z, V, C
```

20. Микропрограммный уровень БЭВМ. Структура МПУ. Форматы микрокоманд.

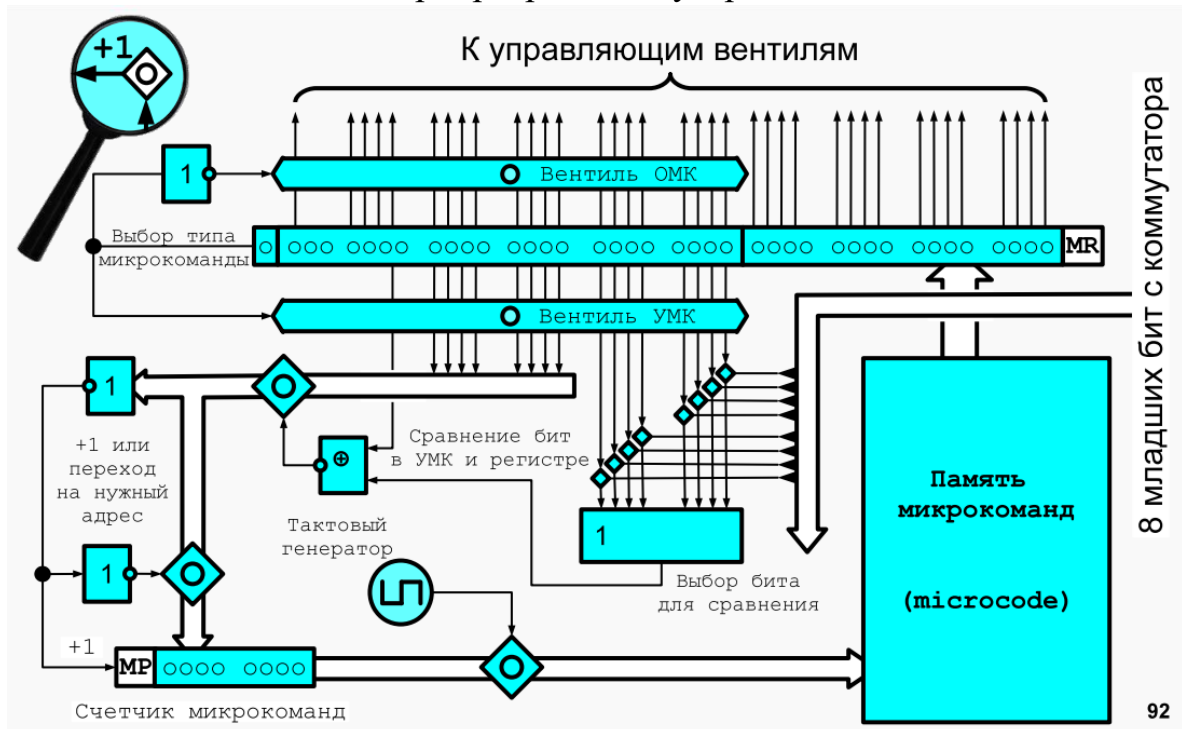


Если рассматривать БЭВМ как многоуровневую вычислительную машину, то на неё самом нижнем уровне выполняются элементарные действия по отношению словам с информацией. Такие действия называют микрооперациями.

Порядок следования микроопераций задаётся устройством управления БЭВМ, которая представляет из себя простейшую ЭВМ. Для такой ЭВМ регистры и вентильные схемы БЭВМ являются как бы внешними устройствами, с которыми производится ввод-вывод.

Программа такой ЭВМ (быть точнее МПУ) называется микропрограммой, а её команды, содержащие информацию об элементарных действиях и выполняющиеся в течение одного такта ТГ, называют микрокомандами. Микропрограмма хранится в ПЗУ – в памяти микрокоманд. В БЭВМ реализована возможность изменения микропрограммы через консольный режим. В каждом такте работы в регистр микрокоманд (РМК) пересылается очередная микрокоманда, на которую указывает счётчик микрокоманд (СчМК), выполняющий одновременно функцию регистра счётчика микрокоманд.

## Микропрограммное устройство



Внутри МПУ помимо вышеописанных памяти микрокоманд, СчМК и РМК есть устройство управление. Конкретно оно отвечает за логику обработки микрокоманды.

При поступлении микрокоманды в РМК первые 16 бит сразу отправляются на вентили. Дальше выбор вентиля зависит от типа команды. В БЭВМ существует два вида микрокоманд:

- Операционная микрокоманда (39 бит равен нулю) – все закодированные биты поступают на вентили, то есть происходит какая-то элементарная операция, будто пересылка данных между регистрами, арифметическая или логическая операция и так далее
- Управляющая микрокоманда (39 бит равен единице) – используется как команда ветвления внутри интерпретатора БЭВМ. Проверяет определённый бит с каким-то эталонным значением (записывается 32 бит) и, в случае совпадения, совершает переход по адресу, закодированной в команде.

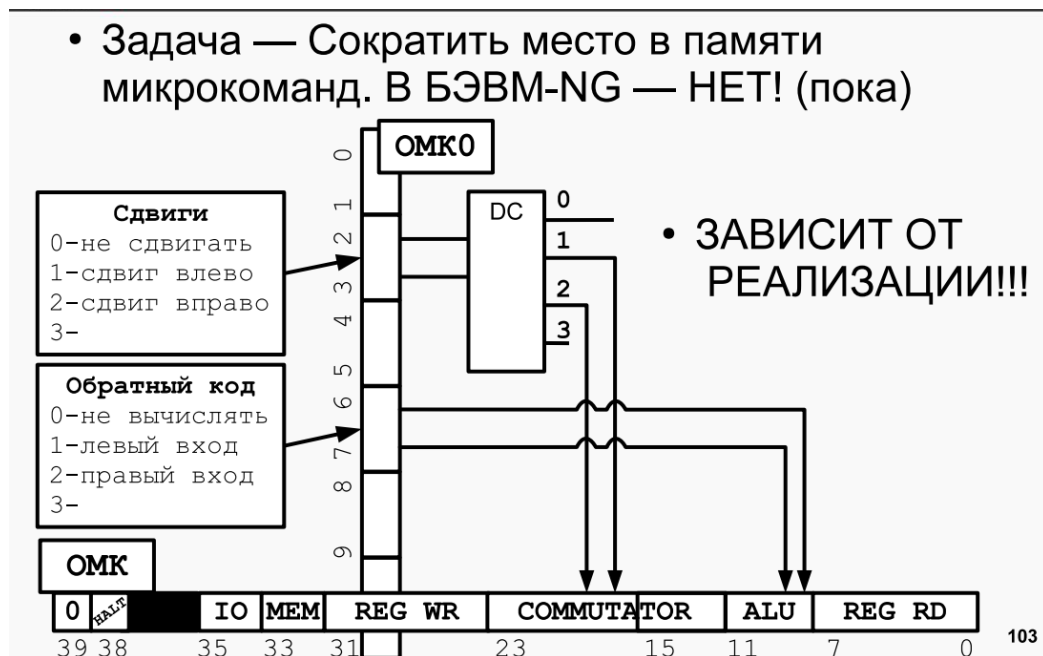
Логика изменения значения в СчМК проста. Если это операционная команда, то вентиль со стороны схемы сравнения не откроется, ибо сравнения так такого даже не было. Значит на шину к СчМК подаётся ноль, который в силу реализации инкрементируем СчМК.

Если же микрокоманда является управляющей, то всё зависит от успеха сравнения битов. Если биты совпали, то схема сравнения, основанная на NOT XOR, откроет вентиль и адрес перехода запишется в СчМК.

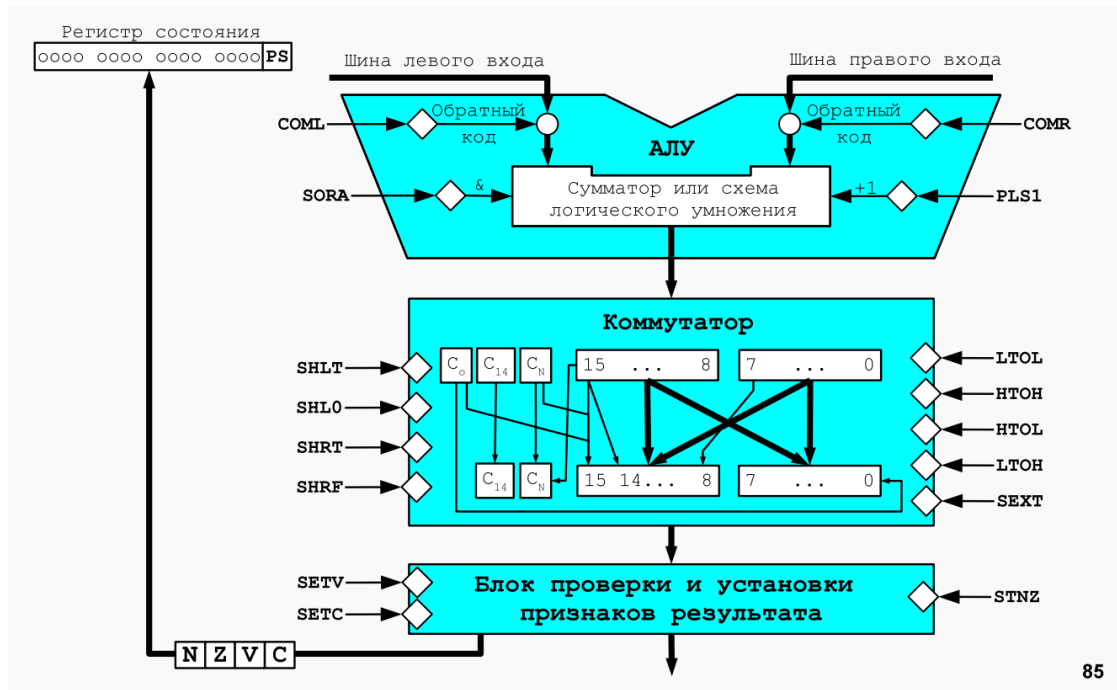
## Горизонтальные и вертикальные микрокоманды.

Внутри БЭВМ используются горизонтальные микрокоманды, где каждый бит отвечает за определённый клапан или хранит служебный бит. Однако в больших ЭВМ, где количество клапанов сильно больше, горизонтальные микрокоманды становятся слишком длинными. Поэтому в таких ситуациях используют кодируемые поля, которые в совокупности представляют из себя вертикальные микрокоманды.

Сократить количество битов можно за счёт кодирования клапанов, которые не могут одновременно открыться или же кодирование полей регистра проверяемого слова. Перевод из горизонтальной в вертикальную микрокоманду производится при помощи дешифратора. Поэтому полезность вертикальной микрокоманды зависит от выгоды установки дешифратора перед количеством памяти, занимаемом горизонтальными микрокомандами.



## 21. Структура и принципы работы АЛУ и коммутатора. Регистр Состояния БЭВМ.



АЛУ

Арифметически-логическое устройство имеет два входа для двух операндов. Все арифметические и логические команды БЭВМ и вспомогательные операции, нужные для нормального функционирования БЭВМ (Например, инкрементирование IP), можно выполнить с помощью АЛУ.

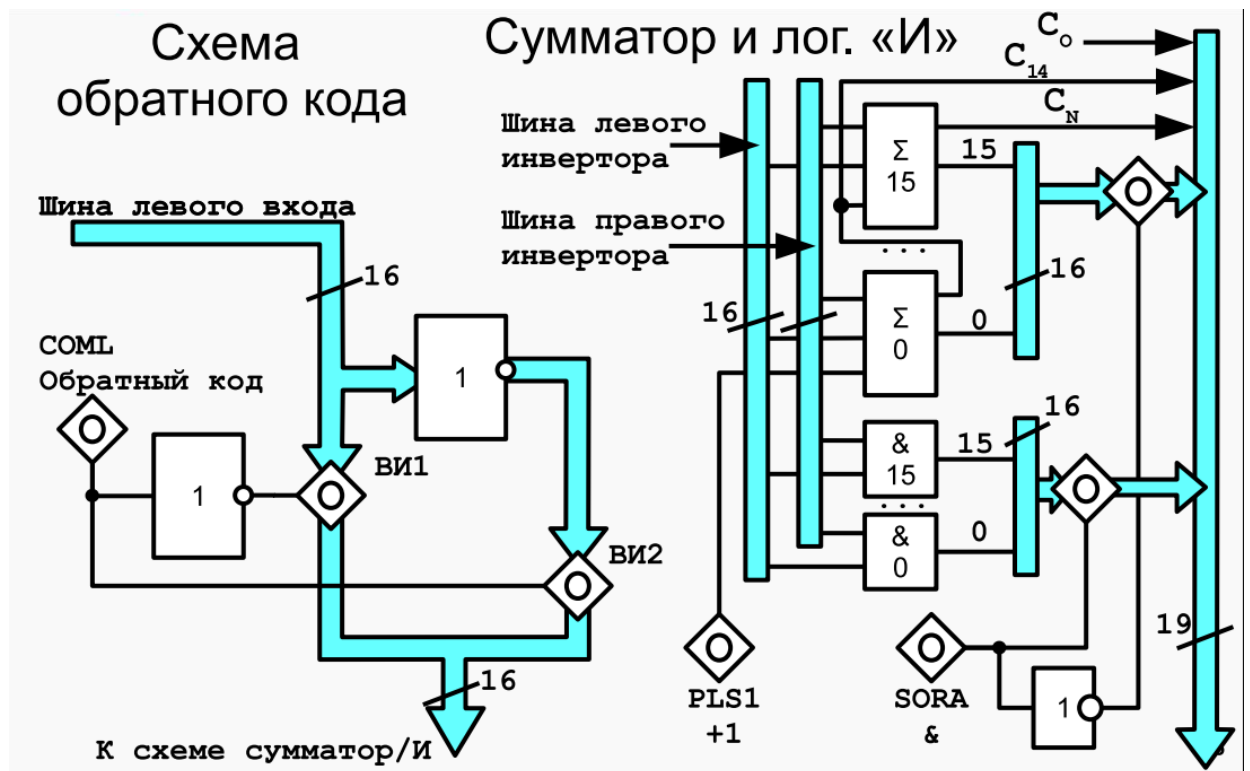


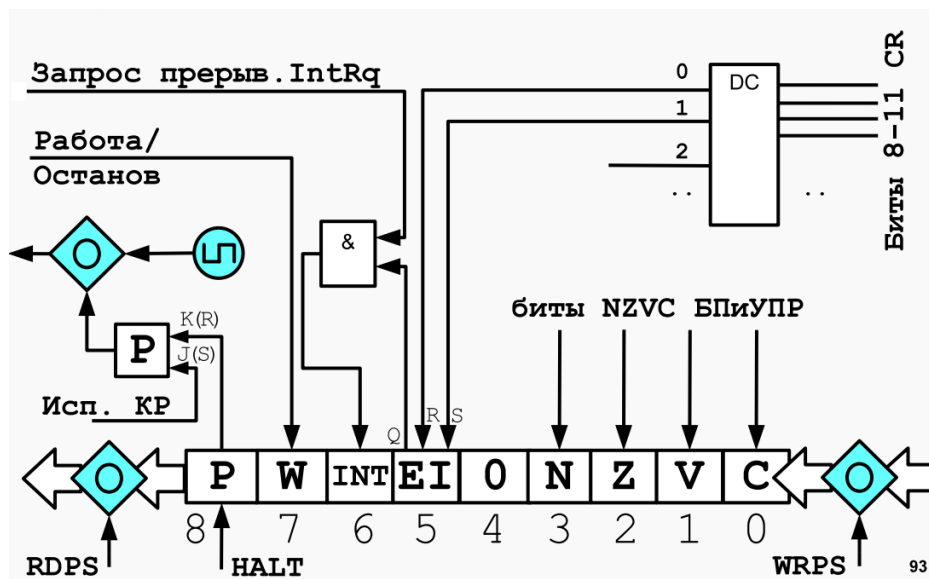
Схема обратного кода реализована таким образом, то либо открывается вентиль прямого кода, либо же обратного. Реализуется это при помощи подключения к цепи управляющего сигнала инвертора к одному из вентиляей. Обратный же код получается при помощи побитового инвертирования битов операнда.

Схема сложения реализована при помощи типичной схемы сумматора. Причём вентиль PLS1 добавляет единицу как сигнал переноса в сумматор 0-ого бита.

Логическое “И” реализовано как логическое “И” между каждым разрядом двух операндов. Бит переноса за разрядную сетку (Позже это будет Керри флагом) реализуется как выход переноса последнего сумматора. Таким же образом сохраняется бит переноса между 14 и 15 разрядом, которые будет нужен для определения Overflow флага. Не забываем ещё про старый перенос Carry. Итого в коммутатор у нас поступит аж 19 битов.

Выбор конкретной операции реализовано аналогичным образом, как и в схеме обратного кода.

### Регистр состояния

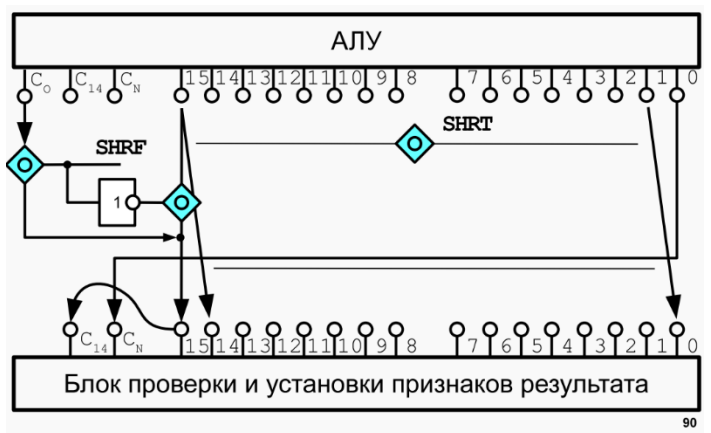
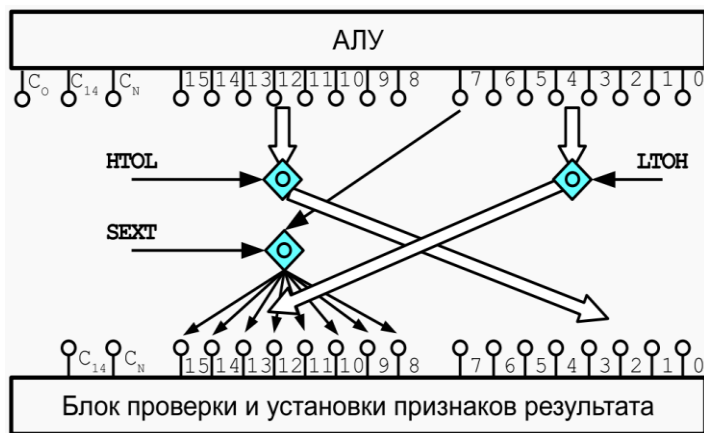
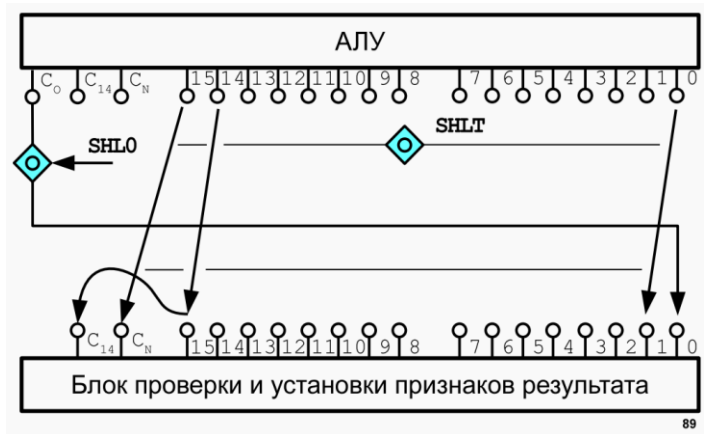
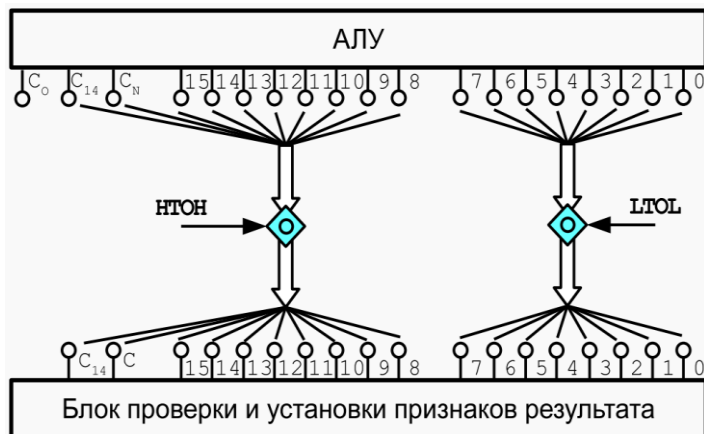


### Коммутатор

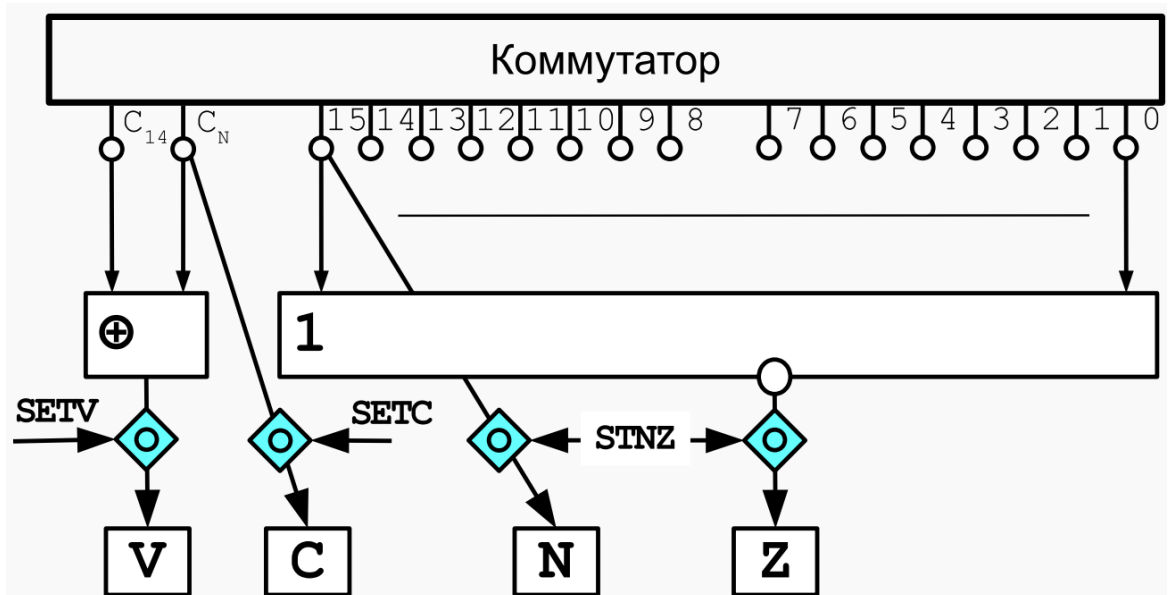
Коммутатор коммутирует (капитан очевидность, знаю). Он позволяет переставлять биты нужным нам образом: менять старший и младший бит, делать расширение знака, сдвигать как арифметически, так и логически биты влево или вправо.

Важно уточнить моменты, связанные с битами переноса. При прямой передаче байтов биты так и остаются неизменными. При расширении знаков или обмена байтами эти биты не передадутся. При смещении битов переносы

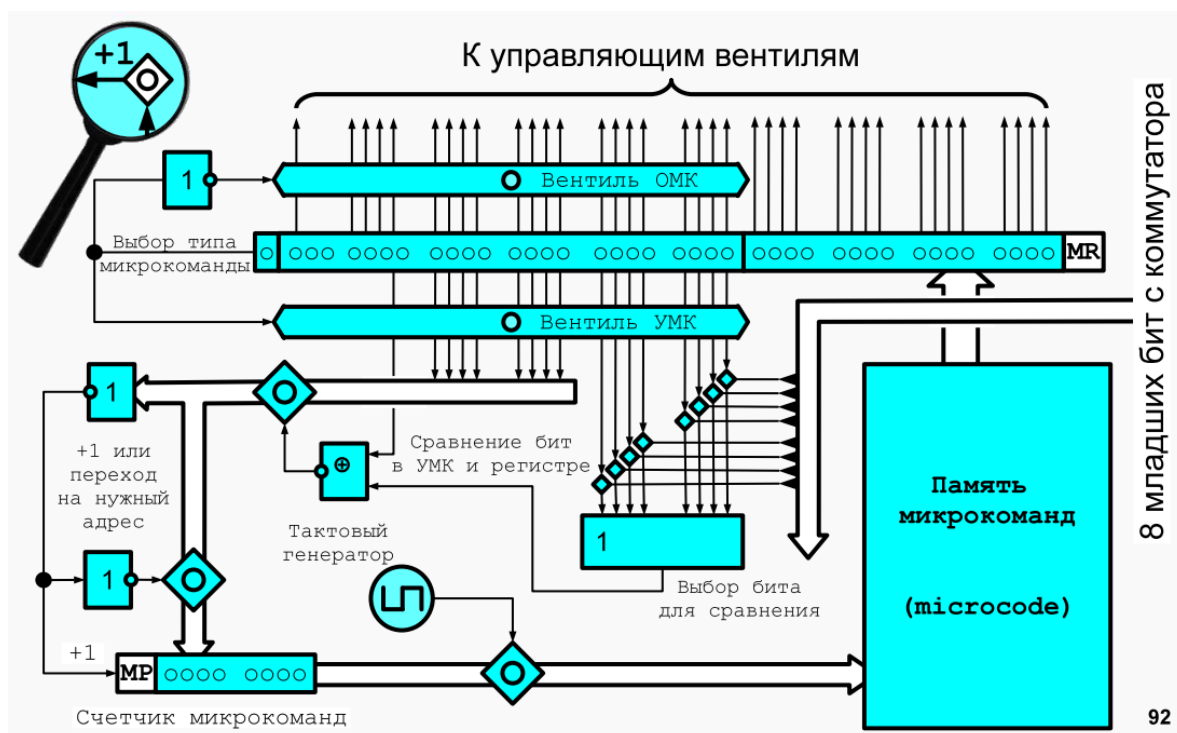
заполняться таким образом, что если из 14 в 15 мы сделали перенос в сдвиге вправо, то мы ставим бит переноса C14 и так далее.



Блок проверки и установки признаков результата:



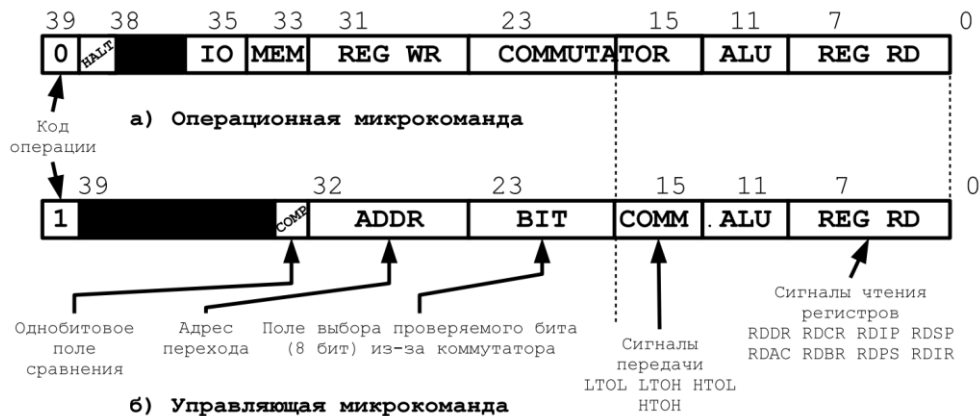
22. Микропрограммное управление вентиляльными схемами. Схема управления.  
Интерпретатор БЭВМ.



Чтобы разобраться в том, как работают вентиляльные схемы, нужно рассмотреть работу УУ. Как работает УУ было в общих чертах описано в 20 билете. Опишем принцип работы УУ в трёх случаях:

1. 39 бит равен 0, следовательно, в РМК лежит ОМК. Схема сравнения не сработает, поэтому клапан адреса перехода не откроется. Из этого следует, что после этого клапана будет сигнал, эквивалентный нулю. Верхний инвертор сработает и подаст сигнал об инкрементировании СчМК, при этом второй инвертор в этой цепи на выход даст нолик. Благодаря такой схеме СчМК не обнуляется при поступлении, а всего лишь инкрементируется.
2. 39 бит равен 1, что свидетельствует об УМК в РМК. Допустим, что схема сравнения выдала положительный результат. Значит, что клапан открылся и на шину перед СчМК подался какой-то адрес перехода. Первый инвертор выдаст ноль на выход, второй выдаст уже единицу, которая откроет клапан. Как следствие, в СчМК запишется адрес перехода.
3. 39 бит равен 1, что свидетельствует об УМК в РМК. Допустим, что схема сравнения отрицательный результат сравнения. Клапан не откроется, следовательно, дальше логика рассуждения аналогична пункту 1.

## Микрооперации, разновидности



## ОМК

| Бит                             | Мнемоника | Назначение                                                                |
|---------------------------------|-----------|---------------------------------------------------------------------------|
| <b>Чтение регистров</b>         |           |                                                                           |
| 00                              | RDDR      | DR (ПД) → правый вход АЛУ                                                 |
| 01                              | RDCR      | CR (ПК) → правый вход АЛУ                                                 |
| 02                              | RDIP      | IP (СК) → правый вход АЛУ                                                 |
| 03                              | RDSP      | SP (УС) → правый вход АЛУ                                                 |
| 04                              | RDAC      | AC (А) → левый вход АЛУ                                                   |
| 05                              | RDBR      | BR (БР) → левый вход АЛУ                                                  |
| 06                              | RDPS      | PS (РС) → левый вход АЛУ                                                  |
| 07                              | RDIR      | KR (КР) → левый вход АЛУ                                                  |
| <b>АЛУ</b>                      |           |                                                                           |
| 08                              | COMR      | Обратный код правого входа АЛУ                                            |
| 09                              | COML      | Обратный код левого входа АЛУ                                             |
| 10                              | PLS1      | Операция $A + B + 1$ (PLuS 1)                                             |
| 11                              | SORA      | Операция И (Sum OR And) (да, мы наркоманы)                                |
| <b>Управление коммутатором</b>  |           |                                                                           |
| 12                              | LTOL      | Прямая передача младшего байта                                            |
| 13                              | LTON      | Передача младшего байта в старший                                         |
| 14                              | HTOL      | Передача старшего байта в младший                                         |
| 15                              | HTON      | Прямая передача старшего байта                                            |
| 16                              | SEXT      | Расширение знака младшего байта (sign extend)                             |
| 17                              | SHLT      | Сдвиг влево (арифметический) (SHift Left)                                 |
| 18                              | SHLO      | Передача старого значения С в младший бит (для циклического сдвига влево) |
| 19                              | SHRT      | Сдвиг вправо (арифметический) (SHift Right)                               |
| 20                              | SHRF      | Переключатель сдвига вправо (для циклического сдвига 15 разряд)           |
| 21                              | SETC      | Установка С                                                               |
| 22                              | SETV      | Установка V                                                               |
| 23                              | STNZ      | Установка N и Z                                                           |
| <b>Запись в регистры</b>        |           |                                                                           |
| 24                              | WRDR      | АЛУ → ПД                                                                  |
| 25                              | WRCR      | АЛУ → ПК                                                                  |
| 26                              | WRIP      | АЛУ → СК                                                                  |
| 27                              | WRSP      | АЛУ → УС                                                                  |
| 28                              | WRAC      | АЛУ → А                                                                   |
| 29                              | WRBR      | АЛУ → БР                                                                  |
| 30                              | WRPS      | АЛУ → РС                                                                  |
| 31                              | WRAR      | АЛУ → РА                                                                  |
| <b>Работа с памятью</b>         |           |                                                                           |
| 32                              | LOAD      | ОП → ПД                                                                   |
| 33                              | STOR      | ПД → ОП                                                                   |
| <b>Организация ввода-вывода</b> |           |                                                                           |
| 34                              | IO        | Передача адреса и приказа на ВУ                                           |
| 35                              | INTS      | Предоставление прерывания                                                 |
| 36                              |           | Зарезервирован                                                            |
| 37                              |           | Зарезервирован                                                            |
| <b>Останов БЭВМ</b>             |           |                                                                           |
| 38                              | HALT      | Останов                                                                   |
| <b>Тип команды</b>              |           |                                                                           |
| 39                              | TYPE      | Бит выбора ОМК/УМК                                                        |

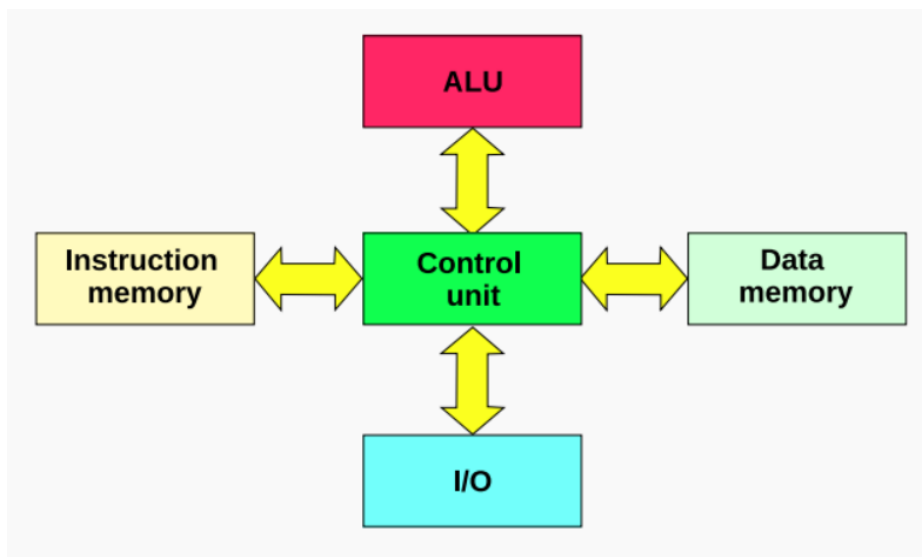
## Интерпретатор

Интерпретатор хранится в ПЗУ МПУ и представляет из себя огромную последовательность микрокоманд, по которым проходит МПУ при выполнении машинной команды БЭВМ. В интерпретаторе есть 256 ячеек для хранения горизонтальных команд. Сам по себе интерпретатор состоит из следующих циклов:

- Цикл выборки команд
- Цикл выборки адреса операнда и обработки режимов адресации
- Цикл выборки операнда
- Цикл исполнения
  - Декодирование и исполнение адресных команд
  - Декодирование и исполнение ветвлений
  - Декодирование и исполнение безадресных команд
- Декодирование и исполнение команд ввода-вывода
- Цикл прерывания
- Пультовые операции
- Свободные ячейки для:
  - Арифметической команды
  - Команды перехода
  - Безадресной команды

*23. Архитектура ЭВМ. Гарвардская и фон-Неймановская архитектура. Организация обмена архитектуры ЭВМ с использованием шин.*

Гарвардская архитектура

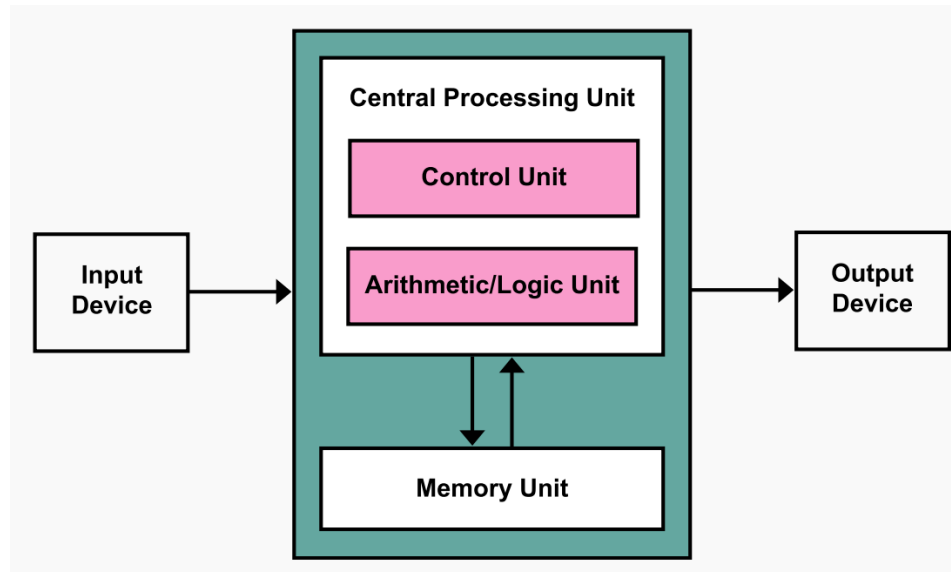


Гарвардская архитектура характеризуется физическим разделением памяти команд и памяти данных. Каждая память соединяется отдельной шиной, что позволяет одновременной с чтением-записью данных при выполнении текущей команды производить выборку и декодирование следующей команды, что значительно повышает производительность всей системы, в отличие от архитектуры фон-Неймана.

Недостатком такой архитектуры заключается в необходимости проведения большого количества шин, а также в фиксированном объеме данных, выделенной для команд и данных соответственно. Вследствие этого перераспределение такой памяти происходит крайне неоперативно. Однако с развитием микроэлектронной техники в значительном степени удалось преодолеть указанные недостатки.

Гарвардская архитектура широко используется во внутренней структуре современных высокопроизводительных МП, где используется отдельная кэш-память для хранения команд и данных.

## Архитектура фон-Неймана



Принстонская архитектура, которая часто называется архитектурой фон-Неймана, характеризуется общей оперативной памятью для хранения программ, данных, а также для организации стека. Для обращения к данной памяти используется общая системная шина, по которой передаются команды и данные.

Наличие общей памяти позволяет оперативно перераспределять её объём для хранения отдельных массивов команд, данных и реализации стека в зависимости от решаемых задач. Таким образом, обеспечивается более эффективное использование оперативной памяти в каждом конкретном случае применения МП.

Использование общей шины упрощает отладку, тестирование и текущий контроль функционирования системы, повышает её надежность. Поэтому данная архитектура очень долго доминировала в вычислительной технике.

Из недостатков можно отметить последовательность выполнения выборки команды и обрабатываемых данных по общей системной шине. При этом общая шина становится узким горлышком, которое ограничивает производительность всей вычислительной машины.

## 24. Архитектура многопроцессорных ЭВМ. Системный коммутатор. Архитектуры UMA и NUMA.

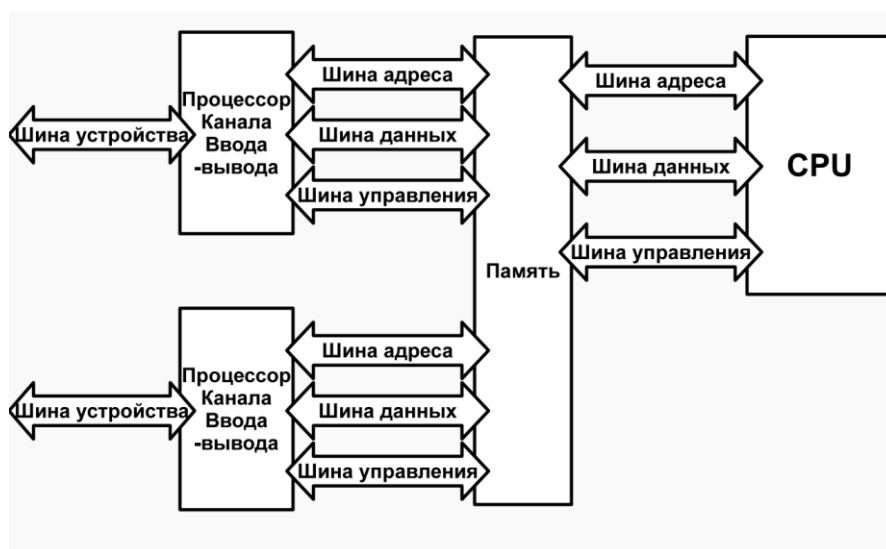
### История развития вычислительных машин

1. Нулевое поколение – механические компьютеры, поколение актуально до 1945 года;
2. Первое поколение – электронные лампы (1945-1955 гг.);
3. Второе поколение – транзисторные компьютеры (1955-1965 гг.). Знаковый пример: PDP-8;
4. Третье поколение – интегральные схемы (1965-1980 гг.). Пример: PDP-11;
5. Четвёртое поколение – сверхбольшие интегральные схемы (1980-?). Характеризуется появлением первых ПК.
6. Пятое поколение – небольшие и “невидимые” компьютеры (1989-?).

По невидимыми компьютерами подразумеваются такие вычислительные машины, которые есть уже в бытовых вещах, например, процессоры внутри стиральной машины.

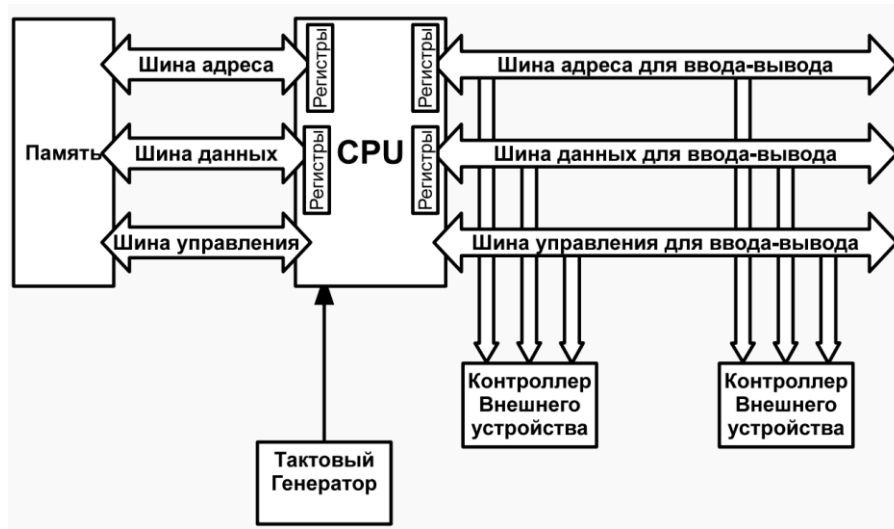
### История развития архитектуры процессоров

1) Канальная организация – это первая организация вычислительных машин, где всё строится вокруг памяти. Скорость всех блоков примерно одинакова. У внешних устройств есть свои процессоры. Все устройства независимы друг от друга

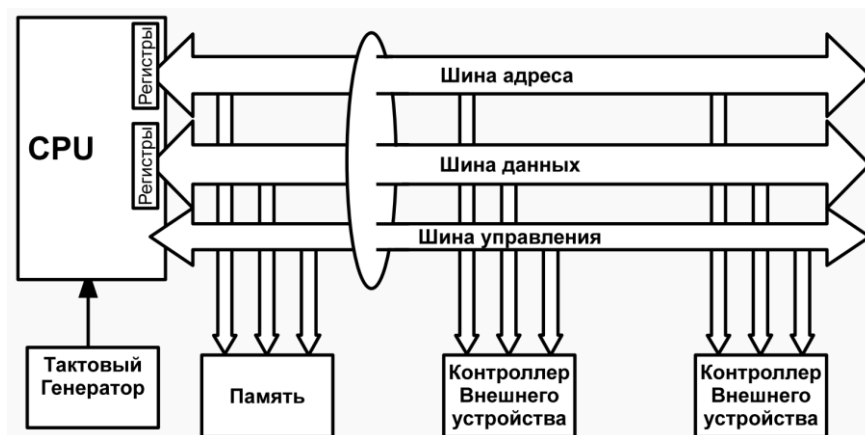


2) Раздельные шины – производительность процессора стала быстрее, чем производительность памяти. Поэтому главным звеном в архитектуре стал

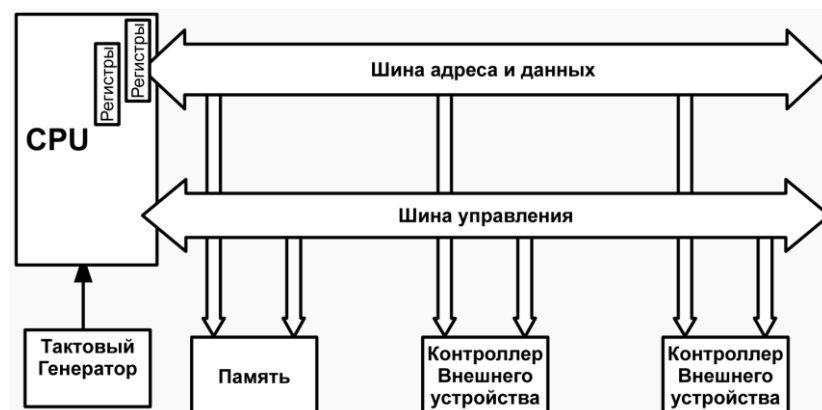
сам процессор. При этом скорость работы внешних контроллеров осталась прежней.



3) Общие шины – между 2, 3 и 4 поколениями пытались как можно сильнее повысить производительность, пришла идея все остальные устройства, кроме самого процессора, поместить на общую шину, дабы упростить устройство самого процессора.



4) Мультиплексирование шин – чтобы сократить количество шин между блоками вычислительной машины, адрес и данные решили передавать по одним и тем же проводам, что было предпринято во время развития 3 поколения. В один такт ТГ передавался адрес, в другой – данные.

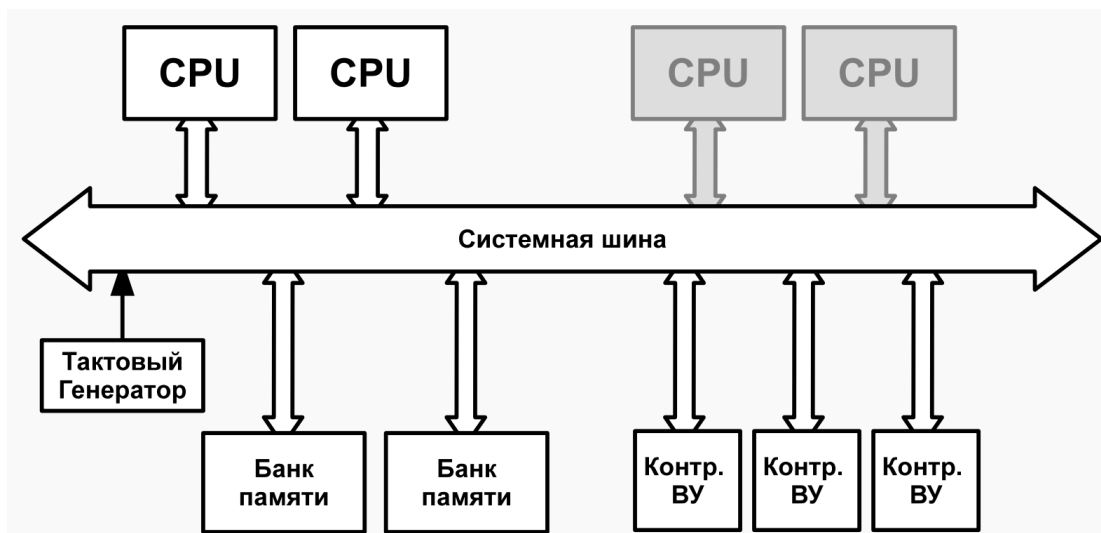


## Мультипроцессорность

5) UMA (Uniform Memory Access) – самая быстрая составляющая – это сам процессор. Где-то в 100 раз медленнее память, где-то в 1.000.000 раз – КВУ. Стояла задача повысить эффективность работы процессора. Конечно, можно повышать частоту ТГ, но мы упираемся в ограничения из-за скорости света, максимально можно добиться частоты в 5 ГГц.

Тогда решили использовать сразу несколько процессоров внутри вычислительной машины. Но возникает проблема с системной шиной в связи с потерей времени и отсутствием синхронизации.

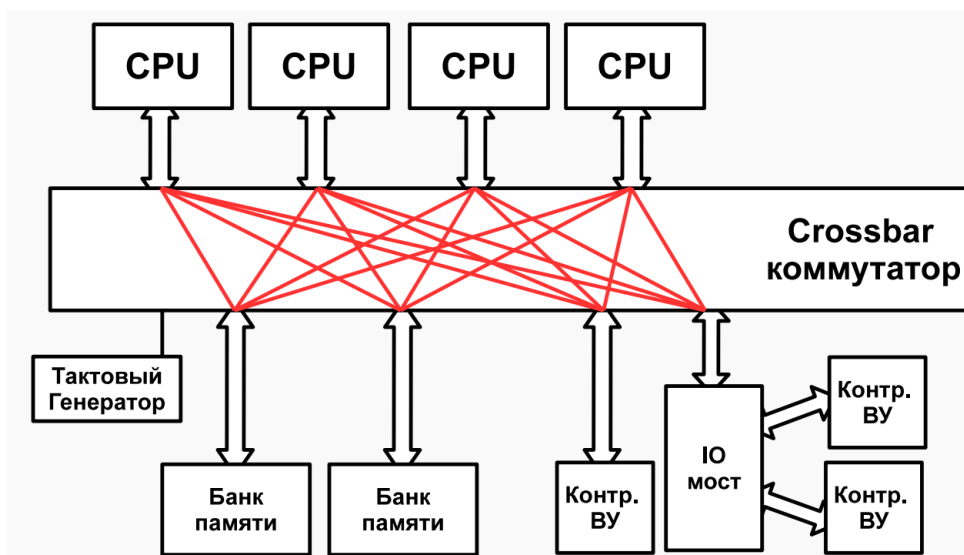
При 2-х процессорах эффективность возрастает, так как пока один процессор выполняет вычисления внутри себя, второй же взаимодействует с КВУ или памятью. При 3-х и более процессорах возникает “узкое горлышко”, коим является системная шина.



UMA – архитектура многопроцессорных компьютеров с общей памятью. Все микропроцессоры в UMA-архитектуре используют физическую память одновременно. При этом время запроса к данным не зависит от того, какой именно процессор обращается к данной памяти, и не от того, какой именно чип памяти содержит искомые данные.

6) Коммутатор – для решения проблемы с узким горлышком было решено создать коммутатор, где каждый порт соединён с каждым процессором, шину убрали. То есть каждый процессор в один момент времени может обращаться к одному из устройств. В результате производительность многопроцессорных машин резко возросла.

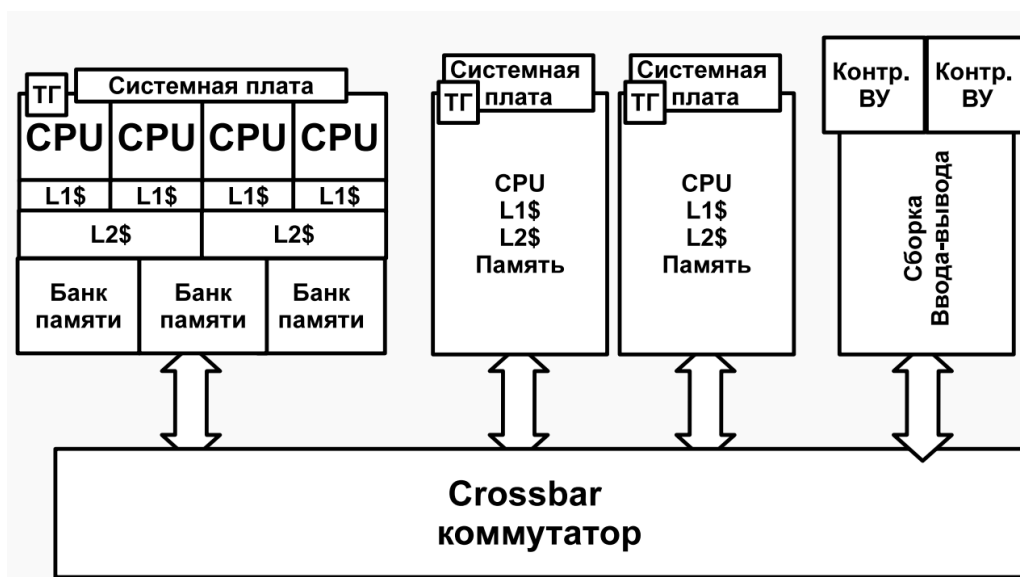
Также в явном виде разделили устройства на быстродействующие и медленные. Например, внешние устройства теперь подключаются через специальные мосты.



7) NUMA (Non-uniform memory access) – такая архитектура предназначена для того, чтобы ещё больше увеличивать количество процессоров.

У каждого процессора есть свой локальный кэш, локальная память и свой ТГ. Вся эта каша реализована как одна единая плата, соединённая при помощи коммутатора. Есть общая память для процессоров, но время обращения к ней зависит от того, кто к ней обращается.

Каждый процессор, теоретически, может работать на своей тактовой частоте, ну и плюсом ко всему можно на горячую менять элементы системы. Все процессоры такой архитектуры находятся под управлением ОС. ОС старается, чтобы вычисления не выходили за рамки системной платы, иначе производительность тупо упадёт.



NUMA – схема реализации компьютерной памяти, которая используется в многопроцессорных машинах, когда время доступа к памяти определяется расположением к ней обращающегося процессора.

## 25. Структура современных процессоров. Окружение процессора. CISC, RISC, VLIW.

### Характеристика современных коммерческих процессоров

- Разрядность адреса и данных: 16/32/64 бита.
- Тактовые частоты: 500МГц – 5ГГц.
- Многопроцессорные с CPU от 1 до 100+ штук;
- Многоядерные с 1-16 ядер.
- От 1Гб то терабайтов ОЗУ.
- Используют кэш-память разных уровней.
- Суперскалярные (процессор может выполнять более 1 команды за 1 такт)
- CISC, RISC, VLIW

### CISC

CISC (Complete Instruction Set Computer) – полный набор команд микропроцессора, который характеризуется:

- Сравнительно небольшое количество регистров общего назначения
- Большое количество машинных команды, которые семантически нагружены как операторы в высокоуровневых языках. Выполняется за большое число тактов
- Большое количество видов адресаций
- Большое количество команд различной разрядности

К подобным процессорам можно отнести семейство процессоров x86 от Intel, которые были отягощены обратной совместимостью со старыми процессорами.

Из недостатков CISC процессоров можно отметить:

- Высокая стоимость аппаратной части
- Сложность с распараллеливанием вычислений
- Состав и назначение их регистров существенно неоднородны
- Широкий набор команд усложняет декодирование инструкций, что расходует аппаратные ресурсы. Вследствие чего, возрастает число тактов, необходимых для выполнения инструкций.

## RISC

RISC (Reduced Instruction Set Computer) – архитектура компьютера с сокращённым набором команд. Предполагает реализацию в ЭВМ сокращённого набора простейших, но наиболее часто используемых команд, что позволяет упростить аппаратную часть и, благодаря этому, повысить эффективность машины. Современные процессоры с RISC-архитектурой характеризуются следующим:

- Упрощённый набор команд, имеющих фиксированную длину
- Отсутствуют макрокоманды, усложняющие структуру процессора и замедляющие его работу.
- Взаимодействие с ОЗУ ограничивается лишь пересылкой данных.
- Уменьшено количество видов адресаций.

Новый подход к архитектуре позволил значительно сократить площадь, требуемую для покрытия на кристалле интегральной схемы. Это позволило увеличить количество регистров в процессоре, из чего последовало заметное уменьшение количества обращения к ОЗУ.

Главной особенностью RISC архитектуры стал механизм перекрывающихся окон, предназначенный для уменьшения числа обращения к ОЗУ и межрегистровых передач, что увеличило эффективность работы ЭВМ.

В пример можно привести новомодные процессоры Intel, где сам процессор является CISC, при этом ядро – RISC.

## VLIW

VLIW (Very Long Instruction Word) – архитектура, которая появилась в России. Она не попадает под принципы фон Неймана, так как нарушается принцип последовательного выполнения команд.

Архитектура ЭВМ с VLIW позволяет сократить объём оборудования, нужного для реализации параллельной выдачи нескольких команд. Вместо того, чтобы последовательно выдавать команды, операции упаковываются в одну очень длинную команду. Ответственность за упаковку команд кладётся на компиляторы.

Для машин с VLIW архитектурой был разработан новый метод планирования выдачи команд – трассировочное планирование. То есть помимо процесса обычной упаковки команд в одну длинную, происходит ещё и оптимизация всей программы.

Примеры процессоров: Intel Itanium и микропроцессоры “Эльбрус”.

## Структура современного процессора

Принцип работы ядра процессора основан на цикле, описанном ещё Джоном фон Нейманом. В упрощённом виде этапы цикла работы ядра процессора можно представить следующим образом.

1. Блок инструкций проверяет наличие прерываний. Если оно было, то происходит сохранение текущего состояния ВУ. То есть данные регистров, счётчика команд заносятся на вершину стека. В счётчик адреса помещается адрес обработчика прерываний. По окончании прерывания, исходное состояние ВУ восстанавливается.
2. Блок выборки инструкций из счётчика команд считывает адрес команды, предназначенной для выполнения. По этому адресу находится сама команда в кэше, либо в ОЗУ. Полученные данные передаются в блок декодирования.
3. Блок декодирования расшифровывает команду, при необходимости используя для интерпретации команд микрокод из ПЗУ МПУ. Если это команда перехода, то в счётчик команд заносится адрес перехода, иначе счётчик команд инкрементируется (для процессора с длиной команды 32 бита – на 4 шага) и передаёт управление в блок выборки данных.
4. Блок выборки данных считывает из Кеш-памяти или ОЗУ нужные для выполнения команды данные. Управление передаётся планировщику.
5. Управляющий блок определяет, какому блоку выполнения инструкций обработать текущую задачу, и передаёт управление этому блоку.
6. Блоки выполнения инструкций делают своё дело и передают управление блоку сохранения результатов.
7. При необходимости, результат заносится в ОЗУ. После всех действий, управление передаётся блоку выборки команд.

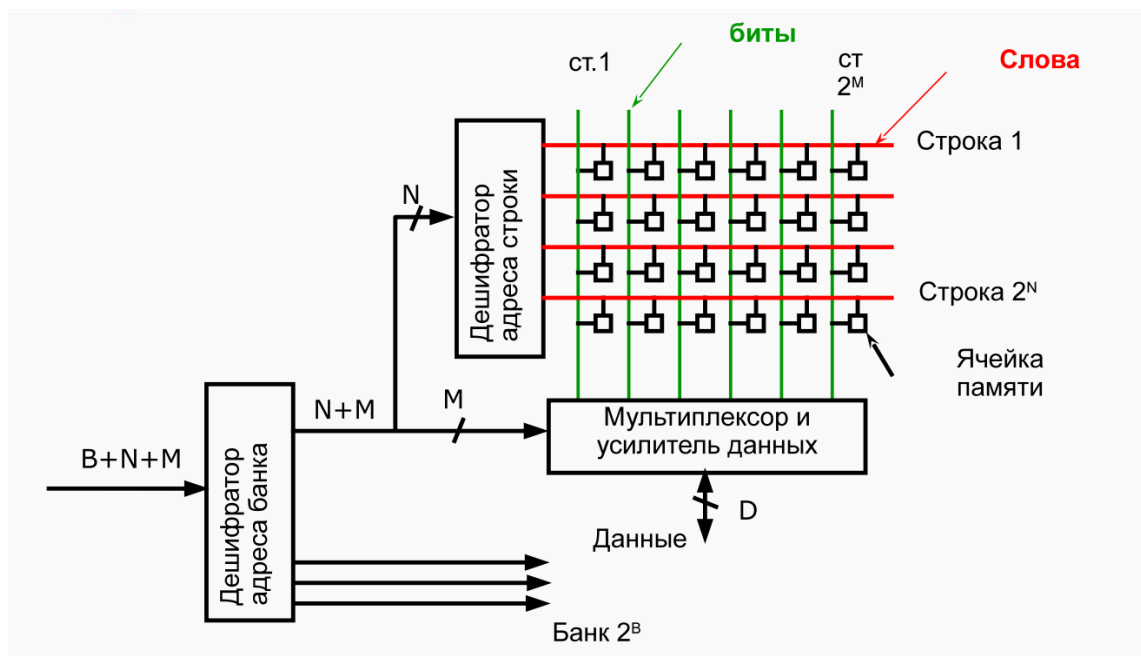
Описанный выше цикл называется процессом. Отсюда и название самого процессора.

Скорость перехода от одного этапа цикла к другому определяется тактовой частотой ТГ процессора, а время работы каждого этапа и цикла зависит от внутреннего устройства ядра процессора.

Если говорить в общем, то у современных процессоров есть микросхемы поддержки и чипсеты. В основном преобладают высокоскоростные интерфейсы, например, слоты PCI и порты HDMI. Микросхема поддержки нужна для управления КВУ, которая связана с процессором каналом прямого доступа к памяти.

26. Адресуемая память, организация и временные диаграммы.  
Конструктивные особенности современной памяти.

Адресуемая память



Заместо того, чтобы использовать огромнейший дешифратор как в БЭВМ, решили разбить память на банки. Картинка из презы не совсем точная, так как сигнал  $N$  и  $M$  сразу подаётся к дешифратору и мультиплексору соответственно. Адресация к такой памяти выглядит следующим образом:

| Bank | $N$ | $M$ |
|------|-----|-----|
|------|-----|-----|

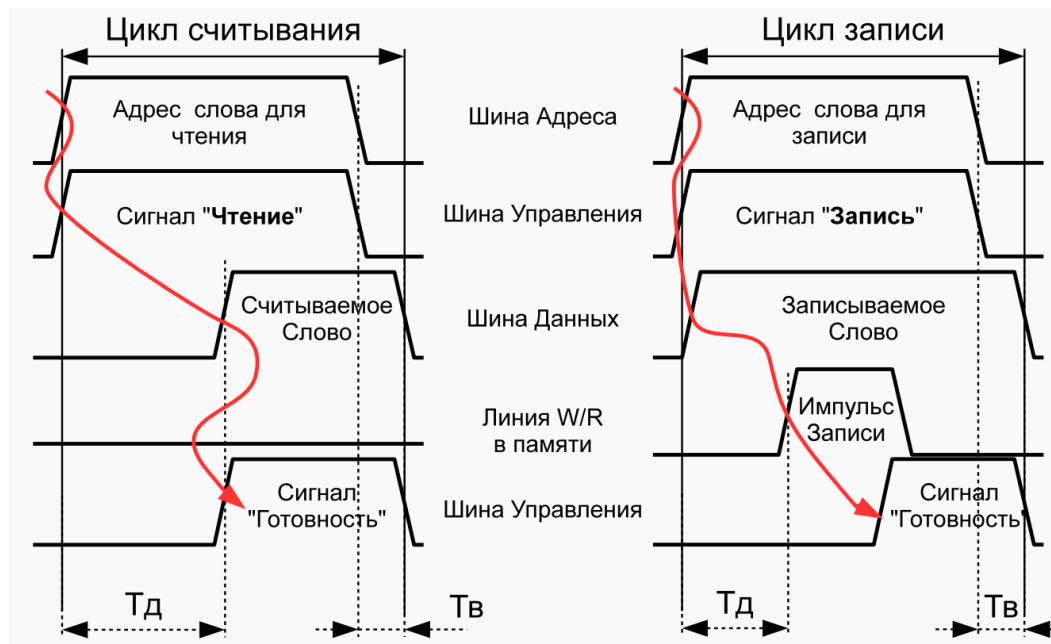
Соответственно каждый банк представляет из себя вот такую матрицу. Подобная конструкция памяти позволяет значительно упростить её схему.

Мультиплексор работает таким образом, что на вход подаётся сигнал, который указывает, какой конкретно вход нужно пустить на выход мультиплексора и так далее.

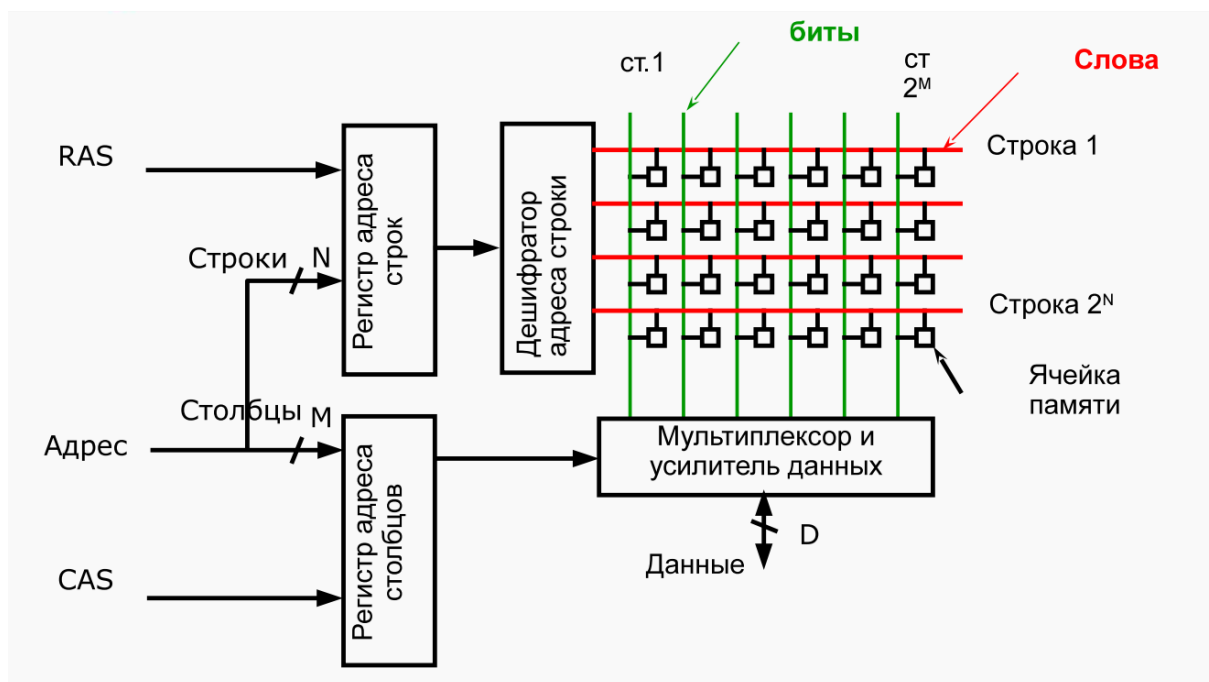
Временные диаграммы

Важно отметить, что линия  $W/R$  не на шине, а внутри памяти. По сигналу чтения срабатывают дешифраторы.

- $T_d$  – время доступа к памяти. То есть это промежуток времени от начала запроса данных до получения первых данных.
- $T_v$  – время восстановления памяти. То есть это промежуток времени, за которое память возвращается в исходное состояние.



### Адресуемая память с фиксацией строк и столбцов

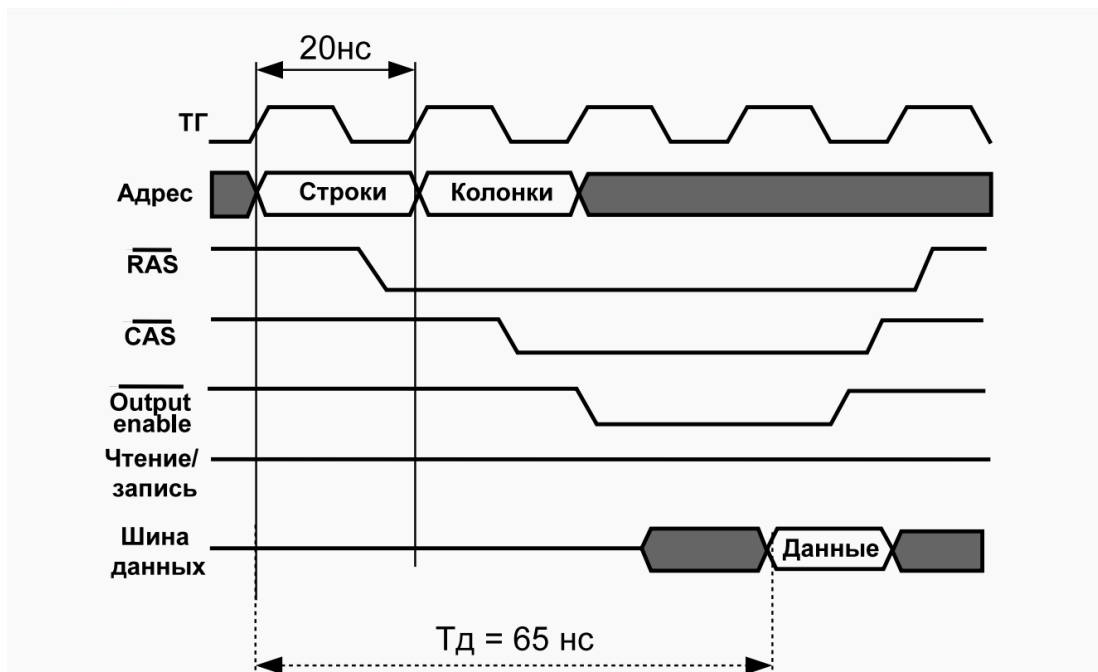


Люди решили передавать адрес не полностью, а разбить его на две части: адрес строки и адрес столбца. Половина адреса передаётся по шине памяти в один такт времени, другая половина – в следующий такт времени. Для этого используются новые управляющие сигналы RAS (Row Address Select) и CAS (Column Address Select). Теперь строка и столбец хранятся в регистрах, мы можем инкрементировать и декрементировать их, чтобы обращаться к последующим ячейкам в банке. Память поначалу будет работать в 2 раза медленнее, чем просто адресуемая память, но обращаться к ячейкам мы будем гораздо быстрее.

## Синхронная память SDRAM

SDRAM (Synchronous Dynamic Random Access Memory) – память, которая уже используется в современных ЭВМ. Появился собственный тактовый генератор, сигналы инвертированы, так как это было удобнее производителям. Output enable – подключение к выходу памяти. Теперь память синхронизируется с частотой процессора.

DDR SDRAM (Double Data Rate SDRAM) – отличается от обычной SDRAM тем, что имеет вдвое больше пропускную способность. Она может принимать и передавать данные и по фронту, и по спаду тактовых импульсов.

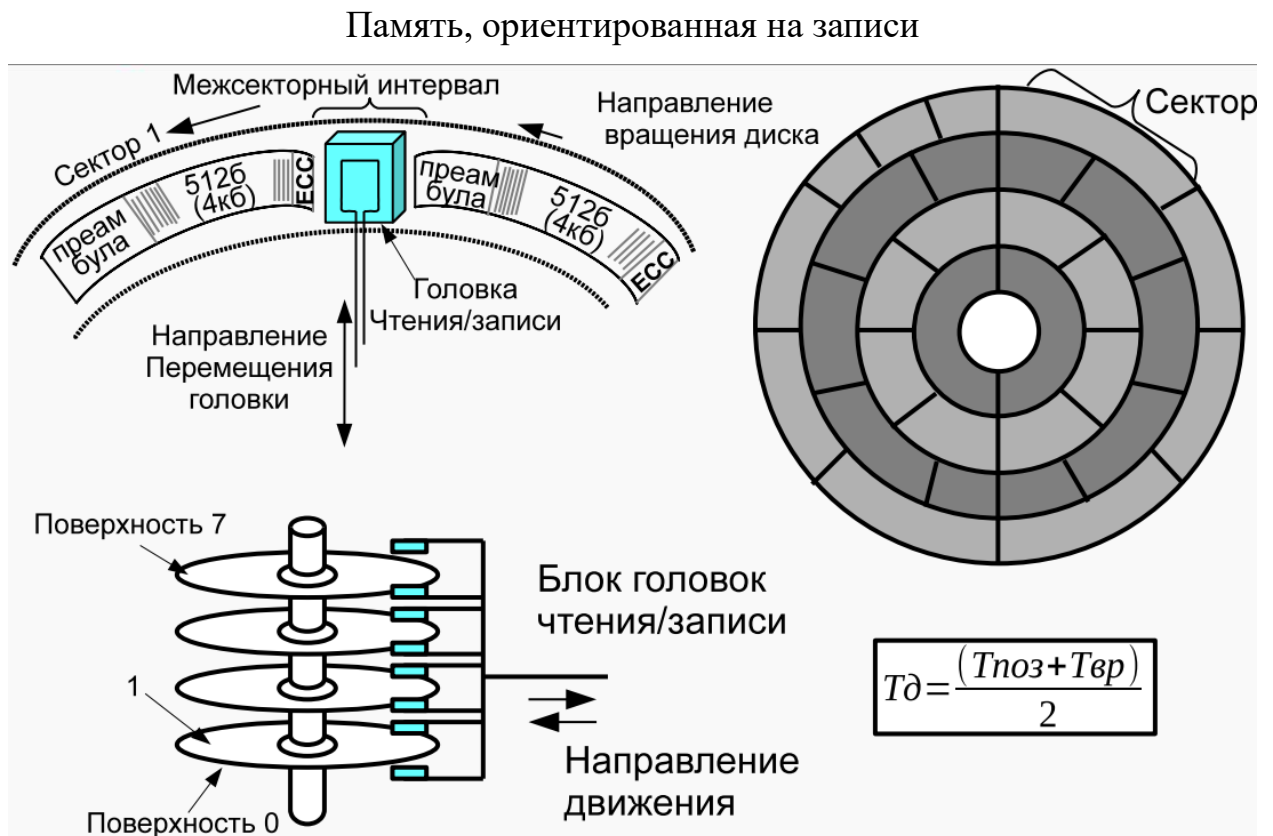


### Конструктивные особенности современной памяти

- Burst mode – пакетный режим, при котором запрос по конкретному адресу возвращаются не только данные, хранящиеся по этому адресу, но и пакет данных по нескольким последующим адресам.
- DDR – передача данных и по фронту и по спаду.
- SPD – чип, содержащий идентификационную информацию. В ней прописано, на какой частоте должна работать память. Чип читается процессором.
- Interleaving – расслоение памяти, при котором соседние по адресам ячейки размещаются в различных модулях памяти, так что появляется возможность производить несколько обращений одновременно. Такое расположение кусков памяти позволяет параллельно читать больше данных.

27. Память, ориентированная на записи (блочная память). Организация дисковой памяти и памяти на магнитных лентах.

Блочная архитектура – адресное пространство разбивается на группы последовательных адресов, и каждая такая группа обеспечивается отдельным банком памяти.



Диски (здесь пойдёт речь о жёстких) – память с блочным доступом. Каждый раз передаётся блок диска.

Блин – это поверхность, на которой расположена информация. Головки читают информацию с намагниченной поверхности. Головка идёт по диску, диск же разбит на секторы, у сектора есть преамбула (чтобы понять, что начался новый сектор), а также есть поле контроля чётности диска (ECC – Error Correction Code). Дорожки имеют разный радиус, следовательно, их угловая скорость разная. Чем дальше от центра вращения, тем больше скорость чтения/записи. Поэтому обычно файлы подкачки располагают в начале диска, чтобы ЭВМ работала быстрее.

Время доступа к памяти считается как средняя сумма время позиционирования (время смещения головки до нужного блока) и время вращения (до нужного сегмента на диске).

## Память с последовательным доступом



Чтобы получить доступ к ячейке памяти  $n$ , нужно пролистать  $n-1$  штук ячеек до неё. Это дорого и медленно с точки зрения времени доступа, однако такие носители позволяют хранить огромные объёмы данных при сравнительной дешевизне.

Примером является ленточный накопитель. Ленту быстро разогнать нельзя, поэтому вращается головка и дорожки записываются под наклоном.

Плётка энергоэффективна – плётке не требуется питание для хранения данных. Она также является очень надёжной, так как вероятность возникновения ошибки при чтении и записи на 4-5 порядков ниже, чем у жёстких дисков. Плёнка безопасна, так как её необязательно подключать к компьютеру, что обрывает возможность несанкционированного доступа к данным.

Чаще всего ленты используют для хранения каких-то бэкапов, которые целиком пишутся на неё.

## 28. Характеристики запоминающих устройств. Пирамида памяти

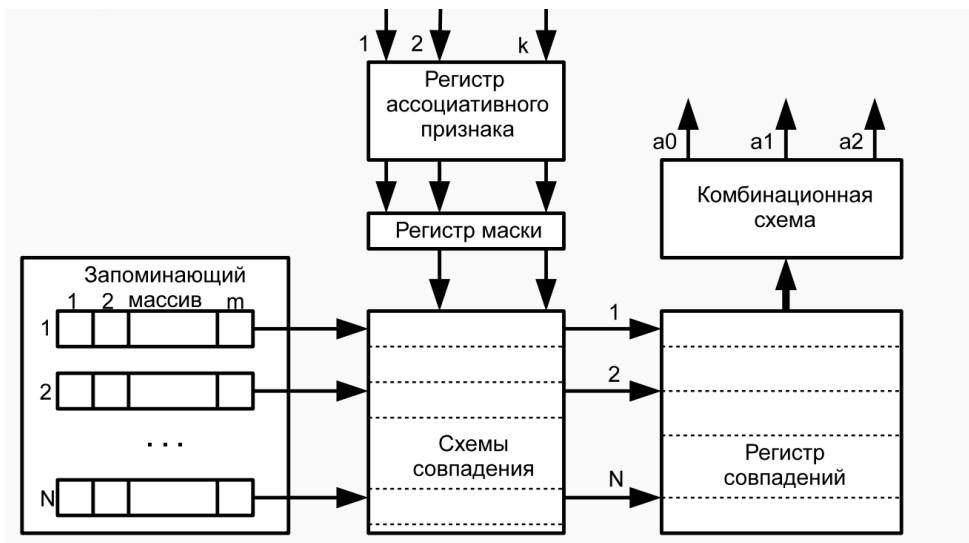
### Характеристика памяти

1. Месторасположение: процессорные, внутренние, внешние.
2. Ёмкость: в метрических (Кило-) и двоичных (Киби-) множителях.
3. Единица пересылки: слово, строка кэша, блок на диске.
4. Метод доступа: произвольный (адресный), ориентированный на записи (прямой), последовательный, ассоциативный
5. Быстродействие и временные отношения:
  - а. Время доступа,  $T_d$  – промежуток времени от начала обращения до передачи информации
  - б. Длительность цикла памяти (время обращения),  $T_{\text{ц}}$  – обратились, получили, произошло восстановление в исходное состояние.
  - с. Время чтения и время записи
  - д. Время восстановления памяти,  $T_v$  – промежуток времени, за которое память возвращается в исходное состояние.
  - е. Скорость передачи информации. Используется единица измерения БОД – бит в секунду.
6. Физический тип и особенности
7. Стоимость (одного байта)

|                 | Объем           | $T_d$     | *       | Тип       | Управл.    |
|-----------------|-----------------|-----------|---------|-----------|------------|
| CPU             | 100-1000 б.     | <1нс      | 1с      | Регистр   | компилятор |
| L1 Cache        | 32-128Кб        | 1-4нс     | 2с      | Ассоц.    | аппаратура |
| L2-L3 Cache     | 0.5-32Мб        | 8-20нс    | 19с     | Ассоц.    | аппаратура |
| Основная память | 0.5Гб-4ТБ       | 60-200нс  | 50-300с | Адресная  | программно |
| SSD             | 128Гб-1Тб/drive | 25-250мкс | 5д      | Блочн.    | программно |
| Жесткие диски   | 0.5Тб-4Тб/drive | 5-20мс    | 4м      | Блочн.    | программно |
| Магнитные ленты | 1-6Тб/к         | 1-240с    | 200л    | Последов. | программно |

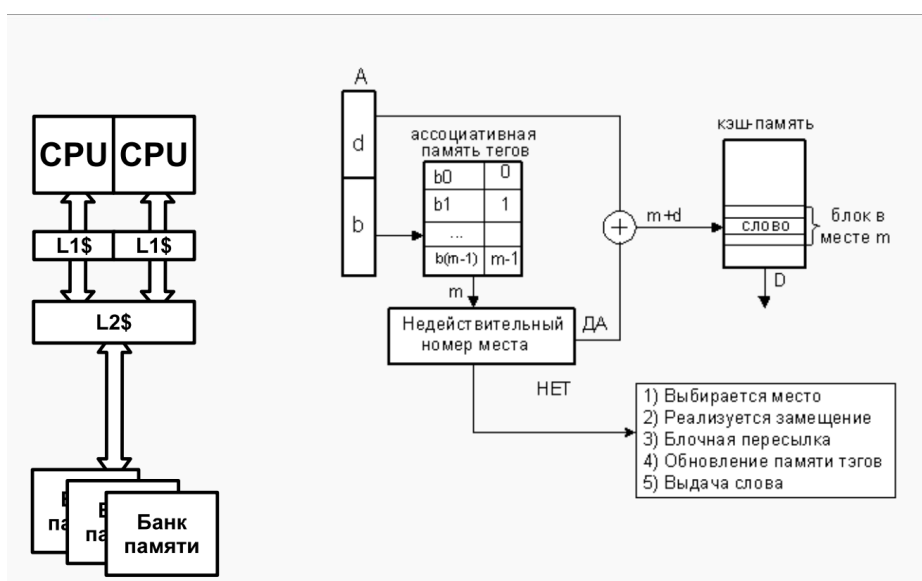
29. Ассоциативная память, Кэш-память. Влияние промахов кэш-памяти на производительность.

Ассоциативная память



Ассоциативная память построена по иному принципу, чем адресуемая память. Она выбирает информацию не по адресу, а по нужному нам признаку. То есть у нас есть регистр ассоциативного признака, а каждая ячейка памяти содержит в себе схему сравнения (компаратор), то есть всё, что поступает на неё, сравнивается на наличие необходимого содержимого и те ячейки, которые совпадают, передаются при помощи регистров сравнения в дальнейшую комбинационную схему. Подобным образом устроена кэш-память. За счёт того, что у каждой ячейки есть свой компаратор, такая память довольно дорогая.

Кэш-память



Кэш-память используется для уменьшения времени доступа к данным. Она хранит копии часто используемых данных из основной памяти. Например,

список инструкций, которые могут храниться на так называемом нулевом уровне кэша. Структура кэш-памяти выглядит следующим образом:

1. Кэш-контроллер – управляет работой кэш-памяти, загружает данные и оперативной памяти и возвращается модифицированные данные обратно.
2. Процессор – запрашивает данные и кэш-контроллера.
3. Оперативная память – хранит все данные, к которым обращается процессор.

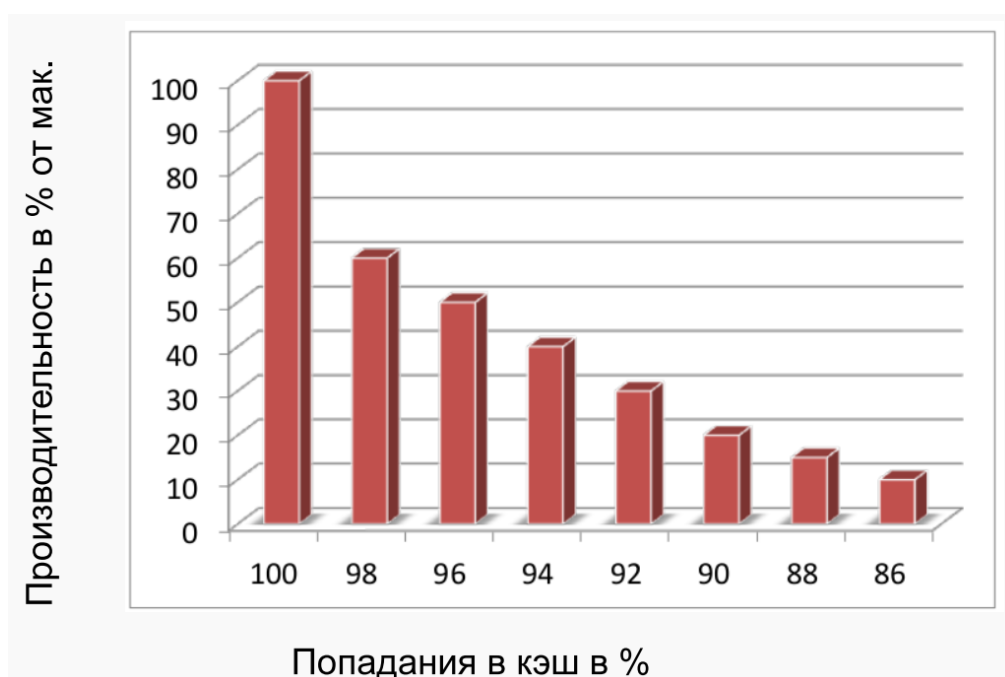
Адрес бьётся на некоторое количество частей, которые делятся на тег и просто адрес. В кэше нужные данные выбираются по тегу, и уже благодаря нему кэш возвращает искомую строку. Структура записи кэша выглядит следующим образом:

| Блок данных | Тэг | Бит актуальности |
|-------------|-----|------------------|
|-------------|-----|------------------|

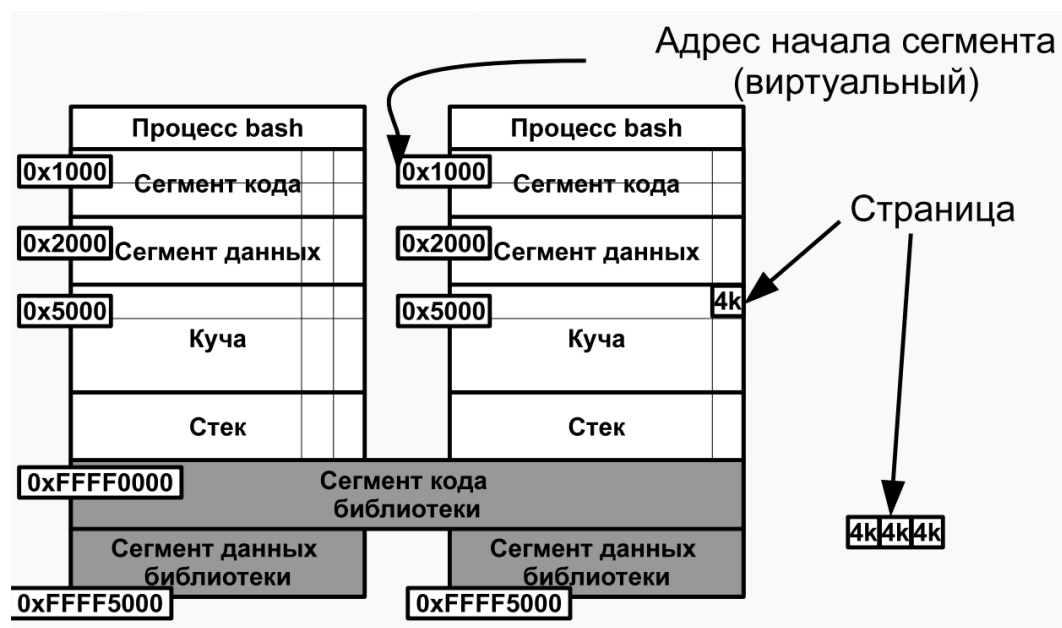
При доступе процессора к кэшу сначала производится проверка, если ли искомые данные. Для этого производится сравнение адреса запроса со значениями всех тэгов кэша, в которых эти данные могут храниться. Если они есть, то данные извлекаются, то есть происходит кэш-попадание. Иначе происходит кэш-промах, и кэш-контроллер запрашивает данные уже из оперативной памяти компьютера.

Каждая строка – это группа ячеек памяти, которые содержит данные, организованные в кэш-линии. Для большинства процессоров x86 размер кэш-линии составляет 64 байта.

Влияние промахов кэш-памяти



30. Предназначение и организация виртуальной памяти. Сегментно-страничная организация. Устройство управление памятью (MMU), буфер трансляции (TLB).



Виртуальная память – это технология управления памятью компьютера, которая создаёт у каждого пользовательского процесса иллюзию того, что ему единолично предоставлен большой, непрерывный и однородный массив адресов (виртуальное адресное пространство), независимый от реального объёма физической памяти. Виртуальная память нужна для:

1. Изоляция и безопасность процессов. Процессы не могут обратиться к памяти друг друга или ядру ОС напрямую. Если один процесс упадёт из-за ошибки адресации, остальные процессы продолжат свою работу
2. Абстракция от физических адресов. Компилятору и программисту не нужно знать, сколько физической памяти установлено в системе и по каким адресам она распаяна по плате. Программа должна рассчитывать на стандартный диапазон адресов.
3. Эффективное использование ОЗУ. Неиспользуемые на текущий момент файлы можно временно выгрузить на жёсткий диск в файл подкачки, освободив физическую ОЗУ для активных задач.
4. Упрощение совместного использования памяти. Разные процессы могут использовать одни и те же разделяемые библиотеки, которые физически выгружены в ОЗУ в единственном экземпляре, но отображаются в адресном пространстве каждого процесса.

## Сегментно-страничная организация памяти

Сегментная организация виртуальной памяти (логическое деление) – вид организации, при котором виртуальное пространство делится на части произвольного размера – сегменты. Такой механизм позволяет разделять память на логические блоки. Как пример: сегмент кода, сегмент данных, сегмент стека. У каждого сегмента есть свои права доступа для пользователя или процесса.

Страничная организация виртуальной памяти (физическое деление) – и физическая, и виртуальная память делится на жёсткие блоки фиксированного размера – страницы. Виртуальная страница отображается на физическую страницу в ОЗУ.

В современных системах общего назначения чаще всего используют как раз гибрид вышеописанных организаций памяти.

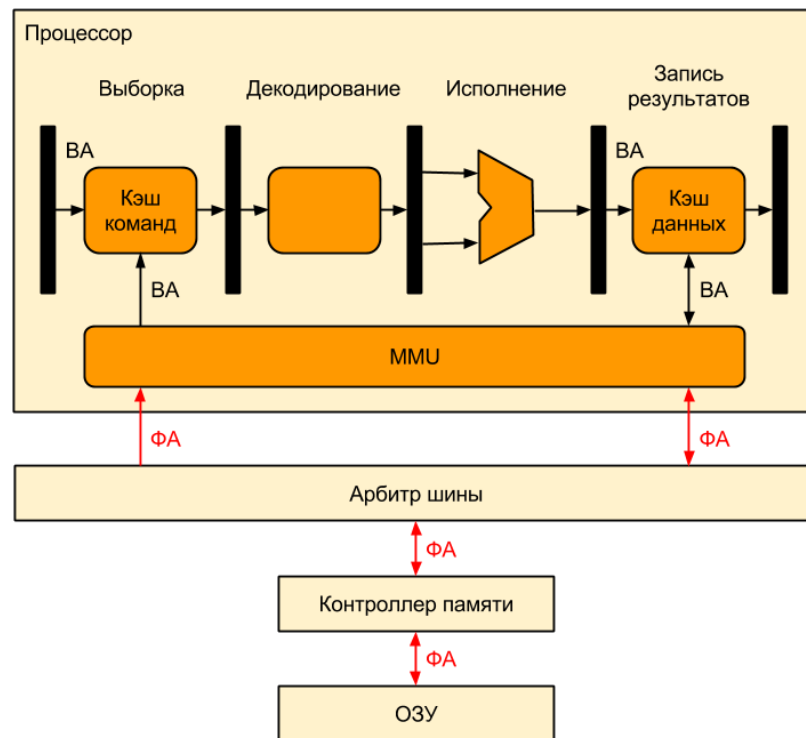
Принцип работы основан на том, что сначала виртуальный адрес, генерируемый процессором, проходит через блок сегментации. Он проверяет права доступа к сегменту и преобразует логический адрес в линейный (промежуточный) адрес.

Полученный линейный адрес передаётся в блок страничной организации, который делит его на три части:

1. Номер каталога страниц (Page Directory Index) – для многоуровневых таблиц.
2. Номер таблицы страниц (Page Table Index)
3. Смещение внутри страниц (Offset)

По вышеописанным индексам находится нужная запись таблицы страниц (PTE), где хранится базовый индекс физической страницы. К базовому индексу смещения прибавляется смещение, в результате чего получается физический адрес в ОЗУ.

## MMU (Memory Manager Unit)

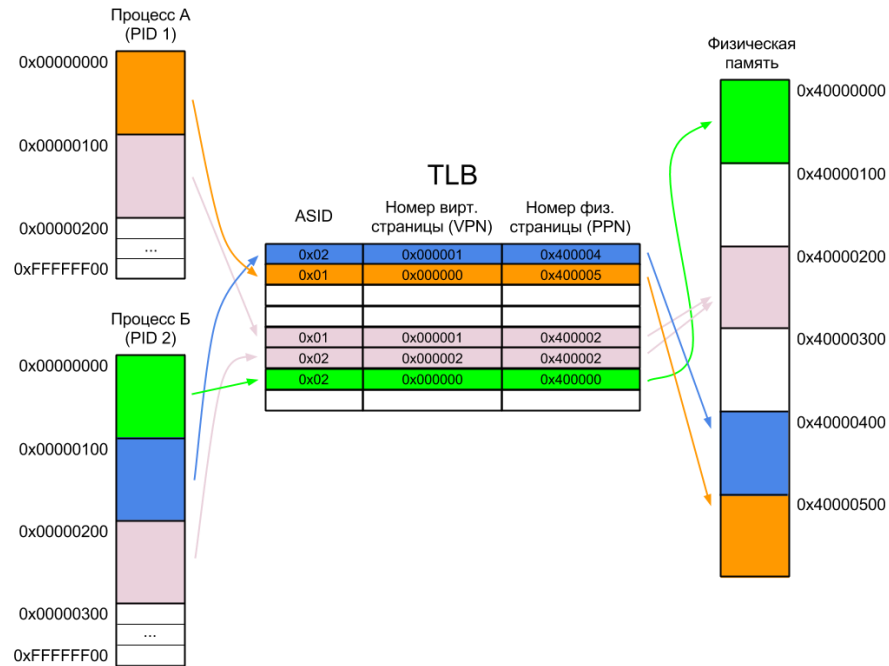


MMU – аппаратный блок, расположенный непосредственно между ядром CPU и системной шиной памяти. Блок используется для:

- Преобразования виртуальных адресов команд и данных, с которыми работает процессор, в реальные физические адреса ОЗУ.
- Проверки наличия прав у процесса на выполнение определённой операции, например, запись в страницу, помеченную как “только для чтения”. Если происходит нарушение, то MMU генерирует аппаратное исключение (Page Fault).
- Управления кэшированием страниц: в записях таблиц страниц хранятся биты, указывающие, можно ли кэшировать данную область или нет.

Если страничное управление выключено, то MMU просто приравнивает виртуальный адрес к физическому.

## TLB (Translation Lookaside Buffer)



Поскольку таблицы страниц хранятся в ОЗУ, для каждой трансляции адреса процессору пришлось бы делать несколько дополнительных обращений к медленной памяти. Для решения данной проблемы внутри MMU встроен TLB – ассоциативный буфер трансляции адресов.

TLB – это специализированная, очень быстрая аппаратная кэш-память, которая хранит соответствия между номерами виртуальных страниц и номерами физических страниц для самых последних обращений к памяти. Когда виртуальный адрес необходимо преобразовать в физический адрес, то вначале просматривается TLB. Если совпадение найдено (TLB hit), то возвращается физический адрес, и доступ к памяти может продолжаться. Если совпадение не найдено (TLB miss), то просматривается вся таблица страниц. Если сопоставление существует, то оно записывается обратно в TLB, так как аппаратное обеспечение обращается к памяти через TLB в системе виртуальной памяти.

*31. Сетевые технологии. Понятие сети ЭВМ, классификация компьютерных сетей. Сообщение и пакет. Модель взаимодействия открытых систем.*

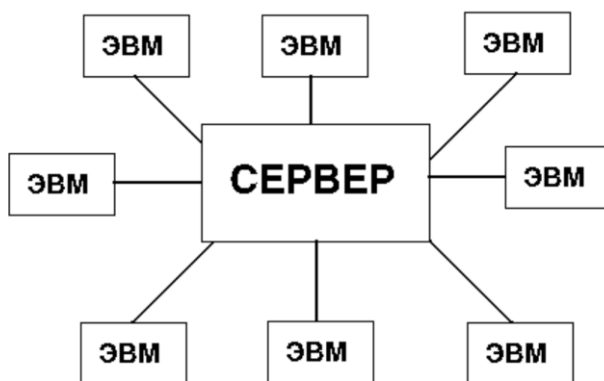
Под сетью ЭВМ подразумевается соединение двух или более ЭВМ с целью совместного использования их ресурсов (процессоров, устройств памяти, устройств ввода-вывода, данных).

По степени охвата территории можно выделить:

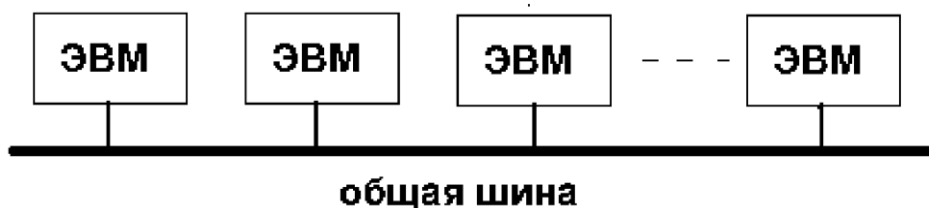
1. Локальные (местные) – в пределах одного учреждения, помещения или в радиусе 1 километра.
2. Региональные – внутри населённого пункта, района
3. Национальные – внутри государства
4. Глобальные

По степени доступности отмечают корпоративные и общедоступные сети.

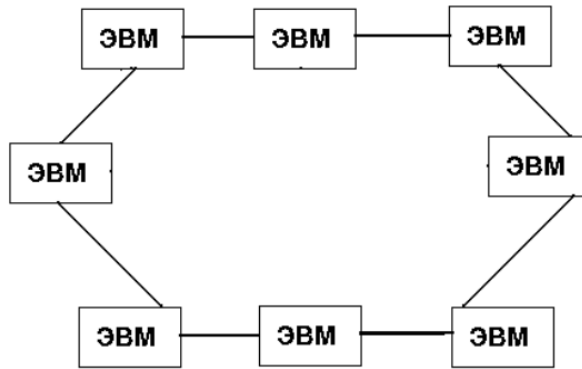
По топологии:



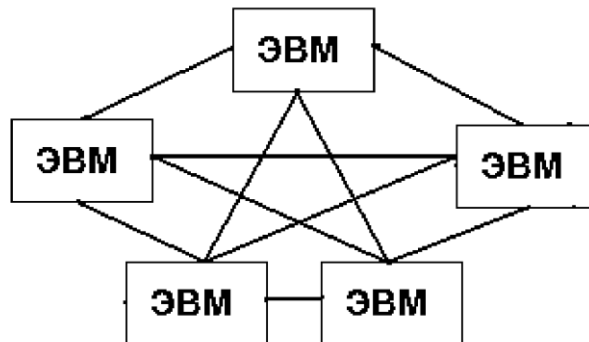
1) Звезднообразная топология – при такой топологии обмен данными между ЭВМ происходит через более мощную ЭВМ – сервер. Недостатком такой сети является низкая живучесть, ведь если сервер упадёт, то упадёт вся система. Однако из плюсов можно отметить простоту и дешевизну всей схемы.



2) Топология с общей шиной – при таком способе обмен данными происходит через общую шину, которая, используя механизм прерывания, может захватить тот или иной компьютер. Характерной особенностью здесь является отсутствие сервера. Часто используется в домашних сетях.

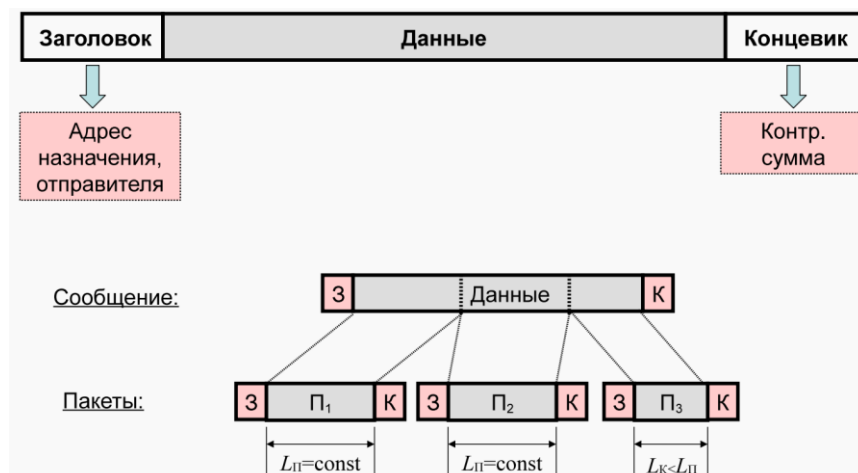


3) Кольцевая топология – в этой структуре каждая ЭВМ, используя механизм прерывания, работает в качестве ретранслятора. Важно, что система более живучая, чем с общей шиной, ибо при обрыве связи между соседними ЭВМ, сеть продолжает функционировать.



4) Полная топология – соединение ЭВМ “каждая с каждой”. Позволяет получить хоть и дорогую сеть, но при этом самую живучую.

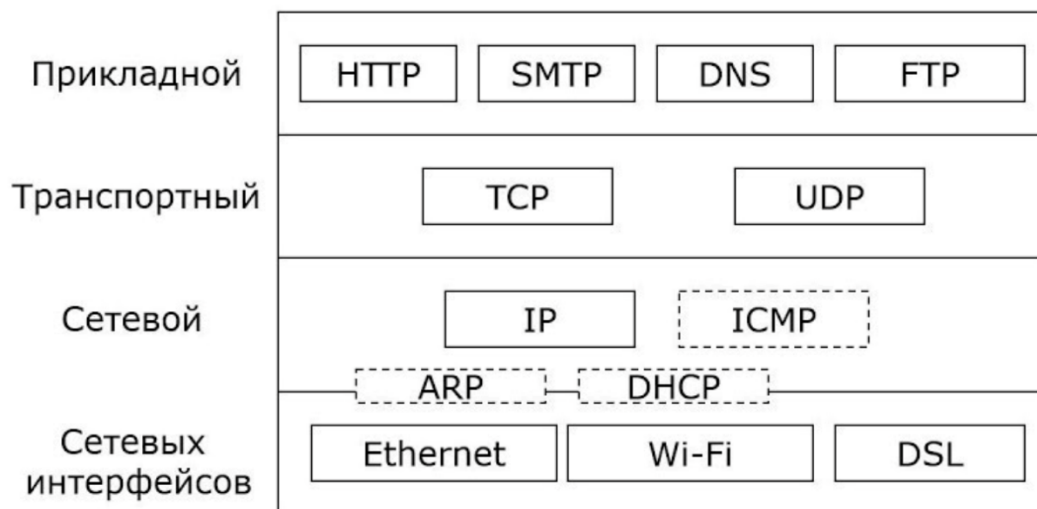
- Сообщение – цифровые данные определённого формата, предназначенные для передачи
- Пакет – это определённым образом оформленный блок данных, передаваемый по сети. Часто состоит из заголовка и полезной нагрузки (payload)



## Модель открытых систем OSI

| Модель открытых систем OSI |                                                     |                                             |                                                          |                                         |                                                                      |
|----------------------------|-----------------------------------------------------|---------------------------------------------|----------------------------------------------------------|-----------------------------------------|----------------------------------------------------------------------|
| Уровень (layer)            |                                                     | Тип данных                                  | Функции уровня                                           | Особенность адресации                   | Примеры протоколов                                                   |
| Уровни хоста (узла)        | 7. Прикладной (application)                         | Данные (строки из байтов)                   | Доступ к сетевым службам                                 | URL                                     | HTTP(S), FTP(S), RPC, POP3                                           |
|                            | 6. Представительский (представления) (presentation) |                                             | Представление (кодировка) и шифрование данных            |                                         | ASCII, EBCDIC                                                        |
|                            | 5. Сеансовый (session)                              |                                             | Управление сеансом связи                                 |                                         | PAP                                                                  |
|                            | 4. Транспортный (transport)                         | Сегменты (segment) / Дейтаграммы (datagram) | Прямая связь между конечными пунктами и надёжность       | Порт                                    | TCP, UDP, SCTP, PORTS                                                |
| Уровни связи (сети)        | 3. Сетевой (network)                                | Пакеты (packet)                             | Определение маршрута и логическая адресация              | IP-адрес                                | IPv4, IPv6, IPsec, AppleTalk                                         |
|                            | 2. Канальный (data link)                            | Биты (bit) / Кадры (frame)                  | Физическая адресация                                     | MAC-адрес (физический адрес компьютера) | PPP, IEEE 802.22, Ethernet, DSL, ARP, L2TP, сетевая карта.           |
|                            | 1. Физический (physical)                            | Электрические сигналы, Биты (bit)           | Работа со средой передачи, сигналами и двоичными данными |                                         | USB, кабель ("витая пара", коаксиальный, оптоволоконный), радиоканал |

32. Модель TCP/IP: передающая среда, канальный и сетевой уровень. Адресация, передача и маршрутизация пакетов.



Модель TCP/IP – это многоуровневая структура, или же сетевой стек, который стандартизирует обмен информации в интернете. Набор интернет-протоколов описывает следующие правила взаимодействия устройств:

1. В стеке протоколы нижних уровней работают непосредственно с сетевым оборудованием, в то время как протоколы верхних уровней взаимодействуют с пользовательскими приложениями.
2. При отправке данные проходят вниз по уровням стека, при получении они поднимаются вверх в обратном порядке.

Если в модели OSI 7 уровней абстракций, то в модели TCP/IP уже 4 уровня:

1. Канальный уровень
2. Межсетевой/сетевой уровень
3. Транспортный уровень
4. Прикладной уровень

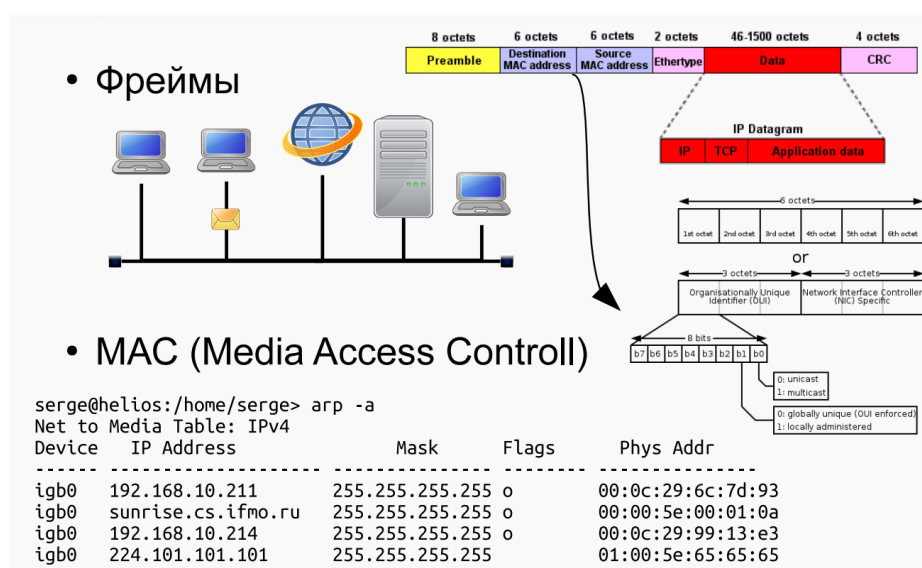
#### Уровень передающей среды (Физический уровень)

Передающая среда определяет физическую основу для работы канального уровня. В зависимости от используемого сетевого оборудования выделяют следующие типы среды:

1. Коаксиальный кабель – является устаревшим вариантом (используется, например, при передаче сигнала с тарелки в телевизор на даче). Делится на тонкий (рассчитан на передачу до 50 метров) и толстый (до 500 метров).

2. Витая пара – состоит из медных проводов, разделяя на категории в зависимости от пропускной способности. Такой проводник может передавать до 100 гбит/с до 15 метров без потерь. Подключается через специальные разъёмы RJ45
3. Оптика – одно из самых распространённых решений для передачи данных на большие расстояния. Позволяет передавать данные на расстояние до 80 километров в режиме одного луча.
4. Беспроводная среда – в эту категории входят Wi-fi, 3g, 4g и так далее. Обеспечивает пропускную способность до 15 гбит/с.

### Канальный уровень



Канальный уровень отвечает за физические и логические соединения между устройствами в пределах одной локальной сети.

Единица данных здесь является кадр или фрейм. Заголовок кадра содержит MAC-адреса источника и назначения, а также тип передаваемых данных в кадре данных.

Адресация по отношению к узлу в локальной сети производится при помощи MAC-адреса (Media Access Control), состоящий из 6 байт, уникального для каждого узла в рамках одной локальной сети. Для его записи обычно используется 16-ричная форма.

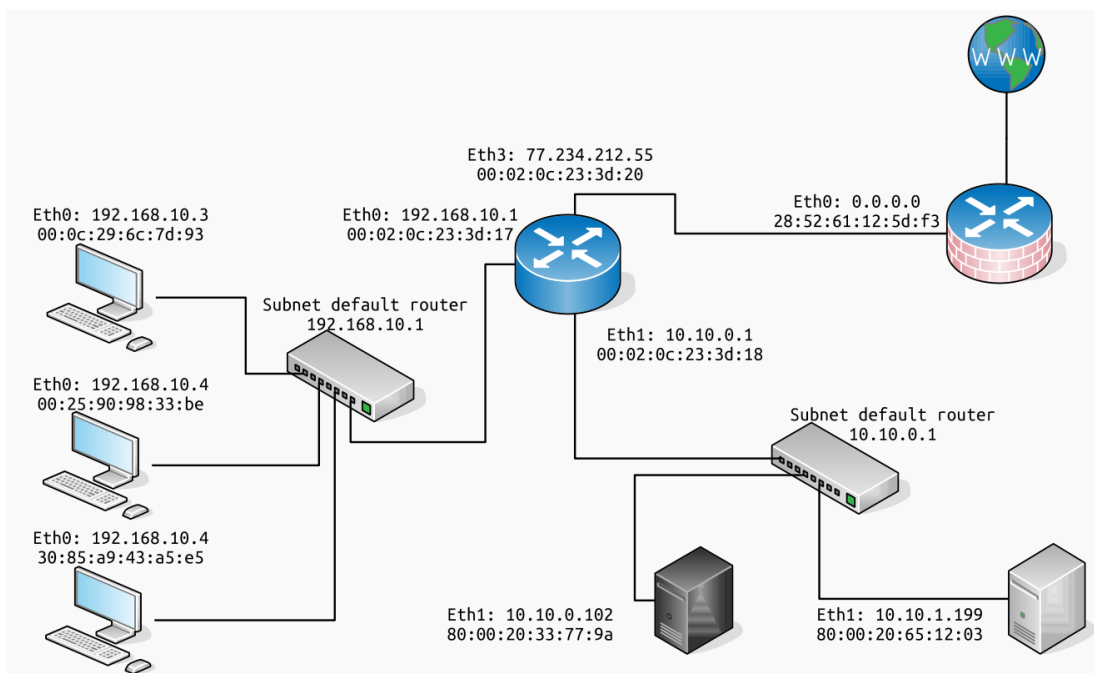
XX:XX:XX:XX:XX:XX

Первые два бита в первом байте задают уникальность адреса. Уникальность адресации адаптеров обеспечивается специальным соглашением. Байты 4-6 заполняются изготовителем – на нём лежит ответственность за их уникальность. Эта информация, кстати, может рассматриваться как серийный номер платы.

Базовой технологией и протоколом передачи на канальном уровне является Ethernet.

Из ключевых оборудования данного уровня можно отметить сетевой концентратор (или же хаб), который принимает на одном порту фреймы и ретранслирует сигнала на всё остальные порты, не анализируя сами данные.

### Сетевой уровень и IP-адресация



Сетевой уровень обеспечивает взаимодействие между различными подсетями и выход данных за пределы локальной сети. Основной протокол, использующийся на данном уровне, называется IP (Internet Protocol).

Единицей пересылки данных является пакет, который при передаче инкапсулируется внутри кадра канального уровня.

Адресация на данном уровне осуществляется при помощи IPv4 (IPv6), состоящий из 4 8-ричных чисел, записанных в 10-ной СС через точки. Например: 198.168.1.1

Структура IP адреса выглядит следующим образом:

1. Сетевой префикс – общая часть для всех устройств в пределах одной подсети.
2. Идентификатор узла (хоста) – уникальный номер устройства в рамках одной сети.

Способы представления сетевого префикса:

1. CIDR (бесклассовая адресация) – запись IP-адреса со слэшем, после которого указывается количество бит префикса (например, /24)

2. Маска подсети: 32-битное число, где двоичные единицы соответствуют битам сетевого префикса, а нули – битам номера узла

При выполнении операции логического “И” между узлом сети и маской сети получается первый адрес подсети – сетевой идентификатор. Устройства с одинаковым сетевым идентификатором находятся в одной подсети.

### Передача и маршрутизация пакетов

Маршрутизация – это процесс определения пути следования данных между различными сетями через интернет. Алгоритм передачи выглядит следующим образом:

1. Отправитель сравнивает IP-адрес получателя со своей маской подсети
2. Если адрес назначения принадлежит той же подсети, то происходит прямая передача, то бишь данные передадутся на канальном уровне.
3. Если же получатель находится в другой подсети, отправитель передаёт пакет в роутер (маршрутизатор) – устройство, связывающее подсети.

### Работы маршрутизатора и таблица маршрутизации

Для выбора пути роутеры используют таблицу маршрутизации. Каждая запись в такой таблице содержит:

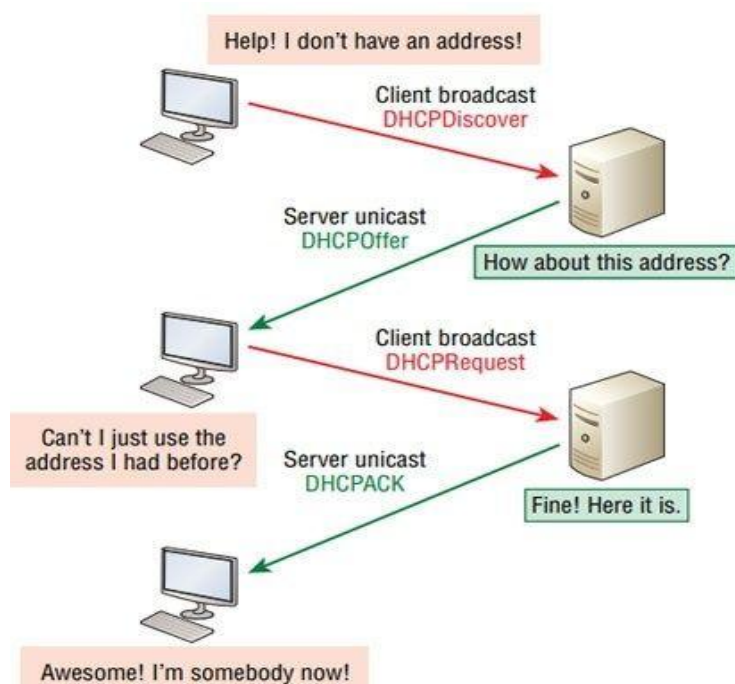
- Сеть назначения и маску подсети
- Адрес следующего узла
- Выходной интерфейс
- Метрику (приоритет маршрута)

Пакет сначала поступает в роутер, где извлекается его IP-заголовок. IP-адрес назначения сопоставляется с записями в таблице маршрутизации. Если подходящий путь найден, то пакет пересылается на соответствующий интерфейс следующему узлу. Если маршрут отсутствует, то пакет отправляет по маршруту по умолчанию. Если такого маршрута нет, пакет отбрасывается и отправителю уходит уведомление о недостижимости узла.

На каждом транзитном уровне (роутере) пакет претерпевает изменения на нижних уровнях. Канальный уровень меняется таким образом, что в кадре перезаписывается MAC-адрес текущего роутера на MAC-адрес следующего роутера или же конечного узла.

Сетевой уровень остаётся неизменным: исходные IP-адреса отправителя и получателя в заголовке IP-пакета сохраняются на протяжении всего пути.

33. Модель TCP/IP: выделение адресов (DHCP), сервисы имён, транспортный и прикладной уровни.



Протокол DHCP (Dynamic Host Configuration Protocol) позволяет пользовательским ЭВМ при подключении к сети в автоматическом режиме получать необходимые настройки от DHCP-сервера: IP-адрес, маску подсети, основной шлюз и стандартный DNS-сервер.

Процесс получения настроек со стороны клиента выглядит следующим образом:

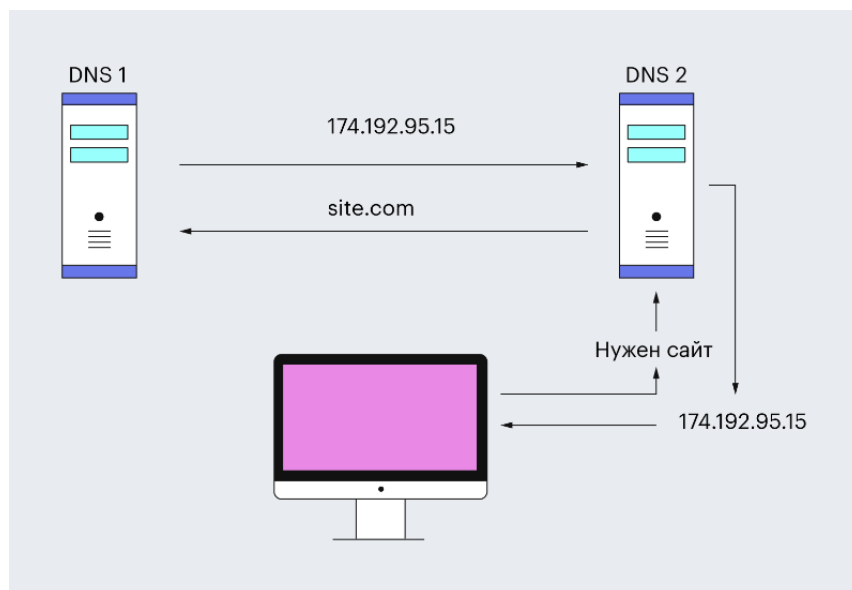
1. **DHCPDISCOVER:** клиент отправляет в сеть широковещательный запрос (его получают абсолютно все узлы локальной сети), чтобы обнаружить доступные DHCP-сервера.
2. **DHCPOFFER (предложение):** DHCP-сервер, получив запрос, отвечает клиенту предложением выдать определённый IP-адрес и сопутствующие параметры.
3. **DHCPREQUEST (Запрос):** Если клиент соглашается на предложение, то он отправляет запрос DHCPREQUEST. Данный пакет отправляется как широковещательный. Это необходимо для того, чтобы уведомить оставшиеся DHCP-серверы в сети, что их предложения отклонены и их услуги не требуются.
4. **DHCPACK (Подтверждение):** Выбранный сервер резервирует IP-адрес за клиентом и окончательно подтверждает право на его использование, отправляя пакет подтверждения DHCPACK.

## DNS (Domain Name System)

Для удобства пользователей TCP/IP предоставляет возможность заменить сложные для запоминания числовые IP-адреса на текстовые аналоги – доменными именами. За эту задачу отвечает служба DNS.

Принцип работы DNS основан так:

1. Когда устройству нужно узнать IP-адрес хоста по его доменному имени, оно отправляет запрос локальному DNS-серверу.
2. Если у локального DNS сервера есть запись о данном соответствии, то он отправляет IP-адрес клиенту
3. Если информации на сервере нет, то DNS-сервер обращается к вышестоящему DNS-серверу. На самом вершю иерархии стоят корневые DNS-серверы.
4. Полученную от вышестоящего сервера информацию локальный сервер не только передаёт клиенту, но и кэширует у себя. При повторном запросе к этому же доменному имени, локальный сервер сможет сразу выдать адрес.



### Транспортный уровень

Транспортный уровень обеспечивает надёжную доставку данных между приложениями на передающем и принимающем устройстве. Он выполняет три важные функции:

1. Отслеживание индивидуальных сеансов общения: (подобный прикол есть только у TCP) перед отправкой данных узел-инициатор устанавливает соединение с получателем, чтобы убедиться, что последний готов к приему. Все данные, передающие в рамках данной сессии, которая заканчивается по завершении обмена.

2. Сегментация данных: большие объёмы данных разбиваются на более мелкие блоки – сегменты – для их индивидуальной отправки по сети. На стороне получателя эти сегменты собираются обратно в исходном порядке.
3. Идентификация приложений: для распределения данных между разными программами используются номера портов. Каждое сетевое приложение регистрирует за собой порт в ОС. Получив данные, транспортный уровень считывает номер порта и перенаправляет данные нужному приложению.

### Прикладной уровень

Прикладной уровень – это верхний уровень модели, представляющий собой набор разнообразных протоколов. Через эти протоколы пользователи и системные службы получают непосредственный доступ к сетевым ресурсам: файлам, принтерам, сайтам или электронной почте. На данном уровне единица данных считается сообщением.

В качестве примеров протоколов можно привести HTTP(S), которым пользуются веб-серверы для получения и обновления веб-контента.

*34. Интерфейсы ввода-вывода. Контроллеры внешних устройств. Уровни стандартизации, сопряжения с системной шиной, циклы обмена. Регистры контроллера.*

### Интерфейсы ввода-вывода

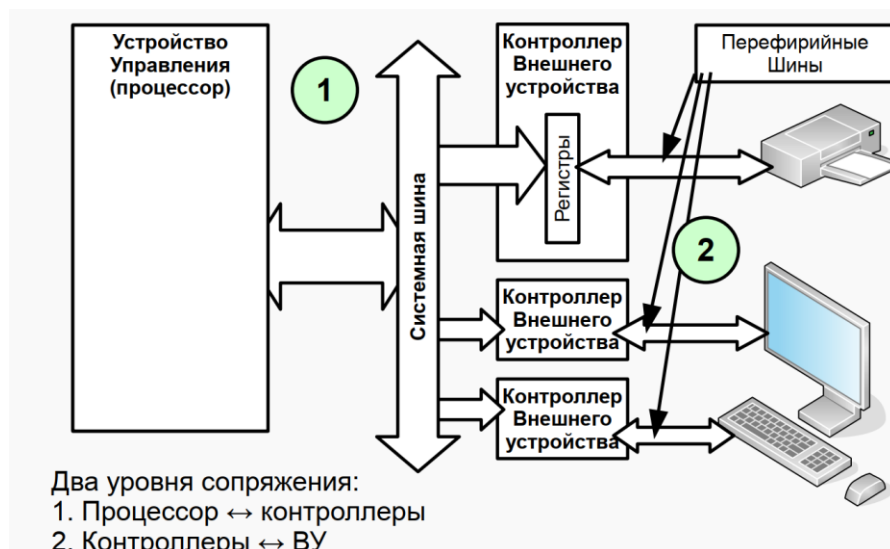
Интерфейсы определяют конкретные детали обмена, например, частота, набор каналов передачи, способ кодирования, команды, представления данных, набор данных и их последовательность обмена.

Требуется аппаратная и программная реализация. Нуждаются в точной спецификации и стандартизации. Стороны должны однозначно интерпретировать детали обмена.

### Уровни стандартизации

- Логическое подключение (все алгоритмы подключения понимаются системой) – в качестве примера можно привести подключение вилки в розетку (ток идёт и всё хорошо).
- Физический параметр сигнала (уровень напряжения, тока)
- Конструктивные особенности (физическая реализация разъёмов, портов и так далее)

### Уровни сопряжения



1. Первый уровень сопряжения: соединение процессора с КВУ. На данном уровне какая-никакая стандартизация присутствует, например, в x86 процессорах от Intel. По большей части используется PCI Express в современных машинах

2. Второй уровень сопряжения: соединение КВУ с ВУ. Здесь уже присутствует большое количество стандартов, например, интерфейс USB с большим видов штекеров и скорость передачи информации.

Управляющие сигналы позволяют организовать 2 режима обмена информацией с КВУ:

1. Программно-управляемый. Сюда входит асинхронный и управляемый по прерыванию ввод-вывод
2. Прямой доступ к памяти

### Сопряжение с системной шиной

При использовании команд ввода-вывода для обмена с ВУ адрес ВУ передаётся по шине адреса. По этой же шине передаются адреса ячеек памяти.

Информация на шине адреса имеет смысл только при наличии соответствующих управляющих сигналов. Например, сигнал на осуществление ввода или вывода.

При реализации обмена с ВУ нет необходимости в специальных сигналах, которые указывает на то, что в шине адреса расположен адрес ВУ. Для таких адресов выделено отдельное адресное пространство, а внутри КВУ происходит селекция адресов ВУ. Из управляющих сигналов можно только отметить приказ на “запись” или “чтение”, которые и регулируют обмен информацией между ВУ и процессором.

### Циклы обмена

1. Операция вывода: со стороны шины ВУ по адресной шине подаётся адрес искомого ВУ, по шине данных передаётся слово, необходимое для вывода на ВУ, по управляющей шине поступает сигнал “Вывод”. КВУ принимает входные данные и при выполнении операции отправляет сигнал процессору о готовности. Данный сигнал как бы говорит процессору, что нужно снять информацию со всех шин.
2. Операция ввода: со стороны шины ВУ по адресной шине подаётся адрес искомого ВУ, по управляющей шине подаётся сигнал “Ввод”. КВУ принимает сигналы и отправляет данные на ВУ. После КВУ отправляет сигнал о готовности к процессору.

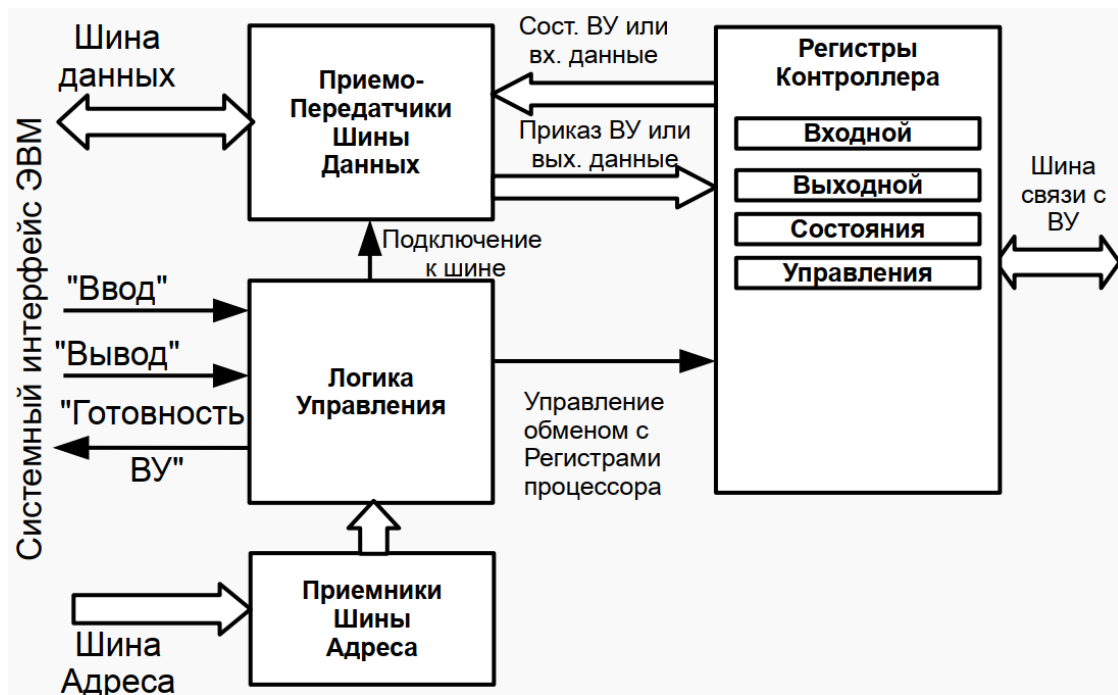
## Регистры контроллеров

В основе контроллера лежит несколько регистров для временного хранения передаваемой информации. Каждый регистр имеет свой адрес (такие регистры называют портами ввода-вывода).

- Регистр входных данных – работает в режиме чтения
- Регистр выходных данных – работает в режиме записи
- Регистр состояния – содержит информации о готовности ВУ к обмену
- Регистр управления – служит для приёма приказов из микроЭВМ для ВУ.

Логика управления выполняет селекцию адресов регистров контроллера, приём, обработку и формирование управляющего сигнала системного интерфейса, обеспечивая обмен информацией между регистрами контроллера и шиной данных системного интерфейса.

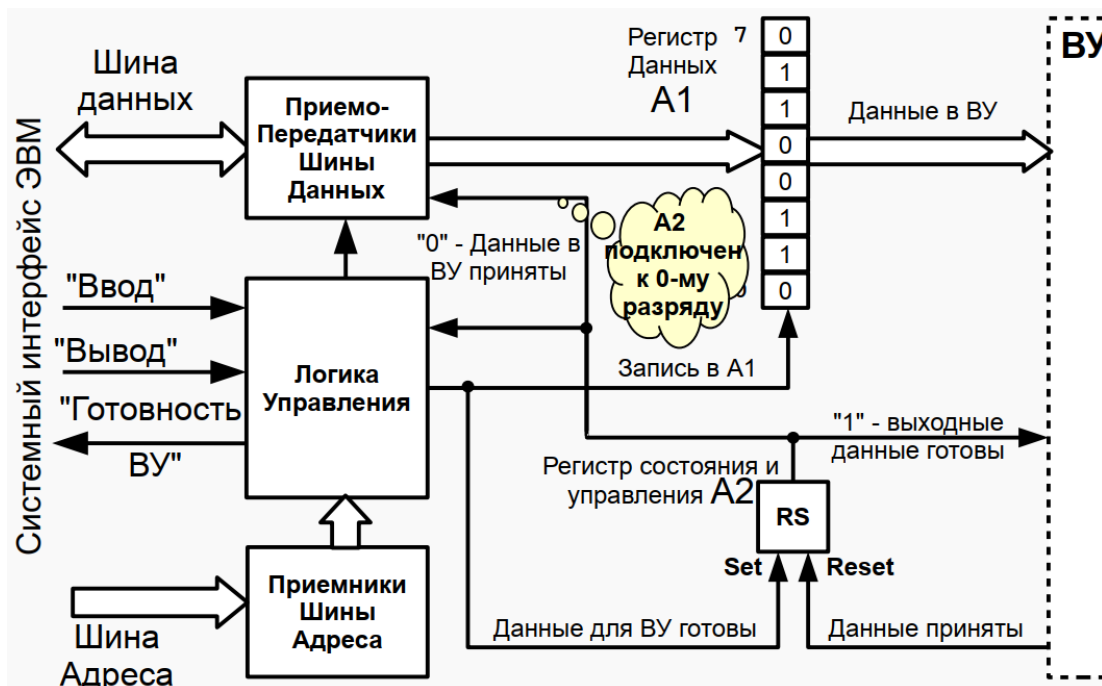
Приёмопередатчики шин адреса и данных используются для физического подключения электронных схем контроллера к шинам системного интерфейса.



### 35. Параллельная передача данных. Контроллеры параллельной передачи и приема.

Параллельная передача данных в ВУ под управлением асинхронного обмена:

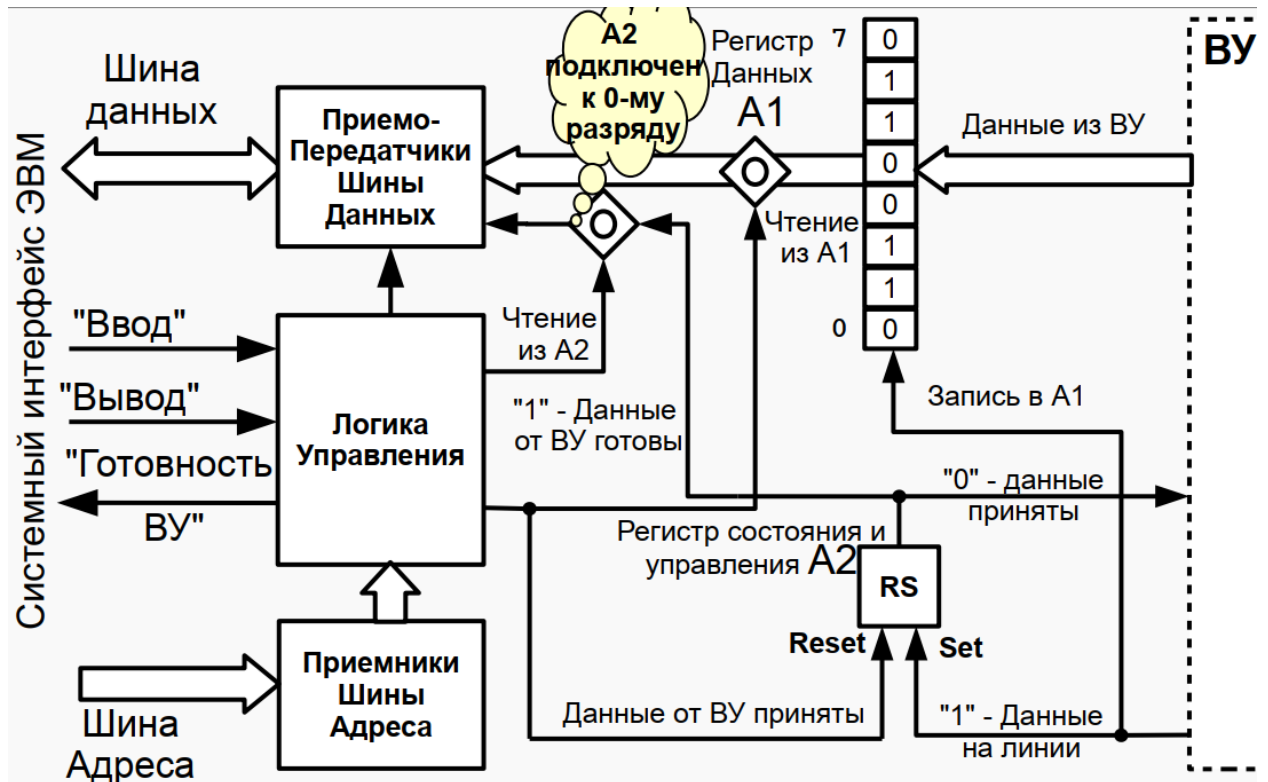
1. Процессор проверяет готовность ВУ к обмену данных.
2. Если ВУ готово к приему данных (логический 0 в регистре A2), то данные передаются из шины данных в регистр данных A1 контроллера. Если ВУ не готово, то идём в 1-ый пункт



При передаче данных в регистр A1, у нас идёт сигнал на установку логической единицы в RS-триггер. Состояние готовности на чтение со стороны процессора сбрасывается и отправляется сигнал к ВУ о готовности данных к выводу. Как только ВУ прочитает данные, отправляется сигнал о сбросе состояния RS-триггера. Логический ноль сигнализирует блоку логики управления, что данные были приняты со стороны ВУ и можно выводить следующую пачку данных

Особенность параллельной передачи данных заключается в том, что слово целиком передаётся на внешнее устройства, а не последовательно бит за бит. В плане реализации КВУ это проще, но с точки зрения количества шин (11 проводов vs 3 провода в последовательном) не очень хорошо.

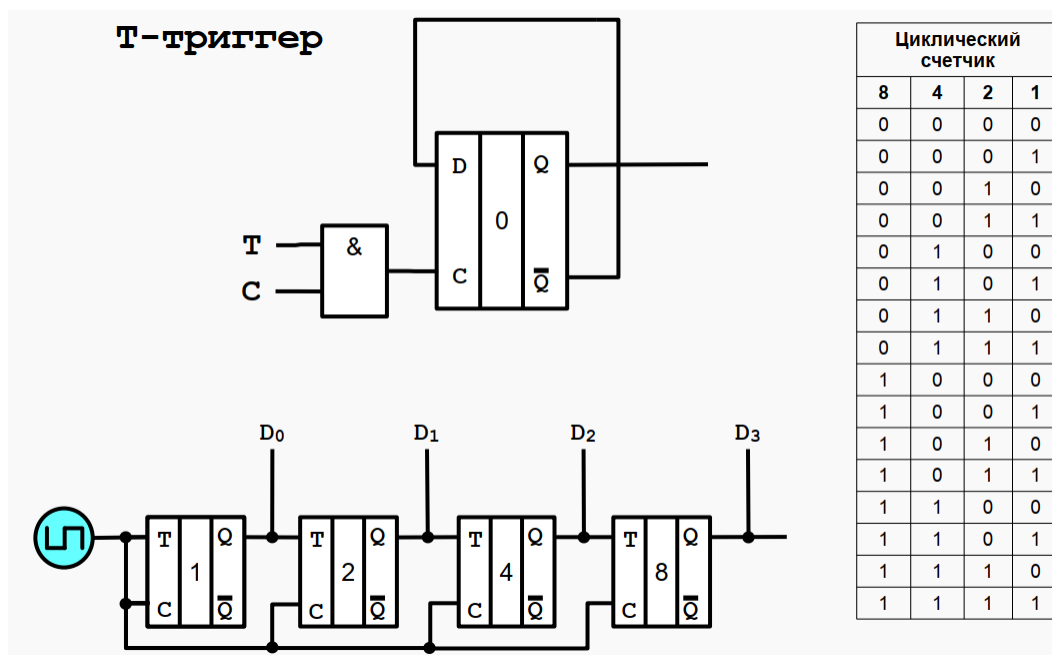
## Контроллер приёма параллельного асинхронного интерфейса.



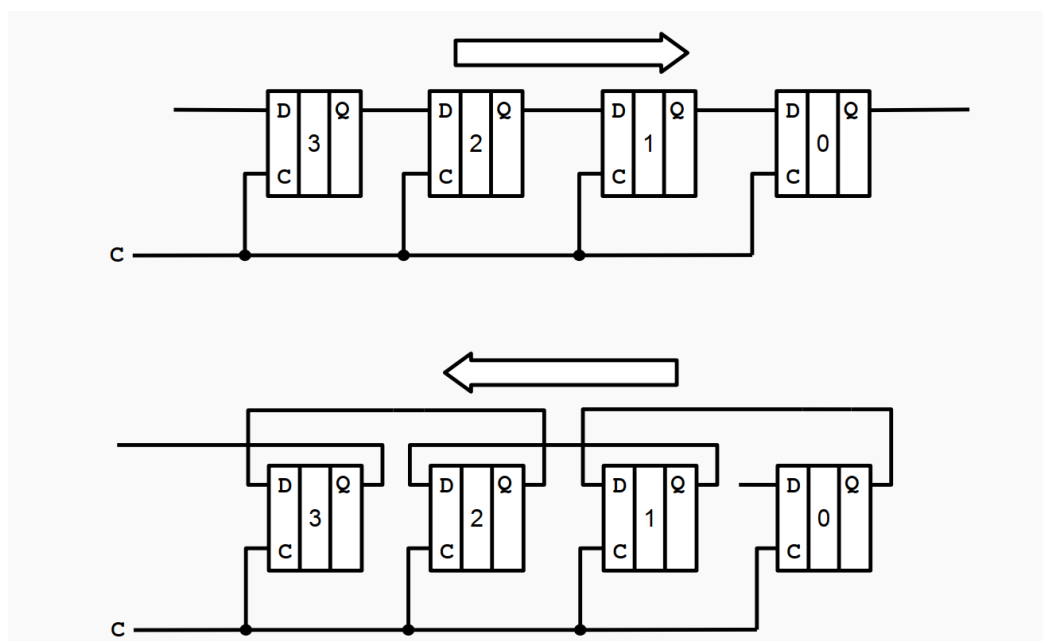
Логика работы такого контроллера противоположна контроллера передачи. Здесь логическая единица в регистре A2 сигнализирует о том, что данные готовы к чтению с КВУ. В блоке логики управления появился отдельный вентиль на чтение регистра состояния, который будет записываться в младший бит в шину данных. Чтобы разобраться в этой штуке, достаточно просто выучить схему.

### 36. Синхронные последовательные интерфейсы. Контроллеры последовательной передачи и приёма.

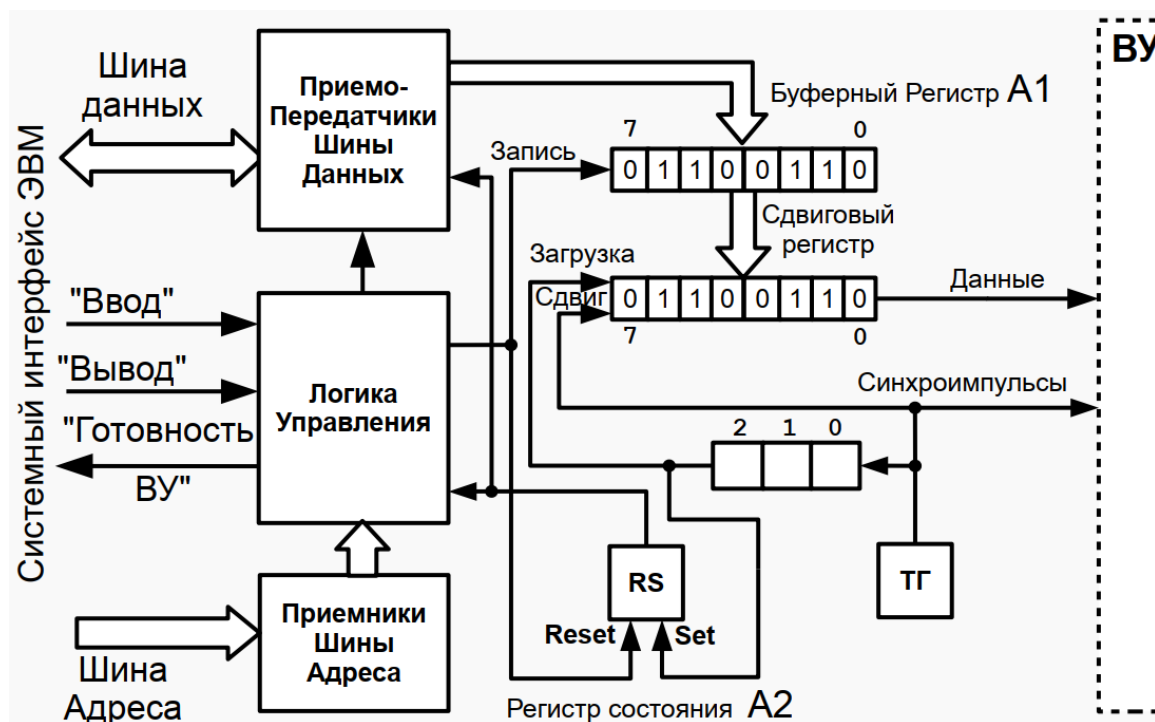
В основе реализации последовательного ввода-вывода лежит Т-триггер, который позволяет считать количество импульсов тактового генератора. Цепочка из Т-триггеров образует счётчик, нужный в схеме реализации последовательного ввода-вывода.



Чтобы организовать битовый сдвиг, без участия коммутатора, можно воспользоваться схемой, также состоящая из Т-триггеров. В схеме контроллера последовательного интерфейса есть сдвиговый регистр, позволяющий преобразовать параллельный в последовательный код.



## Контроллер передачи последовательного синхронного интерфейса

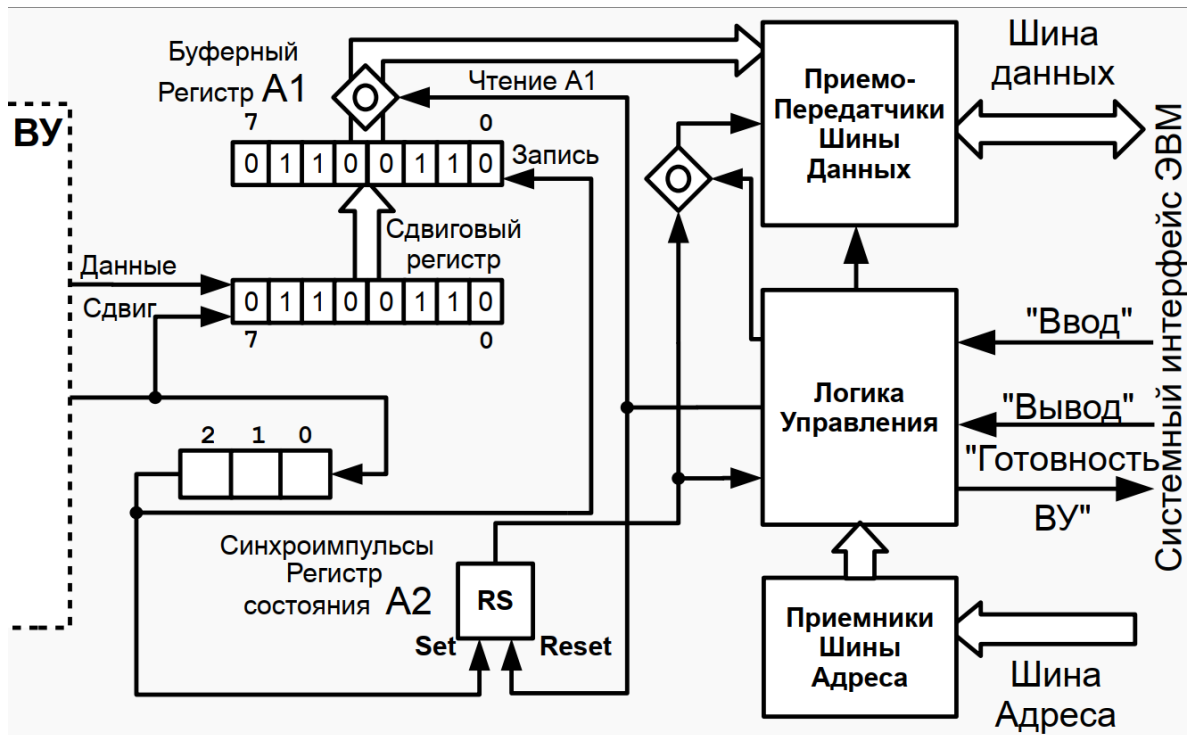


Под последовательным вводом-выводом подразумевается побитовая передача данных по одной шине. Это сокращается количество проводов, нужных для соединения контроллера с внешним устройством. Но при этом аппаратная часть усложнилась.

Когда идёт речь о синхронном интерфейсе, то всегда идёт речь о своём тактовом генераторе внутри КВУ. Он отправляет синхроимпульсы сдвиговому регистру для осуществления сдвига и внешнему устройству как сигнал подтверждения отправки данных.

Когда счётчик импульсов ТГ насчитает 8 импульсов, он отправит управляющий сигнал на загрузку нового значения в сдвиговый регистр из буферного регистра. К тому моменту ЭВМ должна подготовить новое значение для передачи данных, иначе последующая передача не будет осуществлена. Следующим тактовым импульсом счётчик будет сброшен до 0 и начнётся очередной цикл выдачи 8 бит на ВУ.

## Контроллер приёма синхронного последовательного интерфейса



Тактовый генератор располагается только на передающей стороне, то есть, внутри внешнего устройства. После 8 импульсов (сдвиговый регистр имеет все переданные биты), счётчик формирует сигнал записи в буферный регистр и одновременно с этим устанавливает готовность в регистр состояния.

Дальше ЭВМ уже может считывать данные, так как видит эту самую готовность. При чтении данных открывается вентиль чтения буферного регистра и данные идут в приёмо-передатчики шины данных, одновременно с этим снимается готовность регистра состояния.

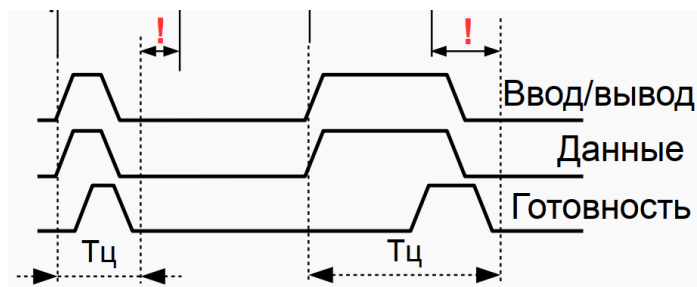
## Последовательный vs. параллельный обмен

### 1) Параллельный обмен:

- Быстрее при одинаковых условиях
- Дороже, так как требует больше аппаратных ресурсов
- Менее помехозащитный
- Дальность передачи меньше

### 2) Последовательный обмен – всё наоборот

### 37. Асинхронный обмен. Принцип деления частоты, формат кадра.



При реализации асинхронного обмена интервал между командами ввода-вывода данных задаётся самим внешним устройством. Контроллеры подобных устройств снабжаются регистрами состояния (Как будет дальше в схемах – RS-триггеры), которые информируют ЭВМ о готовности внешнего устройства к обмену данными.

Преимущества:

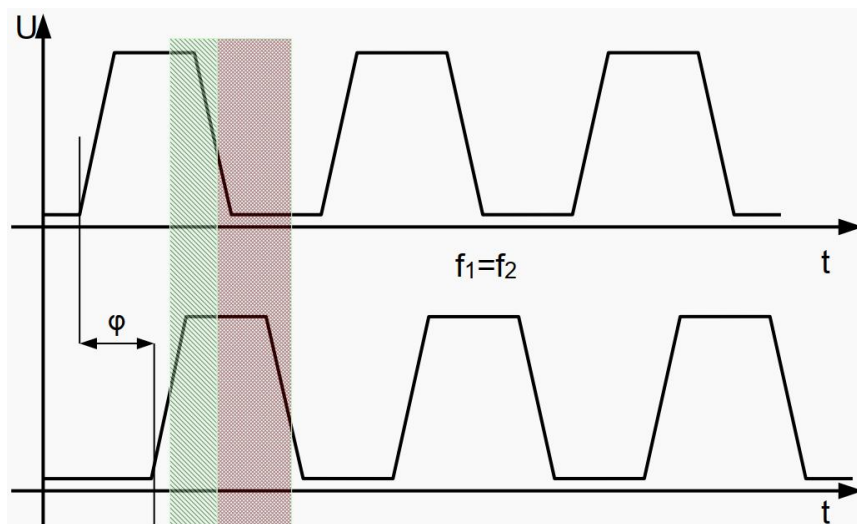
1. Не нужно знать точное время выполнения операции на ВУ.
2. ВУ всегда успеет выполнить обработку данных перед началом следующей операции.

Недостатки:

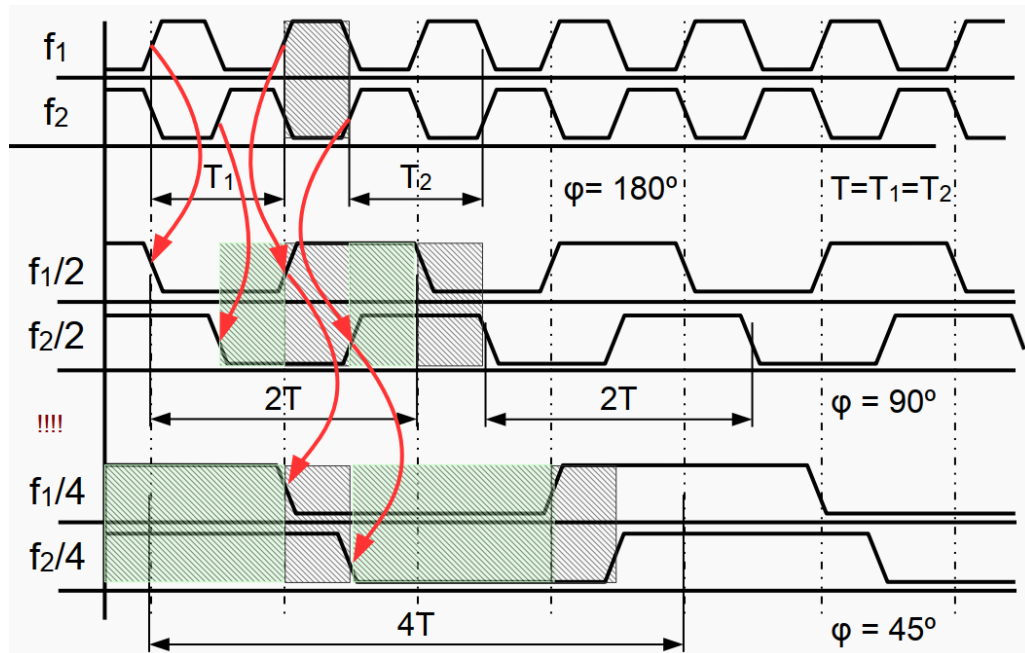
1. ЭВМ простаивает в ожидании готовности ВУ.

#### Принцип деления частоты

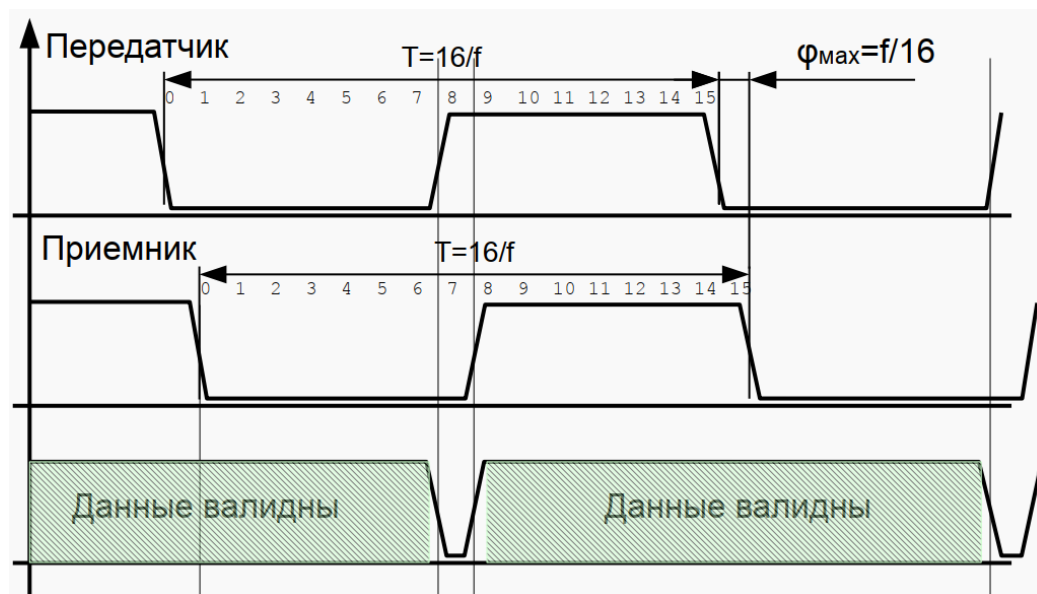
Люди решили, что нужно сократить накладные расходы в проведении кабеля между ВУ и КВУ. Шину данных и землю убрать нельзя, поэтому решили убрать шину с синхроимпульсом. Вследствие чего у КВУ и ВУ появились свои собственные тактовые генераторы. При асинхронной передаче данных рано или поздно может произойти рассинхронизация ТГ у передатчика и приёмника, что ведёт к искажению или потере данных.



Для решения данной проблемы решили делить тактовую частоту при помощи счётчика по модулю. Принцип основан на том, что длительность фазы увеличивается за счёт деления тактовой частоты и мы получаем больше промежутков тактового времени, когда фазы двух ТГ совпадают. Чем больше мы производим деление тактовой частоты, тем меньше будет ошибок, связанный с фазовым сдвигом.

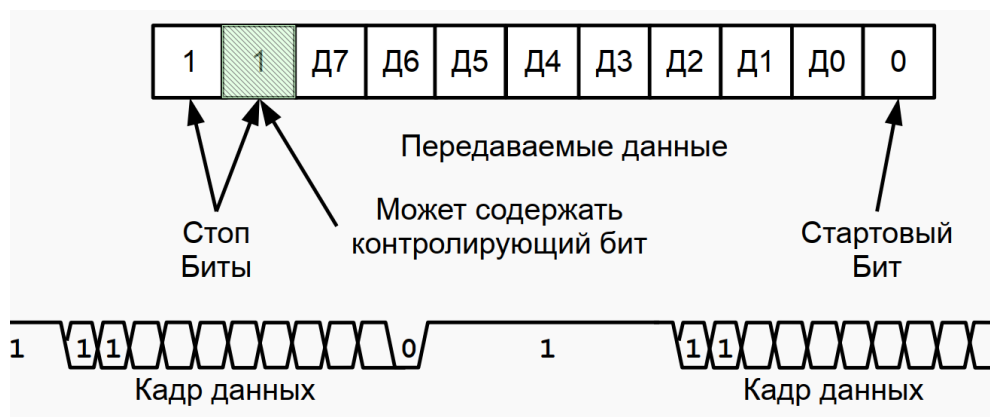


Причём скорость всей системы не будет проседать, так как можно изначально сильно повысить частоту тактового генератора (а её можно будет сделать сильно больше, чем максимальную частоту работы процессора), и потом просто её поделить.



Важно выбрать тот момент такта, когда можно осуществить передачу данных. Обычно выбирают момент времени, когда мы прошли какую-то часть валидной зоны. На картинке выше, например, это 14 или 15 момент времени.

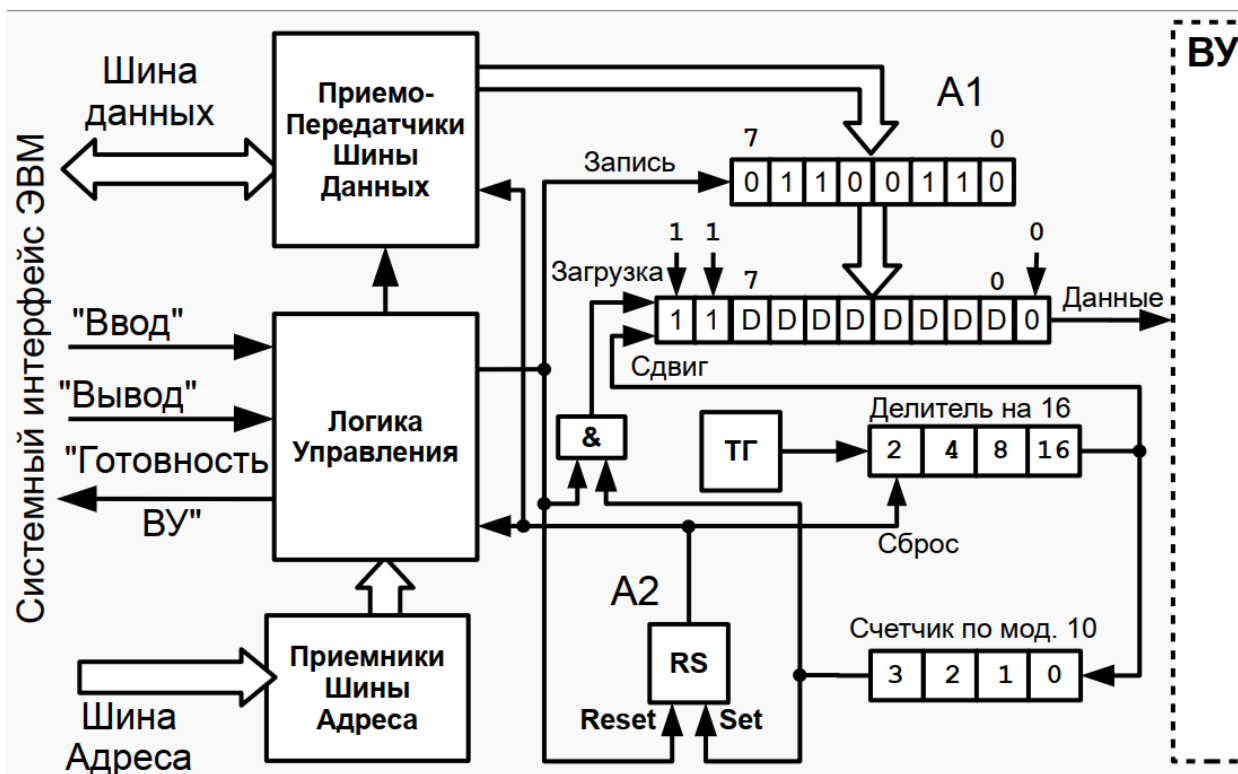
### Формат кадра



Кадр представляет из себя последовательность битов, которые мы передаём при асинхронном обмене. Кадр состоит из стартового бита, всегда равный нулю, передаваемые данные, бит контроля чётности и стоп-биты.

В момент времени, когда данные не передаются, сигнал равен 1. Он обрабатывается приёмником таким образом, что его собственный генератор блокируется. При появлении стартового бита, равного 0, ТГ разблокируется и начинается процесс приёма данных. Подробно рассмотрено в следующих 2 вопросах.

### 38. Контроллер передачи асинхронного последовательного интерфейса.



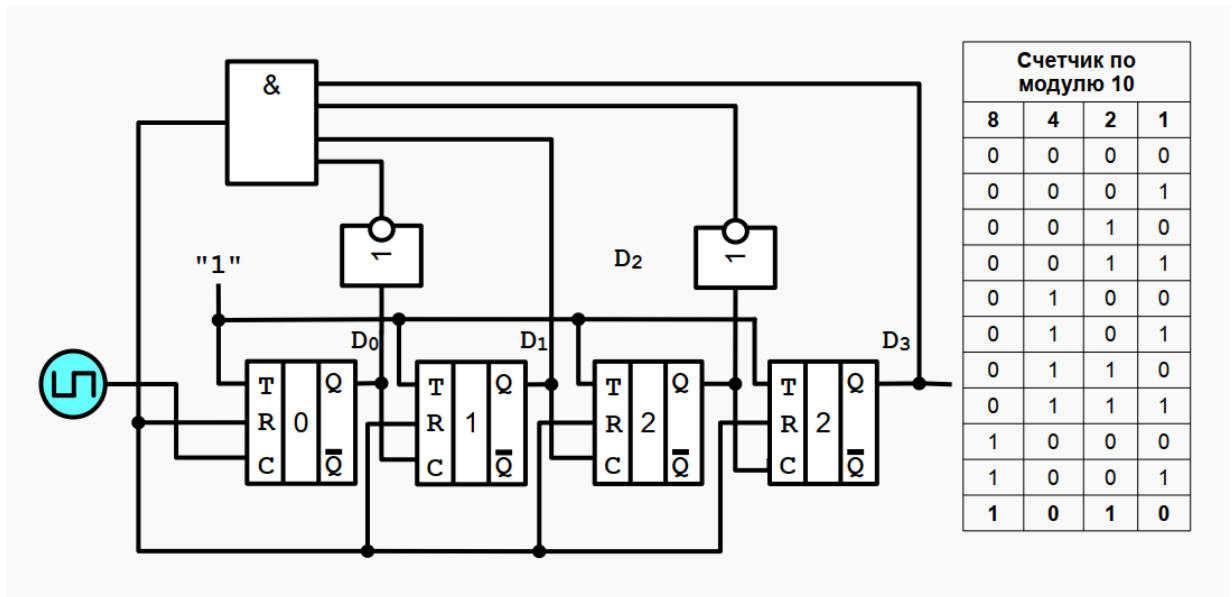
После передачи предыдущих байтов данных в регистр состояния A2 записывается 1, сигнализирующая о готовности к передаче новых данных. Этот же сигнал через шину “сброс” запрещает формирование импульсов сдвига в делителе частоты ТГ (mod 16). Счётчик по модулю 10 находится в нулевом состоянии, то есть на его выходе сигнал равен логической единице, которая поступает к схеме логического “И”.

Система находится в ожидании сигнала “Вывод” из логики управления, который сбросит значение регистра состояния, освободит ТГ и произведёт загрузку данных в буферный регистр A1 (можно это сделать заранее до сигнала готовности со стороны КВУ), и после в регистр сдвига.

Передаваемые биты записываются с 1 по 8 биты сдвигового регистра, в нулевой разряд помещается 0 как стартовый бит, а в 9,10 биты – константные значения стоп-битов.

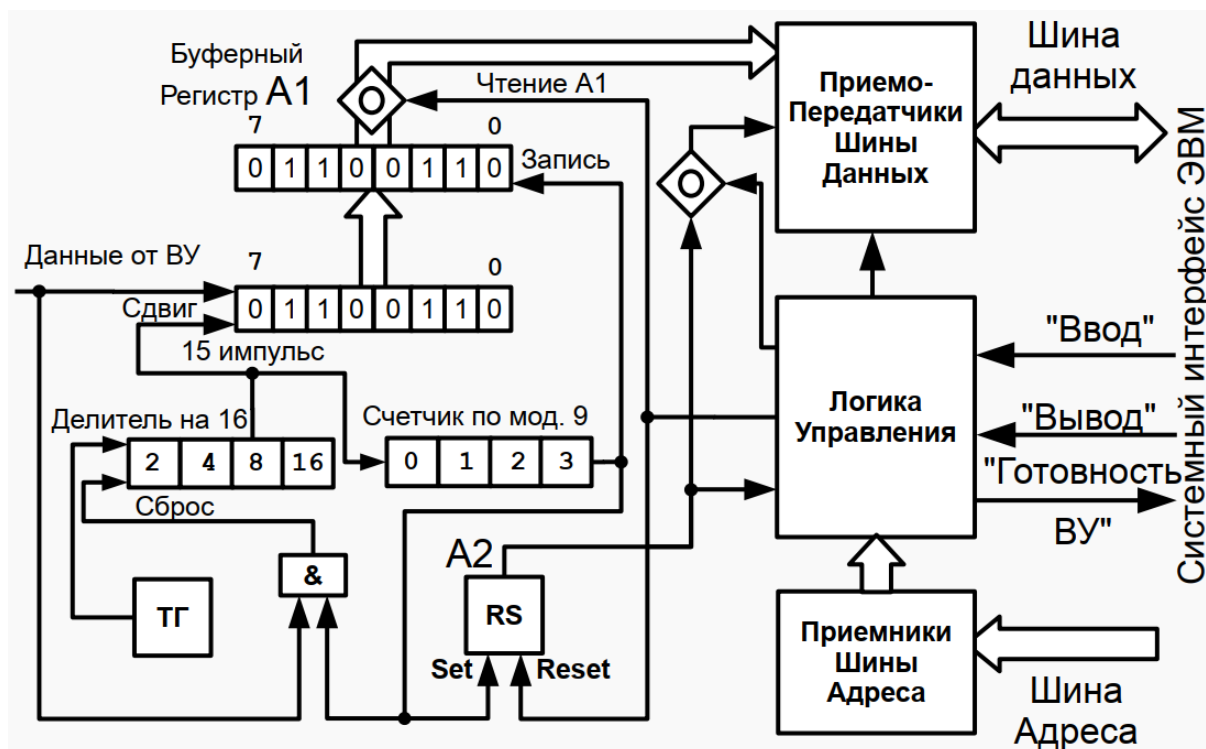
При начале передачи счётчик по модулю обнулится, и прежде пройдёт 10 тактов ТГ, пока не будет сброшено состояние вновь. По ходу этих тактов по шине передадутся все 11 битов. Дальше цикл обмена повторится.

## Счётчик по модулю 10.



Счётчик по модулю 10 реализован с помощью обычного счётчика, однако к нему добавлена схема логического “И”, проверяющая конкретную комбинацию значений. В нашем случае  $1010_2$ , что равно как раз 10. При появлении такой комбинации схема срабатывает и отправляется сигнал сброса состояния счётчика. Таким образом, можно реализовать любой счётчик по модулю.

### 39. Контроллера приёма асинхронного последовательного интерфейса.



Помним про формат кадра, что обычное состояние шины данных – это единица. С предыдущего шага сигнал с счётчика по модулю 9 равен единице. Оба вышеописанных сигнала поступают на схему логического “И”, которая уже подаёт управляющий сигнал “Сброс” на схему делителя частоты ТГ (по аналогии с предыдущим с контроллером).

Первый бит, который поступит со стороны ВУ – это стартовый бит, равный 0. Сигнал сброса сбрасывается (тавтология, знаю) и начинается процесс приёма данных.

Пройдёт 9 тактов (для записи 8 информационных битов и выноса стартового бита) и данные из сдвигового регистра попадут в буферный регистр А1. Одновременно с этим подаётся сигнал в регистр состояния, сигнализирующий о готовности принятия данных.

Блок логики управления видит сигнал готовности и на основе этого уже может считать данные с буферного регистра, что повлечёт за собой сброс регистра состояния, либо чтение регистра состояния.

После того, как пачка данных пришла, на шине данных со стороны ВУ будет снова единица и ТГ снова заблокируется.

#### *40. Организация прямого доступа к памяти. Контроллер ПДП (DMA)*

Прямой доступ к памяти (DMA – Direct Memory Access) – режим обмена данными между устройствами вычислительной машины, либо же между устройством и основной памятью, при котором процессор не участвует в обмене. Так как данные постоянно не пересылаются в процессор и обратно, скорость обмена выше, чем разобранные выше методы обмена.

Подобный обмен реализуется полностью аппаратно и реализуется специальным контроллером ПДП.

### Организация ПДП

Особенности ПДП:

- Возможность начальной загрузки программ в основную память ЭВМ из устройства ввода.
- Обеспечивает возможность использования в ЭВМ быстродействующих внешних запоминающих устройств (SSD например).

Для экономии ресурсов контроллер ПДП свои ресурсы, а подключается к шинам данных и шине адреса системного интерфейса ЭВМ. Из этого делаем вывод, что процессор и контроллер ПДП не могут одновременно использовать эти шины.

Проблема решается либо захватом цикла, либо блокировкой процесса.

### Захват цикла

Простой захват цикла – передача происходит в те машинные циклы, в которых процессор не обменивается данными с памятью. Пометка цикла выполняются либо при помощи специальных управляющих сигналов, указывающие циклы, во время которых процессор не использует память, либо при помощи специальных селектирующих схем внутри ПДП, что усложняет конструкцию процессора.

При таком организации обмена ПДП не снижает производительности машины, но обмен возможен только в случайный момент времени одиночными байтами или словами.

Захват с принудительным отключением процессора от шин системного интерфейса – для реализации такого режима ПДП системный интерфейс дополняется двумя линиями для передачи управляющих сигналов “Требование ПДП” (ТПДП) и “Предоставление ПДП” (ППДП).

Требование ПДП формируется контроллером ПДП. Получив данный сигнал, процессор приостанавливает выполнение очередной команды, не дожидаясь её завершения, выдаёт сигнал предоставления ПДП и отключается от системного интерфейса.

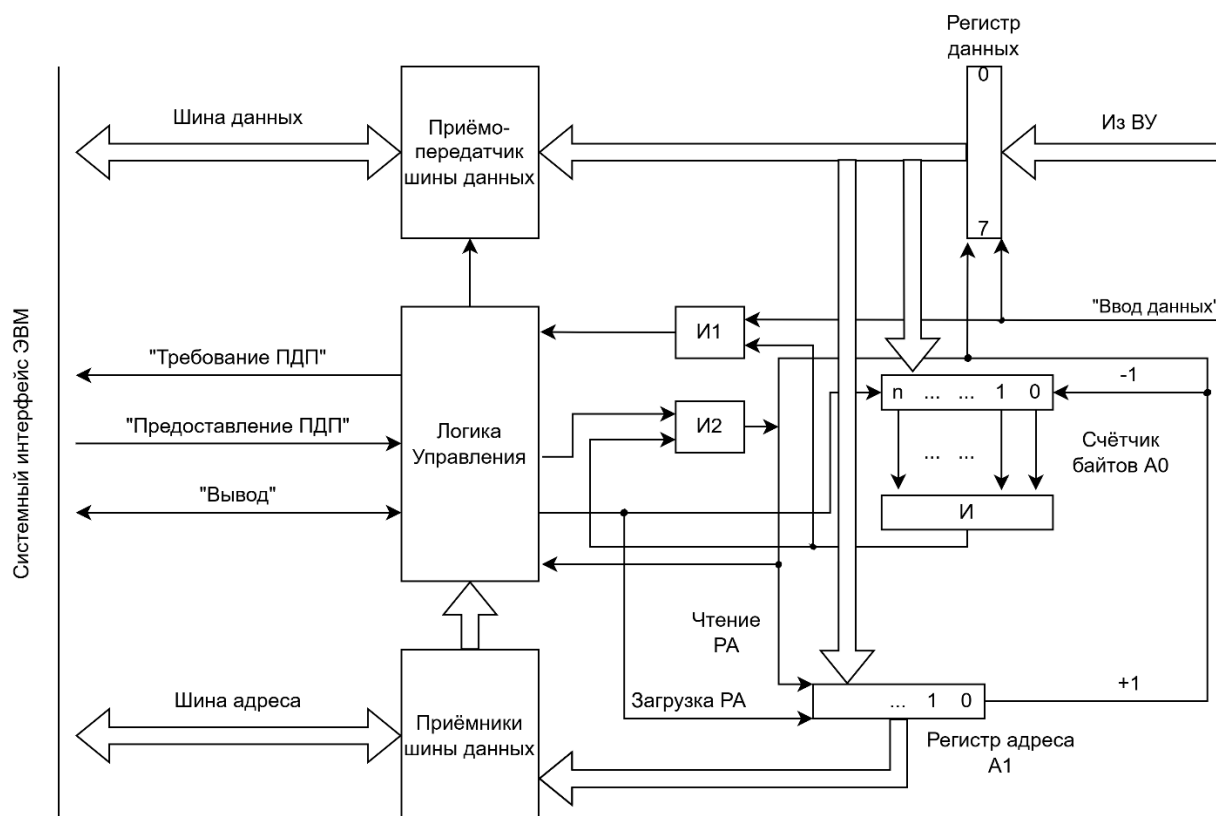
Контроллер ПДП получает доступ к шинам системного интерфейса и осуществляет обмен одним байтом или словом с памятью ЭВМ.

После контроллер ПДП возвращает управление процессору над системным интерфейсом.

### Блокировка процесса

Отличается от захвата цикла тем, что управление над системным интерфейсом передаётся контроллеру ПДП не на время обмена одним байтом, а на время обмена блоком данных.

### Контроллер ПДП ввода данных из ВУ в режиме “Захват Цикла”



1. Процессор загружает в счётчик байтов количество принимаемых байтов, а в регистр адреса помещается начальный адрес области памяти для вводимых данных.
2. Байты данных из ВУ поступают в регистр данных контроллера, при этом каждый байт сопровождается управляющим сигналом из ВУ “Ввод

данных”, который обеспечивает запись байта в регистр данных. По этому же сигналу при ненулевом состоянии счётчика байтов формируется сигнал “Требование ПДП”.

3. По ответному сигналу процессора “Предоставление ПДП” контроллер выставляет на шину адреса и шину данных содержимое регистра адреса A1 и регистра данных соответственно.
4. Формируя сигнал приказ “Выход” контроллер ПДП обеспечивает запись байта данных из своего регистра данных в память ЭВМ.
5. По этому же сигналу “Предоставление ПДП” счётчик байтов декрементируется, а содержимое регистра адреса инкрементируется. Как только счётчик байтов станет равным нулю, контроллер прекратит формирование сигналов “Требование ПДП”.